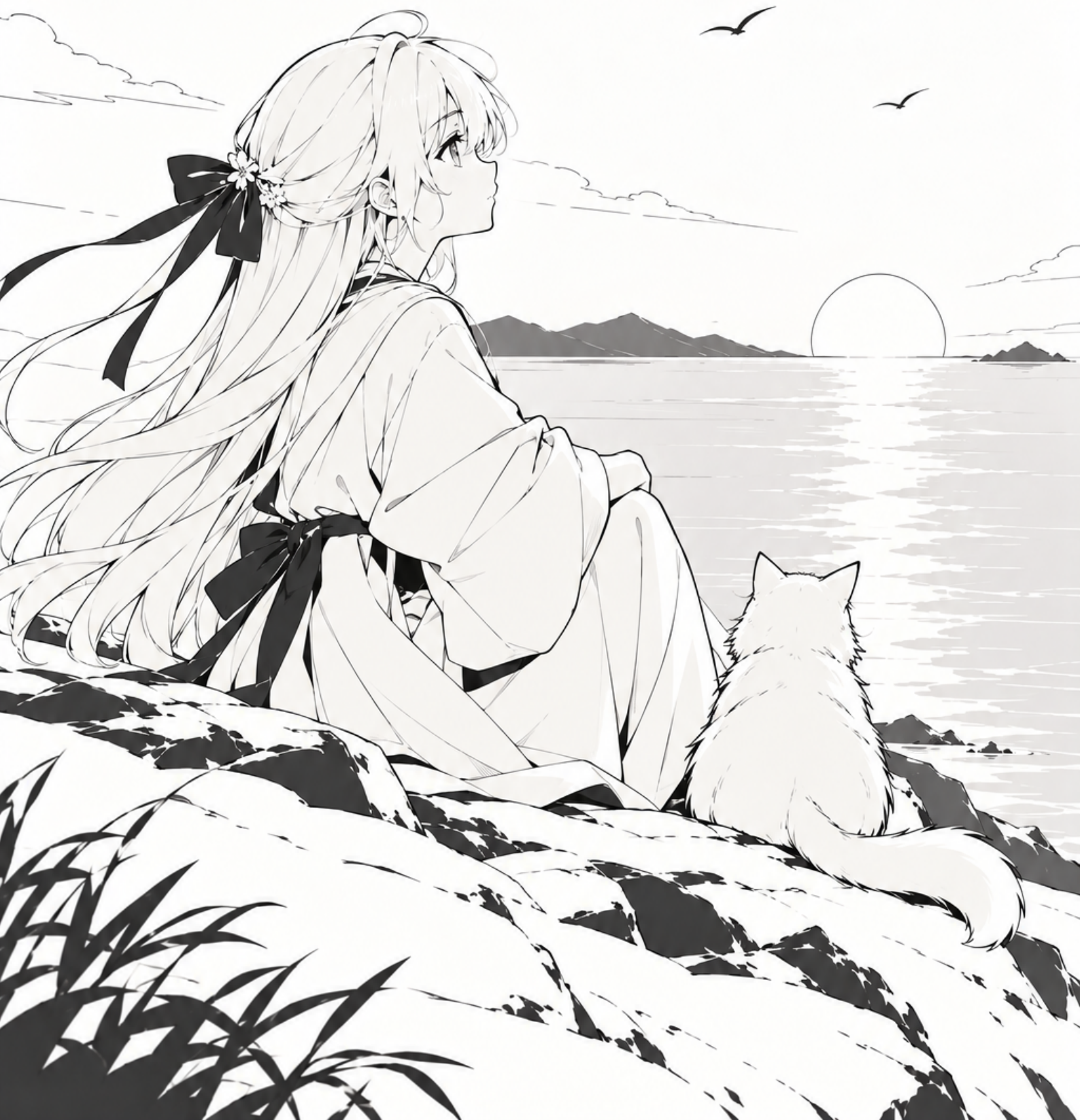


CCF CSP HANDBOOK

CCF CSP 考场手册

题型检索 C++ 模板 易错检查

朝汐 著



使用前说明

这本手册从自用资料整理而来，比较适合 C++ 基础薄弱、算法水平一般，想在 CCF CSP 中争取 300 分左右的同学。内容主要围绕前三题的常见做法，以及后面题目里可以尝试的部分分。

章节主要是为了方便查找。忘了语法，可以翻第四章；拿不准做法，可以看前面的题型索引；已经有思路，就直接找模板和易错点。同一段代码有时会在题型章和模板库各出现一次，是为了查阅时少翻几页。

常用模板最好先亲手写过几遍，弄清变量的含义和使用条件。上场后翻书，更多是提醒某个下标、初始化方式，或者一条容易漏掉的判断。题目换了条件，代码也要跟着改。

打印时可以给第二章、第九章和第十章贴上便签。模板编号右上角是正文页码，按页脚的数字翻即可。

朝夕

翻阅起点

按**页脚印刷页码**翻阅。封面无印刷页码，前置说明和目录使用罗马数字，正文从 1 开始；PDF 阅读器的物理页序号可能不同。

现在要找什么	正文页码
1 分钟定位题型	10
详细判断树与容易误判的分支	15
C++ 语法、输入输出、STL	33
第 1 题：扫描、公式、排序与边界	57
第 2 题：前缀、差分、二分与基础算法	86
第 3 题：对象、命令、解析与模拟	118
第 4/5 题：小数据与特殊性质	159
模板编号与完整模板	175
提交前易错检查	214
历年真题与模板映射	220

编号读法：[T101¹⁷⁷](#) 表示 T101 模板，右上角数字为该模板的正文起始页。电子版可直接点击。模板代码以 GNU C++17 为基线，使用前先看适用条件。

本书用 A/B/C/D/E 简称第 1/2/3/4/5 题；题号只用于安排时间，算法始终由题意、范围和操作决定。遇到熟悉的题型可以先做，不必被题号限制。

目录

使用前说明	i
翻阅起点	ii
第一部分 导航与判断	1
第一章 全书总览与使用路径	2
1.1 本手册的工作方式	2
1.2 全书结构路线图	2
1.3 考场总流程	3
1.4 数据范围与复杂度	4
1.5 A/B/C/D/E 总体策略	5
1.6 总决策树	6
1.7 关键词到章节索引	7
1.8 低级错误总检查	8
1.9 部分分总策略	8
1.10 个人复习标记	9
第二章 考场 1 分钟极速索引	10
2.1 这一章怎样查	10
2.2 1 分钟总流程	10
2.3 题号策略速查	10
2.4 题面主体到模板	11
2.5 数据范围秒判	11
2.6 B 题优化入口	11
2.7 C 题建模速查	12
2.8 D/E 部分分入口	12
2.9 模板编号速查	12
2.10 C++ 语法救急	13
2.11 最后提交前 30 秒	14
第三章 题型详细判断树	15
3.1 这一章怎样查	15
3.2 考场读题时先看什么	15

3.3	总判断流程	15
3.4	各表怎样查	16
3.5	从根节点开始的总决策树	16
3.6	常见问题与模板	17
3.7	题面结构第一判断	18
3.8	数据范围判断树	18
3.9	关键词到模板总索引	19
3.10	操作模式判断树	20
3.11	A 题快速判断树	21
3.12	B 题快速判断树	22
3.13	C 题快速判断树	23
3.14	逐句读题分支树	25
3.15	B 题专项分支树	26
3.16	C 题专项分支树	27
3.17	D/E 题快速判断树	28
3.18	D/E 部分分决策树	28
3.19	容易误判的分支	29
3.20	无法判断时的暂时没有思路时	30
3.21	判断树反向测试表	30
3.22	最后 5 分钟判断	30

第二部分 基础与题型 32

第四章 C++ 语法与 STL 速查 33

4.1	本章使用方式	33
4.2	语言标准与代码拼装	33
4.3	最小程序骨架	33
4.4	输入输出	34
4.5	数据类型	36
4.6	表达式、整除和取整	37
4.7	控制流	40
4.8	auto、引用和范围 for	41
4.9	函数	42
4.10	数组	43
4.11	vector	44
4.12	string	46
4.13	pair 和 tuple	47
4.14	结构体 struct	47
4.15	排序 sort	48
4.16	二分相关函数	49
4.17	map 和 unordered_map	50
4.18	unordered_set	51

4.19 set 和 multiset	51
4.20 queue、stack、deque	52
4.21 priority_queue	53
4.22 常用算法函数	53
4.23 lambda 入门	55
4.24 常见编译错误和原因	55
4.25 C 和 C++ 混用提醒	56
4.26 考场 C++ 快速检查	56
第五章 A 题决策树与高频模板	57
5.1 A 题定位	57
5.2 A 题开题流程	57
5.3 A 题读题标注法	57
5.4 A 题总决策树	58
5.5 A 题关键词到动作	59
5.6 A 题数据范围判断	60
5.7 A 题常见输入形态	60
5.8 简单统计	61
5.9 存在、全部、任意判断	64
5.10 公式题	65
5.11 排序和排名	68
5.12 字符串处理	70
5.13 进制处理	73
5.14 日期时间	75
5.15 简单模拟	76
5.16 坐标和距离入门	77
5.17 二维表格和矩阵入门	78
5.18 频率统计	79
5.19 区间入门	80
5.20 A 题输出格式专项	80
5.21 A 题新手错误对照表	81
5.22 A 题推荐代码组织	81
5.23 A 题自造边界样例	82
5.24 A 题本地调试流程	83
5.25 A 题常见边界库	84
5.26 A 题模板练习清单	84
5.27 A 题实现顺序	84
5.28 A 题提交前总检查	85
5.29 A 题卡住时怎么办	85
第六章 B 题决策树与高频算法入口	86
6.1 B 题定位	86
6.2 B 题开题流程	86

6.3	B 题总决策树	86
6.4	B 题数据范围判断	87
6.5	B 题关键词到动作	87
6.6	从暴力到优化的转换表	88
6.7	前缀和	88
6.8	前后缀预处理	90
6.9	差分	91
6.10	哈希统计	93
6.11	余数和同余统计	94
6.12	排序加扫描	95
6.13	堆和动态维护	97
6.14	基础贪心	99
6.15	单调栈和单调队列入门	100
6.16	双指针入门	101
6.17	二分与二分答案	102
6.18	BFS 入门	103
6.19	DFS 和连通块	105
6.20	Floyd 入门	106
6.21	Dijkstra 入门	107
6.22	并查集入门	108
6.23	拓扑排序入门	109
6.24	简单 DP 入门	111
6.25	基础数学入门	112
6.26	并查集调用示例	113
6.27	B 题提交前检查	114
6.28	B 题优化速查	114
6.29	B 题常见边界库	115
6.30	B 题本地调试流程	115
6.31	B 题卡住时的部分分策略	116
6.32	B 题模板练习清单	116
6.33	矩阵与稀疏数据补充入门	117
第七章	C 题决策树与大模拟方法	118
7.1	C 题定位	118
7.2	C 题开题流程	118
7.3	C 题读题三遍法	118
7.4	从题面到代码的转换	119
7.5	状态维护和容器选择	120
7.6	C 题总决策树	121
7.7	C 题常见题型	122
7.8	C 题细分决策树	122
7.9	大模拟总方法	124

7.10 长题面拆解表	125
7.11 命令解析	126
7.12 状态设计	128
7.13 状态机模拟	129
7.14 事件模拟	130
7.15 优先级模拟	131
7.16 复杂排序和排名	132
7.17 表格和矩阵模拟	133
7.18 目录树和路径模拟	134
7.19 URL 路由和参数解析	135
7.20 配置和 JSON 类解析入口	137
7.21 字符串解析	138
7.22 表达式解析入口	140
7.23 缓存和队列模拟	141
7.24 懒删除优先队列	143
7.25 时间和日程模拟	144
7.26 树形结构模拟	145
7.27 权限和规则匹配	146
7.28 网格和图混合题	148
7.29 大模拟易错点	149
7.30 C 题分阶段实现	149
7.31 拆分函数时的检查	150
7.32 分层测试方法	151
7.33 C 题常见失分原因	152
7.34 C 题本地调试	152
7.35 C 题常见边界库	153
7.36 C 题部分分策略	154
7.37 按子任务逐步完成	154
7.38 来不及写完时	154
7.39 不同题型的实现顺序	155
7.40 C 题模板索引	156
7.41 C 题考场速查页	156
7.42 C 题提交前检查	157
7.43 C 题模板练习清单	158
第八章 D/E 题部分分策略与高级算法入口	159
8.1 D/E 题定位	159
8.2 D/E 题读题流程	159
8.3 部分分决策树	159
8.4 暴力模板	160
8.5 小数据常用算法	161
8.6 特殊性质策略	161

8.7 按题型拿部分分	162
8.8 树状数组入口	163
8.9 线段树入口	164
8.10 Kruskal 最小生成树入口	165
8.11 树上问题入口	166
8.12 状态压缩 DP 入口	168
8.13 KMP 字符串匹配入口	169
8.14 Trie 字典树入口	170
8.15 D/E 考场时间策略	171
8.16 常见高级算法入口	171
8.17 高级算法识别速查	172
8.18 D/E 模板练习建议	172
8.19 完整做法暂时不会时	173
8.20 D/E 题提交前检查	173

第三部分 模板与检查 174

第九章 高频算法模板库 175

9.1 模板库使用方法	175
9.2 模板速查表	175
9.3 T101 一维前缀和	177
9.4 T102 二维前缀和	177
9.5 T103 一维差分	178
9.6 T104 二维差分	179
9.7 T105 排序自定义比较器	180
9.8 T106 哈希统计	180
9.9 T107 二分答案	181
9.10 T108 双指针	181
9.11 T109 坐标离散化	182
9.12 T110 事件排序	183
9.13 T111 矩阵重塑与乘法顺序	184
9.14 T112 稀疏向量点积	184
9.15 T113 有限状态映射与倍增	185
9.16 T201 网格 BFS	185
9.17 T202 DFS 连通块	186
9.18 T203 并查集	187
9.19 T204 Dijkstra	188
9.20 T205 拓扑排序	189
9.21 T206 Floyd	190
9.22 T207 Kruskal	190
9.23 T208 LCA 倍增	191
9.24 T209 无递归树遍历与子树区间	192

9.25	T301 01 背包	193
9.26	T302 完全背包	193
9.27	T303 恰好装满与至少达到	194
9.28	T401 树状数组	195
9.29	T402 线段树	195
9.30	T403 单调栈	197
9.31	T404 单调队列	197
9.32	T405 静态区间最大值	198
9.33	T501 split 字符串分割	198
9.34	T502 KMP	199
9.35	T503 Trie	200
9.36	T510 整除取整和标准取模	201
9.37	T511 快速幂	202
9.38	T512 素数和约数	202
9.39	T513 进制转换	203
9.40	T514 日期处理	204
9.41	T515 质因数分解与筛素数	204
9.42	T601 大模拟骨架	205
9.43	T602 命令分发	206
9.44	T603 状态机	207
9.45	T604 懒删除优先队列	207
9.46	T605 LRU 缓存	209
9.47	T606 路径规范化	209
9.48	T607 URL 参数解析	210
9.49	T608 括号匹配	211
9.50	T609 简单表达式求值	211
9.51	T610 整数四则表达式解析	212
第十章	易错点总表	214
10.1	本章使用方法	214
10.2	输入输出易错点	214
10.3	数据类型和溢出	214
10.4	整除、取整和取模	215
10.5	下标和边界	215
10.6	初始化和清空	215
10.7	排序和比较器	215
10.8	STL 容器易错点	216
10.9	字符串易错点	216
10.10	图论易错点	217
10.11	动态规划易错点	217
10.12	数据结构易错点	217
10.13	提交前总检查	217

10.14 最后十分钟清单	218
-------------------------	-----

第四部分 真题练习	219
------------------	------------

第十一章 历年真题索引与模板映射	220
-------------------------	------------

11.1 怎样查题目	220
11.2 按场次查找	220
11.3 202409 场次速查	220
11.4 202412 场次速查	221
11.5 202503 场次速查	221
11.6 202506 场次速查	222
11.7 202509 场次速查	222
11.8 202512 场次速查	223
11.9 202603 场次速查	223
11.10 202605 场次速查	224
11.11 历年整场索引 (201912–202406)	224
11.12 A 题索引	225
11.13 B 题索引	226
11.14 C 题索引: 大模拟与中档算法入口	227
11.15 D/E 题索引与小数据做法	228
11.16 题目反查模板索引	229
11.17 按题面信号反查真题	229
11.18 练习后怎样复盘	230
11.19 A/B 题型归纳	231

第一部分

导航与判断

第一章 全书总览与使用路径

1.1 本手册的工作方式

翻书时可以从题目正在问什么入手。比如，多次求一段数的和，就先查前缀和；不断合并集合，再判断是否连通，就查并查集。前面的索引按这些常见问法整理，后面再给出写法和限制。

读完题，可以先记下这些信息：

1. 题面中出现了什么信号？
2. 数据范围允许什么复杂度？
3. 它更像 A、B、C、D、E 中哪类题？
4. 先尝试哪个模板？
5. 需要使用哪些 C++ 语法或 STL 容器？
6. 最容易错在哪里？
7. 如果满分做法不会，能否先写部分分？

常用模板最好考前写过几遍。考场上翻书时，能看懂每个变量的含义，比抄下一段不熟悉的代码更

有用。

1.2 全书结构路线图

全书分为四部分。想找做法时看索引，想看具体写法时翻题型章节或模板库；检查代码和复盘练习时，再看后两部分。

分部	章节	用法
导航与判断	第 1-3 章	先看全书用法，再用 1 分钟极速索引定位，必要时进入详细判断树。
基础与题型	第 4-8 章	先补 C++ 和 STL 基础，再按 A/B/C/D/E 题型查看策略和高频入口。
模板与检查 真题练习	第 9-10 章 第十一章	找可复制模板，并在提交前扫易错点。 用历年题反查模板和判断树，发现误判后回到第三章修正。

推荐阅读路线：

1. **考前通读**：第一章、第四章、第 5–8 章、第十章。
2. **刷题复盘**：第三章判断路径、第九章模板、第十一章真题索引。
3. **考场翻查**：第二章极速索引；定位不准时翻第三章；写代码时翻第九章；提交前翻第十章。

1.3 考场总流程

1.3.1 开场 10 分钟

开场可以花几分钟浏览五道题，看看哪些熟悉，哪些有能做的子任务，再决定先做哪道。

1. 看每道题的题面长度。题面短不一定简单，题面长往往意味着模拟或细节多。
2. 看每道题的数据范围。数据范围比题面故事更重要。
3. 判断 A、B 是否可以快速完成。
4. 判断 C 是算法题还是大模拟。
5. 判断 D、E 是否有子任务、小数据、特殊性质或熟悉模板。
6. 在草稿纸上写下计划：先做哪题、卡住多久换题、最后回来看哪题。

1.3.2 每题固定流程

没有思路时，可以先检查下面几项：

1. 读题面，圈出关键词。
2. 记下影响复杂度的几个范围。
3. 估算可接受复杂度。
4. 查本章总决策树，定位章节。
5. 写出暴力思路，用来理解题意和造样例。
6. 如果满分模板明确，再写正式代码。
7. 先过样例，再造边界样例。
8. 删除调试输出后提交。

写双重循环前先算次数。 $n = 10^5$ 时， $O(n^2)$ 可能需要百亿次操作，即使样例很快也不合适。

1.3.3 建议时间分配

题号	建议时间	目标	卡住时的处理
A	20–35 分钟	稳定满分	15 分钟没思路就重新读题；30 分钟没过样例先跳。
B	40–70 分钟	尽量满分	优先查前缀和、差分、排序、哈希、BFS。

题号	建议时间	目标	卡住时的处理
C	60–100 分钟	高分或满分	如果是大模拟，先拆结构体和函数，不要全写在主函数。
D	40–80 分钟	部分分到满分	先找小数据和熟悉模板。不会满分也要写暴力。
E	剩余时间	尽量拿分	重点看子任务、特殊性质和可证明的简化。

表中的时间只是参考，各行的上限不能同时用满。例如，浏览 10 分钟，A 题 30 分钟、B 题 50 分钟、C 题 80 分钟、D/E 合计 50 分钟，最后留 20 分钟检查，总共 240 分钟。卡住时可以先换题。

1.4 数据范围与复杂度

1.4.1 复杂度速查表

输入规模	通常可考虑	常见模板
$n \leq 20$	子集枚举、状态压缩，核算转移次数	2^{20} 约百万；全排列仅宜 $n \leq 9$ 或 10
$n \leq 100$	$O(n^3)$ 通常可试	Floyd、区间 DP、三重枚举
$n \leq 500$	$O(n^3)$ 可能可行，需看时限	Floyd、部分 DP
$n \leq 1000$	$O(n^2)$ 通常可行	双重循环、二维 DP、朴素图算法
$n \leq 5000$	$O(n^2)$ 需谨慎	优化前的 DP、枚举配合剪枝
$n \leq 10^5$	$O(n \log n)$ 或 $O(n)$	排序、二分、前缀和、差分、堆、树状数组
$n \leq 10^6$	$O(n)$ 或较轻的 $O(n \log n)$	线性扫描、计数、前缀和
$n \geq 10^7$	通常只能线性或公式	数学、简单扫描、避免大对象数组

1.4.2 复杂度校验规则

- 看到双重循环，先计算最坏次数。
- 有多组数据时，总复杂度要乘以组数，或者看所有 n 的总和限制。
- 排序一般是 $O(n \log n)$ ， $n = 10^5$ 常见可行。
- Floyd 是 $O(n^3)$ ，只适合点数较小的图。
- Dijkstra 配合堆是 $O((n + m) \log n)$ ，适合正权稀疏图。
- BFS 适合边权全为 1 或每步代价相同的最短步数。
- 前缀和适合不修改的区间查询。

- 差分适合批量区间修改后统一查询。
- 树状数组和线段树适合修改与查询交织出现。

溢出提醒：只要出现求和、乘法、路径长度、方案数、时间累计，默认先考虑 `long long`。两个 `int` 相乘时写成 `1LL * a * b`。

1.5 A/B/C/D/E 总体策略

1.5.1 A 题

A 题通常考察基本实现能力。它可能是简单模拟、统计、字符串、日期、进制、排序或公式题。

- 目标：尽量一次 AC。
- 优先检查：输入格式、输出格式、整数溢出、小数精度、边界值。
- 常用章节：C++ 基础、数组统计、字符串、进制、日期、排序。
- 不建议：为了追求代码短而写不熟悉的技巧。

1.5.2 B 题

B 题常见的问题是暴力能算出来，但次数太多，需要想办法减少重复计算。

- 高频方向：排序、前缀和、差分、哈希、二分、BFS、简单图、简单 DP。
- 读题重点：是否有多次询问，是否有区间，是否需要快速统计。
- 常见错误：用暴力过样例但过不了大数据。

1.5.3 C 题

C 题既可能考长题面模拟，也可能要求图论、DP 等算法。先看操作和范围，再决定做法。

- 如果题面很长，先判断是否是规则模拟。
- 如果输入是命令、事件、状态变化，优先设计结构体和函数。
- 如果出现图、网格、路径、依赖关系，再进入对应算法分支。
- 大模拟题不要把所有逻辑塞进 `main`。

1.5.4 D/E 题

D、E 题要同时考虑满分算法和部分分。

- 先看是否给出子任务或特殊数据范围。
- 先写能过小数据的暴力，避免空题。
- 若满分算法明显且练过，再冲满分。
- 若 20 分钟没有形成可写方案，转为部分分策略。

1.6 总决策树

1.6.1 第一层：看题面信号

题面信号	优先怀疑	先翻章节
字符、单词、路径、命令、表达式、括号	字符串处理或解析模拟	字符串专题、栈、大模拟
二进制、十六进制、位、补码、编码	进制和位运算	进制专题、位运算
日期、时间、星期、天数、日历	日期时间处理	日期专题
区间和、多次查询、子矩阵和	前缀和	前缀和模板
区间加、范围修改、最后输出	差分	差分模板
修改和查询交替出现	树状数组或线段树	数据结构模板
最大值、最小值动态维护	堆、集合、单调结构	优先队列、set、单调队列
排序、排名、优先级、相同条件按某字段	自定义排序	sort 与 lambda
去重、出现次数、频率统计	哈希或 map	map、unordered map、计数数组
最少步数、迷宫、网格移动	BFS	BFS、网格 BFS
非负权最短路、道路费用、距离最小	Dijkstra	最短路模板
任意两点最短路且点数小	Floyd	Floyd 模板
合并集合、是否连通、同一类	并查集	并查集模板
依赖、先后顺序、任务调度	拓扑排序	拓扑排序模板
最大化最小值、最小化最大值、答案可行性	二分答案	二分答案模板
容量、价值、选择若干个、最多或恰好	背包 DP	动态规划模板
规则很多、对象很多、按时间变化	大模拟	大模拟专题

1.6.2 第二层：用数据范围排除错误方向

如果你想写	但数据范围是	应改查
枚举所有点对 $O(n^2)$	$n = 10^5$	排序、哈希、前缀和、树状数组、双指针
枚举所有区间 $O(n^2)$	$n = 10^5$	前缀和、双指针、单调队列、DP 优化
Floyd $O(n^3)$	$n > 500$ 或边很多	Dijkstra、BFS、图的特殊性质
每次查询都扫描数组	查询数 $q = 10^5$	前缀和、树状数组、线段树、离线处理
递归搜索所有方案	$n > 25$ 且无剪枝	DP、贪心、数学、二分答案
字符串反复插入删除	字符串长度和操作数很大	分块、链表思想、栈、模拟优化

1.6.3 第三层：选择满分或部分分

1. 如果模板明确、数据范围匹配、自己练过，写满分做法。
2. 如果模板大致明确但细节多，先写能跑的小版本，再逐步补全。
3. 如果满分算法不明确，找子任务和小数据，写暴力或特殊情况。
4. 如果题目很难但 A/B/C 还没稳，不要长时间停留在 D/E。

1.7 关键词到章节索引

模板编号的上标是正文页码；先由本表确定方向，再查第二章或模板库。

关键词	翻到哪里	注意事项
字符串、字符、大小写	字符串专题	注意 <code>cin</code> 和 <code>getline</code> 混用。
空格分割、命令行	字符串分割	用 <code>stringstream</code> 或手写分割。
括号、嵌套、表达式	栈、表达式解析	注意优先级和匹配失败。
进制转换	进制专题	判断是否超过 <code>long long</code> 。
二进制位	位运算	大位数用 <code>1LL << k</code> 。
日期、星期	日期专题	先确认闰年和月份天数。
静态区间和	T101¹⁷⁷ / T102¹⁷⁷	区间最值不能用两个前缀最值相减。
区间修改	差分	修改后统一查询时优先差分。
在线修改查询	树状数组、线段树	看是单点还是区间。
排序、排名	<code>sort</code> 与 <code>lambda</code>	相同条件要写完整。
频率、出现次数	<code>map</code> 、 <code>unordered map</code> 、计数数组	值域小用数组更快。
最短步数	BFS	边权相同才用 BFS。
非负权最短路	Dijkstra	距离用 <code>long long</code> 。

关键词	翻到哪里	注意事项
连通、合并	并查集	路径压缩和按大小合并。
依赖、先后	拓扑排序	入度为 0 入队。
最优选择	DP 或贪心	先看是否有状态转移。
答案范围	二分答案	必须能写 <code>check</code> 。
大量规则	大模拟	结构体、函数、状态表。

1.8 低级错误总检查

提交前至少检查以下内容：

- 是否删除所有调试输出。
- 是否使用标准输入输出。
- 输出末尾是否多了不该有的内容。
- 是否把答案变量开成 `long long`。
- 数组是否开到最大范围。
- 下标是从 0 还是从 1 开始。
- 循环边界是否包含末尾。
- 多组数据时是否清空数组、图、队列、`map`。
- 比较器是否满足严格弱序。
- `lower_bound` 或 `find` 的返回值是否检查。
- `priority_queue` 是否需要小根堆。
- BFS 是否在入队时标记访问。
- Dijkstra 是否跳过过期状态。
- 递归深度是否可能爆栈。

1.9 部分分总策略

当满分做法暂时不会时，按这个顺序找分：

1. **小数据暴力**：如果 $n \leq 1000$ 的测试点存在，写 $O(n^2)$ ；如果 $n \leq 100$ ，考虑 $O(n^3)$ 。
2. **特殊结构**：图是树、链、完全图、DAG、无权图时，可能有更简单做法。
3. **特殊参数**：如果某个参数为 0、1、很小，单独写分支。
4. **无修改版本**：有修改和查询时，若部分数据没有修改，先用前缀和或静态算法。
5. **少量查询或少量修改**：查询少就每次暴力；修改少就重算或离线处理。
6. **样例和明显边界**：最后时间不足时，也要保证样例和简单边界正确。

部分分代码也要干净。不要写一堆互相冲突的特判。特判越多，越容易把能拿的分弄丢。

1.10 个人复习标记

打印后可用便签标出常用章节，并按练习情况标记：

- 标记第二章、第九章和第十章的起始页。
- 在练熟的模板编号旁打勾；不熟的模板先用小样例练习。
- 将真题索引中的每道题反向连接到本章的某个判断路径。
- 将个人错题中的错误补入“低级错误总检查”。

第二章 考场 1 分钟极速索引

2.1 这一章怎样查

赶时间时可以先翻本章，按题目中的操作查模板编号。需要进一步判断时看第三章，具体代码在第九章，练习题索引在第十一章。

表中的关键词只是线索，还要看数组会不会修改、边权有什么限制、数据有多大。

2.2 1 分钟总流程

时间	做什么	目的
前 10 秒	看题号：A/B/C/D/E	决定稳分、冲分、拆题、部分分。
第 10-25 秒	圈出输入主体：数组、矩阵、字符串、图、树、对象、事件	确定大类。
第 25-40 秒	圈出操作：查询、修改、排序、最优、路径、状态变化	确定模板方向。
第 40-55 秒	看数据范围： n, m, q 、值域、字符串总长	排除不可能复杂度。
最后 5 秒	翻模板并写骨架	不在脑子里硬背。

2.3 题号策略速查

题号	目标	常见题型	第一动作
A	争取满分	统计、公式取整、排序、字符串、日期、二维数组	先写暴力扫描，检查输出格式。
B	尽量满分	前缀、差分、哈希、排序扫描、二分、简单 DP	先写暴力，再找重复计算。
C	尽量拿高分	大模拟、字符串解析、状态机、事件、树/目录	先建对象和状态，不急着手写代码。
D	能拿部分分就拿	图、树、DP、数据结构、数学	先看子任务和特殊性质。
E	尝试明确的子任务	高级算法或综合题	写小数据暴力，不耗尽时间。

2.4 题面主体到模板

主体	题面信号	模板/章节
数组	求和、计数、相邻、分段、排名、分组	A 题、T101 ¹⁷⁷ 、T105 ¹⁸⁰ 、T106 ¹⁸⁰ 、T510 ²⁰¹
矩阵	图像、矩形、邻域、旋转、重塑	T102 ¹⁷⁷ 、T104 ¹⁷⁹ 、矩阵扫描
字符串	分隔符、路径、URL、括号、表达式	T501 ¹⁹⁸ 、T606 ²⁰⁹ 、T607 ²¹⁰ 、T608 ²¹¹ 、T609 ²¹¹
坐标点图	点集、距离、矩形、覆盖、邻居 最短路、连通、依赖、最小代价	T106 ¹⁸⁰ 、T109 ¹⁸² 、坐标公式 T201 ¹⁸⁵ 、T203 ¹⁸⁷ 、T204 ¹⁸⁸ 、T205 ¹⁸⁹ 、T207 ¹⁹⁰
树	子树、祖先、路径、目录、合并	DFS、T208 ¹⁹¹ 、T402 ¹⁹⁵
多命令	create、delete、query、request、release	T601 ²⁰⁵ 、T602 ²⁰⁶ 、T603 ²⁰⁷
时间事件	到期、归还、租约、按时刻发生	T110 ¹⁸³ 、T514 ²⁰⁴ 、T603 ²⁰⁷
选择物品	容量、费用、价值、至少达到	T301 ¹⁹³ 、T302 ¹⁹³
最优答案	最小时间、最大阈值、最小最大	T107 ¹⁸¹ 或 DP

2.5 数据范围秒判

范围	可考虑的做法	通常不合适的做法
$n \leq 20$	枚举、DFS、状压	复杂数据结构优先
$n \leq 100$	$O(n^3)$ 、Floyd	指数暴力
$n \leq 1000$	$O(n^2)$	$O(n^3)$ 大概率危险
$n \leq 10^5$	$O(n \log n)$ 、 $O(n)$	$O(n^2)$
$q \leq 10^5$	每次 $O(\log n)$ 或 $O(1)$	每次扫描全体
值域小	计数数组、桶	map 不一定必要
值域大但数量少	map、hash、离散化	直接开值域数组

2.6 B 题优化入口

暴力写法	应想到	模板
每次区间求和都循环	前缀和	T101 ¹⁷⁷
每次矩形求和都双重循环	二维前缀和	T102 ¹⁷⁷
每次区间加都逐个改	差分	T103 ¹⁷⁸ / T104 ¹⁷⁹
每次查是否出现都循环	set / unordered_set	T106 ¹⁸⁰
排序后每次找位置	二分 / lower_bound	C++ 语法章
两个有序表逐个配对	双指针	T108 ¹⁸¹

暴力写法	应想到	模板
最小时间、最大阈值	二分答案	T107¹⁸¹
选若干个凑金额	背包	T301¹⁹³
前置任务、依赖关系	拓扑排序	T205¹⁸⁹

2.7 C 题建模速查

C 题先填这张表，再写代码。

要素	题面中找什么	代码怎么落地
对象	用户、文件、任务、节点、请求、 词条	<code>struct Obj</code>
编号	数字编号、字符串名、路径	<code>vector / map / unordered_map</code>
状态	有效、过期、占用、空闲、允许、 拒绝	<code>enum/int</code> 状态机
命令	每行第一个单词或操作类型	T602²⁰⁶ 命令分发
时间	当前时刻、持续时间、到期时间	T110¹⁸³ / T514²⁰⁴
优先级	“优先” “若相同” “同时发生”	<code>sort</code> 比较器或 <code>priority_queue</code>
失败规则	不存在、非法、超额、冲突	先判断，后修改，必要时回滚

2.8 D/E 部分分人口

看到子任务	写什么	可能分数来源
$n \leq 20$	子集枚举、DFS、状压	小数据分
$n \leq 100$	Floyd、 $O(n^3)$ 、暴力 DP	中小数据分
链	当数组处理，前缀/差分	特殊性质分
树	每次 DFS，或 DFS 序	树特殊性质分
无修改	预处理后回答查询	静态分
查询少	每次暴力	查询少子任务
边权 1	BFS	图特殊性质分
值域小	计数数组	小值域分

2.9 模板编号速查

编号/页	模板	编号/页	模板
T101¹⁷⁷	一维前缀和	T102¹⁷⁷	二维前缀和
T103¹⁷⁸	一维差分	T104¹⁷⁹	二维差分
T105¹⁸⁰	排序自定义比较器	T106¹⁸⁰	哈希统计
T107¹⁸¹	二分答案	T108¹⁸¹	双指针

编号/页	模板	编号/页	模板
T109 ¹⁸²	坐标离散化	T110 ¹⁸³	事件排序
T111 ¹⁸⁴	矩阵重塑与乘法顺序	T112 ¹⁸⁴	稀疏向量点积
T113 ¹⁸⁵	有限状态映射与倍增	T201 ¹⁸⁵	网格 BFS
T202 ¹⁸⁶	DFS 连通块	T203 ¹⁸⁷	并查集
T204 ¹⁸⁸	Dijkstra	T205 ¹⁸⁹	拓扑排序
T206 ¹⁹⁰	Floyd	T207 ¹⁹⁰	Kruskal
T208 ¹⁹¹	LCA 倍增	T209 ¹⁹²	无递归树遍历与子树区间
T301 ¹⁹³	01 背包	T302 ¹⁹³	完全背包
T303 ¹⁹⁴	恰好装满与至少达到	T401 ¹⁹⁵	树状数组
T402 ¹⁹⁵	线段树	T403 ¹⁹⁷	单调栈
T404 ¹⁹⁷	单调队列	T405 ¹⁹⁸	静态区间最大值
T501 ¹⁹⁸	split 字符串分割	T502 ¹⁹⁹	KMP
T503 ²⁰⁰	Trie	T510 ²⁰¹	整除取整和标准取模
T511 ²⁰²	快速幂	T512 ²⁰²	素数和约数
T513 ²⁰³	进制转换	T514 ²⁰⁴	日期处理
T515 ²⁰⁴	质因数分解与筛素数	T601 ²⁰⁵	大模拟骨架
T602 ²⁰⁶	命令分发	T603 ²⁰⁷	状态机
T604 ²⁰⁷	懒删除优先队列	T605 ²⁰⁹	LRU 缓存
T606 ²⁰⁹	路径规范化	T607 ²¹⁰	URL 参数解析
T608 ²¹¹	括号匹配	T609 ²¹¹	简单表达式求值
T610 ²¹²	整数四则表达式解析	—	—

2.10 C++ 语法救急

需求	写法
快速输入输出	<code>ios::sync_with_stdio(false);</code> <code>cin.tie(nullptr);</code>
long long	所有和、乘法、坐标平方、答案上界优先用 long long。
正整数向上取整	$a \geq 0, b > 0$ 时用 $a/b + (a \% b != 0)$ ；有负数时查 T510 ²⁰¹ 。
标准非负余数	<code>r = x % m; if (r < 0) r += m;</code> ，用于负数取模下标。
vector 排序	<code>sort(a.begin(), a.end());</code>
按结构体字段排序	用 sort 加 lambda；完整写法见第四章排序小节。
map 计数	<code>mp[x]++;</code>
set 判断存在	<code>if (st.count(x)) ...</code>
lower_bound	先排序；返回迭代器后检查是否等于 end()。
小根堆	<code>priority_queue<int, vector<int>, greater<int>>.</code>
保留小数	<code>cout << fixed << setprecision(k) << ans;</code>

2.11 最后提交前 30 秒

1. 数组是否开够，是否有 $n + 1$ 、 $m + 1$ 、边界哨兵。
2. 所有乘法、前缀和、答案是否用了 `long long`。
3. 多组数据是否清空 `vector`、`map`、队列、数组。
4. 排序比较器是否满足严格弱序，相等时是否按题目要求。
5. `lower_bound`、`find`、`map` 查询是否检查不存在。
6. 输出空格、换行、小数位是否和样例完全一致。
7. C 题非法操作是否真的没有改状态。
8. D/E 暴力是否至少能过样例和小数据。

第三章 题型详细判断树

3.1 这一章怎样查

如果读完题还拿不准做法，可以按本章的表逐项排查：

1. 这题属于哪一类？
2. 数据范围允许什么复杂度？
3. 题面信号对应哪个模板？
4. 如果满分不会，有什么部分分做法？

查到候选算法后，先确认它的使用条件。相似的关键词可能对应不同问题，尤其要注意操作顺序和数据范围。

3.2 考场读题时先看什么

可以依次看任务、输入结构和数据范围。

层级	问什么	作用
第一层：题号和任务	A/B/C/D/E? 要求输出一个值、一组答案，还是处理很多命令？	决定目标：稳拿、冲分、拆模拟、拿部分分。
第二层：题面结构	输入是数组、矩阵、图、字符串、对象、事件、树，还是混合规则？	决定先翻哪个章节。
第三层：操作和范围	是否多次查询/修改？ n, m, q 多大？值域多大？	决定具体模板和复杂度。

3.3 总判断流程

步骤	问题	动作
1	这是第几题？	A/B 先求稳，C 先拆题，D/E 先找部分分。
2	输出形式是什么？	单个答案偏统计/公式；多行答案偏查询/命令；复杂文本偏解析。
3	数据范围多大？	用复杂度表排除暴力或高级算法。

步骤	问题	动作
4	有多次查询或修改吗?	查前缀和、差分、树状数组、线段树、离线处理。
5	有图、网格、路径吗?	查 BFS、DFS、Dijkstra、并查集、拓扑排序。
6	有字符串格式吗?	查 split、路径、URL、括号、表达式、KMP。
7	有状态、命令、事件吗?	查大模拟、状态机、事件排序。
8	有最优值、选择、容量吗?	查 DP、贪心、二分答案。
9	满分算法不明确吗?	立即找小数据和特殊性质分。

3.4 各表怎样查

后面的表按同样的顺序列出信息，方便对照题目：

项目	必须回答的问题
题面信号	是什么词、输入结构或输出要求让我走到这里?
数据范围	暴力能不能过? 目标复杂度应是多少?
模板入口	应翻第几章、哪个模板编号?
易错点	下标、边界、溢出、排序规则、输出格式哪里最容易错?
小数据做法	满分不会时，能不能先写暴力、小数据或特殊性质版本?

如果某个分支节点无法回答“数据范围”和“模板入口”，不要立刻写代码，先回到数据范围判断树重新分流。

3.5 从根节点开始的总决策树

先用下表区分大致类型，再看对应题型的详细表。

节点	问题	是：走向	否：走向
R0	这是 A 题吗?	进入 A 题详细树，先求稳。	看 R1。
R1	这是 B 题吗?	进入 B 题详细树，先找重复计算。	看 R2。
R2	这是 C 题吗?	进入 C 题详细树，先建模。	看 R3。
R3	这是 D/E 题吗?	先看子任务，进入部分分树。	回题面确认题号。
R4	输出只有一个数或少量值吗?	优先统计、公式、排序、DP。	看是否多次查询或命令。
R5	每次输入操作后都要输出吗?	命令分发、数据结构、状态机。	可能是最后统一输出的模拟/预处理。
R6	有 q 次查询/修改且 q 大吗?	前缀、差分、树状数组、线段树、暴力或简单模拟可能可行。离线。	
R7	有“最小/最大/阈值/最少资源”吗?	检查单调性，可能二分答案或 DP。	看图、字符串、状态。

节点	问题	是：走向	否：走向
R8	有“路径/连通/最短/依赖”吗？	图论或拓扑。	看字符串和大模拟。
R9	有复杂格式、路径、括号、字符串解析、栈、递归解析、表达式吗？		普通数组/矩阵/对象模拟。

3.6 常见问题与模板

下表把常见问题和模板放在一起，也可以直接从这里查找。

分支	题面信号	数据范围判断	模板入口	小数据做法
L01 统计扫描	求和、计数、最大最小	$O(n)$ 可过	A 题、T106 ¹⁸⁰	一遍循环。
L02 排序排名	排名、中位数、多关键字	$O(n \log n)$ 可过	T105 ¹⁸⁰	完整比较器处理小数据。
L03 一维区间	多次区间和/数量	静态查询多	T101 ¹⁷⁷	每次暴力扫小数据。
L04 二维矩阵	子矩阵、邻域、图像	nm 可扫，多次查询用预处理	T102 ¹⁷⁷ /T104 ¹⁷⁹	小矩阵暴力。
L05 哈希查重	是否出现、频率、点集	值域大或字符串键	T106 ¹⁸⁰	map 替代 unordered_map。
L06 坐标离散	坐标大、点少	不能直接开数组	T109 ¹⁸²	map 坐标。
L07 二分答案	最小时间、最大阈值	check 单调且快	T107 ¹⁸¹	枚举小范围答案。
L08 背包选择	选若干、容量、金额	总容量可 DP	T301 ¹⁹³ /T302 ¹⁹³	DFS 小数据。
L09 BFS	最短步数、边权 1	状态数可控	T201 ¹⁸⁵	小图暴力搜索。
L10 Dijkstra	非负权最短路	n, m 大，边权非 1	T204 ¹⁸⁸	小图 Floyd。
L11 拓扑	前置、依赖、逻辑门	DAG 或需判环	T205 ¹⁸⁹	小图反复找入度 0。
L12 并查集	合并集合、连通性	只合并查连通	T203 ¹⁸⁷	BFS/DFS 小数据。
L13 事件模拟	按时间发生、到期	事件数可排序	T110 ¹⁸³	先排序，再逐事件模拟。
L14 状态机	对象状态变化	状态少且规则明确	T603 ²⁰⁷	if/else 先过样例。
L15 路径/URL	路径、参数、配置	总长度可线性	T606 ²⁰⁹ /T607 ²¹⁰	手写 split。
L16 表达式	括号、优先级、化学式	嵌套深度可递归	T608 ²¹¹ /T609 ²¹¹	暴力解析小表达式。
L17 树上路径	祖先、子树、路径	查询多需预处理	T208 ¹⁹¹ /T402 ¹⁹⁵	每次 DFS。
L18 动态区间	修改和查询交替	q 大	T401 ¹⁹⁵ /T402 ¹⁹⁵	修改少就重算。
L19 大模拟	长题面、多对象规则	规则多于算法	T601 ²⁰⁵ /T602 ²⁰⁶	分命令逐步拿分。
L20 D/E 小数据	子任务 n 小或特殊性	暴力能过一档	第八章	先交可运行版本。

3.7 题面结构第一判断

输入主体	题面信号	第一反应	翻到哪里
一串数字 / 数组	求和、最大、最小、相邻、分段、前缀	扫描、排序、前缀和	A/B、 T101¹⁷⁷ 、 T105¹⁸⁰
二维矩阵	图像、网格、矩形、邻域、旋转、重塑	下标转换、二维前缀、矩阵模拟	A/B、 T102¹⁷⁷ 、 T104¹⁷⁹
很多字符串行	路径、URL、JSON、Markdown、命令、表达式	字符串解析、大模拟	C、 T501¹⁹⁸ 、 T606²⁰⁹ 、 T607²¹⁰
点和边	连通、最短路、依赖、拓扑、图论模板		T201–T207¹⁹⁰
树	子树、祖先、路径、删点、合并	DFS、LCA、树形 DP、DFS 序	C/D/E、 T208¹⁹¹
多条操作	修改、查询、插入、删除、回滚	数据结构或离线处理	T103¹⁷⁸ 、 T401¹⁹⁵ 、 T402¹⁹⁵
时间顺序事件	某时刻发生、到期、优先级、事件排序、状态机、堆调度		C、 T110¹⁸³ 、 T603²⁰⁷ 、 T604²⁰⁷
选择若干物品	容量、费用、价值、至少/至多	背包 DP 或贪心	T301¹⁹³ 、 T302¹⁹³
答案最优值	最小时间、最大阈值、最小最大	二分答案或 DP	T107¹⁸¹ 、D/E

3.8 数据范围判断树

3.8.1 单变量规模

范围	可考虑	不宜考虑
$n \leq 10$	全排列、爆搜、手写模拟所有情况	不必急着写复杂优化
$n \leq 20$	子集枚举、状压 DP、折半搜索	$O(n!)$ 仍危险
$n \leq 100$	$O(n^3)$ 、Floyd、区间 DP	指数算法
$n \leq 1000$	$O(n^2)$ 、简单 DP、稠密图	$O(n^3)$ 需谨慎
$n \leq 10^5$	$O(n \log n)$ 、 $O(n)$	$O(n^2)$
$n \leq 10^6$	$O(n)$ 、计数数组、简单筛法	多层 map 或重型结构
$n \geq 10^7$	只考虑线性或数学公式	大数组套多维结构

3.8.2 多变量规模

范围组合	判断	常见模板
$n, m \leq 500$ 且矩阵	$O(nm)$ 或 $O(nm \log nm)$ 通常可行	二维扫描、 T102¹⁷⁷ 、 T104¹⁷⁹

范围组合	判断	常见模板
$n, m \leq 2000$ 且矩阵	尽量 $O(nm)$, 少做多次全图扫描	二维前缀、BFS
$q \leq 10^5$	每次操作必须快	前缀、差分、树状数组、线段树
值域 $\leq 10^6$	可以开计数数组	计数、桶、前缀计数
值域很大但数量少	不可直接开值域数组	map、unordered_map、离散化
边数 $m \approx n$	稀疏图	邻接表、BFS、Dijkstra 堆优化
边数 $m \approx n^2$ 且 n 小	稠密图	Floyd、邻接矩阵
字符串总长 $\leq 10^6$	按总长度线性处理	split、KMP、Trie

数据范围优先级高于题面关键词。例如题面写“最短路”，但 $n \leq 100$ 且多源查询，可以用 Floyd；若 $n, m \leq 10^5$ ，通常要 Dijkstra 或 BFS。

3.9 关键词到模板总索引

题面关键词	判断	模板	章节
多次区间和	静态区间查询	T101 ¹⁷⁷ 一维前缀和	B / 模板库
子矩阵和、邻域均值	二维静态查询	T102 ¹⁷⁷ 二维前缀和	B / 模板库
区间加，最后查询	离线修改	T103 ¹⁷⁸ 一维差分	B / 模板库
矩形加，最后输出	二维离线修改	T104 ¹⁷⁹ 二维差分	B / 模板库
排名、优先级、多关键字	比较器排序	T105 ¹⁸⁰ 排序比较器	A/B/C
出现次数、频率、查重	统计	T106 ¹⁸⁰ 哈希统计	A/B
最小化最大值、最大化最小值	二分答案	T107 ¹⁸¹ 二分答案	B/D
排序后找两数、合并稀疏向量	双指针	T108 ¹⁸¹ 双指针	B
坐标很大、点不多	离散化	T109 ¹⁸² 坐标离散化	B/D
按时间发生、到期、归还	事件模拟	T110 ¹⁸³ 事件排序	B/C
最短步数、迷宫、扩散	BFS	T201 ¹⁸⁵ 网格 BFS	B/C
连通块、区域大小	DFS/BFS	T202 ¹⁸⁶ DFS 连通块	B/C
合并集合、同组、连通性	并查集	T203 ¹⁸⁷ 并查集	B/D
非负权最短路	Dijkstra	T204 ¹⁸⁸ Dijkstra	B/D
依赖、先后顺序、任务计划	拓扑排序	T205 ¹⁸⁹ 拓扑排序	B/D
任意两点最短路，小图	Floyd	T206 ¹⁹⁰ Floyd	B/D
最小连接代价	Kruskal	T207 ¹⁹⁰ Kruskal	D/E
树上祖先、路径	LCA	T208 ¹⁹¹ LCA	D/E
选或不选、容量	01 背包	T301 ¹⁹³ 01 背包	B/D
可重复选择	完全背包	T302 ¹⁹³ 完全背包	B/D

题面关键词	判断	模板	章节
单点修改区间和	树状数组	T401¹⁹⁵ 树状数组	D/E
区间修改查询	线段树	T402¹⁹⁵ 线段树	D/E
最近更大/更小	单调栈	T403¹⁹⁷ 单调栈	B/D
滑动窗口极值	单调队列	T404¹⁹⁷ 单调队列	B/D
路径、配置分割	split	T501¹⁹⁸ split	C
长串匹配	KMP	T502¹⁹⁹ KMP	D/E
前缀查询	Trie	T503²⁰⁰ Trie	D/E
分组、分页、负数取模	整除取整	T510²⁰¹ 整除取整和标准取模	A/B
快速幂、取模幂	数学	T511²⁰² 快速幂	B/D
素数、约数、因子	数学	T512²⁰² 素数约数	B/D
进制转换、编码	数字处理	T513²⁰³ 进制转换	A/B/C
日期、时间、周期	日期处理	T514²⁰⁴ 日期处理	A/C
长题面、多命令	大模拟	T601²⁰⁵ 大模拟骨架	C
每行操作名	命令分发	T602²⁰⁶ 命令分发	C
多状态对象	状态机	T603²⁰⁷ 状态机	C
动态删除仍取最优	懒删除堆	T604²⁰⁷ 懒删除优先队列	C/D
最近最少使用	LRU	T605²⁰⁹ LRU 缓存	C
目录路径	路径规范化	T606²⁰⁹ 路径规范化	C
URL 参数	URL 解析	T607²¹⁰ URL 参数解析	C
括号合法性	栈	T608²¹¹ 括号匹配	C
简单表达式	扫描求值	T609²¹¹ 表达式入门	C

3.10 操作模式判断树

3.10.1 只有查询，没有修改

查询内容	优先算法	模板
区间和、区间数量	前缀和 / 前缀计数	T101¹⁷⁷
子矩阵和、邻域平均	二维前缀和	T102¹⁷⁷
任意区间最值，静态	区间最值结构；不能排序打乱位置	T405¹⁹⁸
图上单源距离	BFS / Dijkstra	T201¹⁸⁵ / T204¹⁸⁸
任意两点距离，小图	Floyd	T206¹⁹⁰
字符串是否出现	find / KMP	T502¹⁹⁹
子树信息，静态	DFS 序 + 前缀 / 树状数组	T209¹⁹² / T101¹⁷⁷ / T401¹⁹⁵

3.10.2 有修改，有查询

操作形态	优先算法	模板
单点修改, 区间和查询	树状数组	T401¹⁹⁵
区间修改, 单点查询	差分思想 / 树状数组	T103¹⁷⁸ / T401¹⁹⁵
区间修改, 区间查询	线段树	T402¹⁹⁵
动态插入删除, 查最大最小	set / 堆 / 懒删除	T604²⁰⁷
修改很少	每次重算	部分分策略
查询很少	每次暴力	部分分策略
操作可以离线	排序事件、差分、并查集倒序	T110¹⁸³ / T103¹⁷⁸ / T203¹⁸⁷

3.10.3 多轮模拟

模拟形态	判断重点	模板
每轮所有对象同时变化	先存变化量, 再统一更新	T601²⁰⁵ / T603²⁰⁷
事件按时间发生	先排序, 时间相同按题目优先级	T110¹⁸³
状态会过期	每次操作前处理过期, 或懒处理	T603²⁰⁷ / T604²⁰⁷
每次选最优对象	priority_queue / set	T604²⁰⁷
命令种类多	每种命令写独立函数	T602²⁰⁶

3.11 A 题快速判断树

A 题先检查能否直接扫描或模拟, 再看是否需要排序、查找等操作。

1. 是否只要求一个数? 优先一遍扫描、公式、计数。
2. 是否有排序、排名、最大最小? 用 `sort` 后处理。
3. 是否有字符或字符串? 按字符扫描, 先别写复杂解析。
4. 是否有二维数组? 先画 2×3 小例子确认下标。
5. 是否有日期、时间、进制、取整、小数? 翻对应模板, 不临场硬想语法。

题面信号	做法	模板/章节
求和、计数、最大最小	一遍扫描	A 题统计
出现次数、直方图	计数数组 / map	T106¹⁸⁰
小数、百分比、平均、方差	公式 + <code>setprecision</code>	A 题公式
分组、分页、向上取整、负数取模	整数公式, 不用浮点硬凑	T510²⁰¹
排序、排名、中位数	sort	T105¹⁸⁰
字符串、字符、校验码	string 扫描	T501¹⁹⁸
进制、数位、编码	进制转换或取模拆位	T513²⁰³
日期、星期、红绿灯周期	闰年、月份、取模	T514²⁰⁴

题面信号	做法	模板/章节
坐标距离、矩形面积	公式，注意边界	A 题坐标
二维矩阵变换	写原坐标到新坐标公式	A 题二维表格

3.11.1 A 题详细分支表

分支	什么题会用到	先写什么	易错点	翻到哪里
A1 纯统计	求总和、数量、最大最小	一遍扫描	初值、long long	第五章统计
A2 频率统计	出现次数、直方图	数组计数或 map	值域是否能开数组	T106 ¹⁸⁰
A3 排序输出	第几大、排名、中位数	sort 后取值	下标从 0 还是 1	T105 ¹⁸⁰
A4 公式计算	平均、方差、加权、现值、分组	按公式翻译	整数除法、取整、小数位	第五章公式 / T510 ²⁰¹
A5 字符处理	校验码、字符类别、字符串计数	string 扫描	getline 和 cin 混用	T501 ¹⁹⁸
A6 数位/进制	数位和、编码、进制分解	取模除法	0、负数、long long	T513 ²⁰³
A7 日期时间	星期、天数、周期灯	月份表 + 闰年	闭区间、取模	T514 ²⁰⁴
A8 二维表格	图像、矩阵、邻居平均	画小矩阵找下标	行列反、边界	第五章二维
A9 坐标几何	距离、矩形面积、点位置	套公式	交集为负取 0	第五章坐标
A10 简单状态	连续得分、开关、当前状态	变量记录状态	状态重置时机	第五章模拟

3.12 B 题快速判断树

B 题可以先估算暴力要算多少次，再看哪些结果能够提前保存。

- 暴力做法是什么？每次是否重复算同一段、同一块、同一类？
- 重复算区间：前缀和。重复改区间：差分。
- 重复查存在：set/map/hash。
- 排序后规律明显：排序扫描、双指针、二分。
- 有最小时间/最大阈值：先问答案是否单调。

题面信号	做法	模板
多次区间/矩阵查询	前缀和	T101 ¹⁷⁷ / T102 ¹⁷⁷
区间修改最后输出	差分	T103 ¹⁷⁸ / T104 ¹⁷⁹
频率统计、查重、点集匹配	哈希 / map / set	T106 ¹⁸⁰
排序后扫描、阈值统计	sort + 扫描	T105 ¹⁸⁰

题面信号	做法	模板
两个有序序列匹配	双指针	T108 ¹⁸¹
答案具有单调性	二分答案	T107 ¹⁸¹
最短步数、扩散	BFS	T201 ¹⁸⁵
连通关系、合并集合	并查集	T203 ¹⁸⁷
简单依赖、任务计划	拓扑排序	T205 ¹⁸⁹
背包信号、至少达到某值	DP	T301 ¹⁹³ / T302 ¹⁹³
质因数、约数、幂	数学模板	T511 ²⁰² / T512 ²⁰²

3.12.1 B 题详细分支表

分支	暴力形态	优化判断	模板	保底
B1 一维前缀	每次扫区间	只查不改, 多次区间和	T101 ¹⁷⁷	暴力扫小数据
B2 二维前缀	每次扫矩形	矩阵静态, 多次子矩阵	T102 ¹⁷⁷	暴力双循环
B3 一维差分	每次区间逐个加	区间修改, 最后输出/查询少	T103 ¹⁷⁸	直接修改小数据
B4 二维差分	每次矩形逐格加	矩形覆盖, 最后统计	T104 ¹⁷⁹	二维暴力
B5 哈希统计	每次线性找存在	查重、频率、坐标点存在	T106 ¹⁸⁰	map 先写稳
B6 排序扫描	枚举每个阈值再统计	排序后相邻段贡献可更新	T105 ¹⁸⁰ /T101 ¹⁷⁷	枚举阈值小数据
B7 双指针	双重循环匹配	两序列有序或可排序	T108 ¹⁸¹	map 匹配
B8 二分答案	枚举答案	check 单调, 答案范围大	T107 ¹⁸¹	枚举小范围
B9 简单 BFS	枚举路径	无权最短步数/扩散	T201 ¹⁸⁵	DFS 小图
B10 并查集	每次 DFS 判连通	只有合并和查连通	T203 ¹⁸⁷	邻接表 DFS
B11 拓扑	反复扫描找可做任务	前置依赖固定	T205 ¹⁸⁹	小 n 反复扫描
B12 背包	枚举所有选择	选或不选, 容量不太大	T301 ¹⁹³	DFS 子集
B13 基础数学	枚举因子到 n	质因数、幂、约数	T511 ²⁰² /T512 ²⁰²	小范围试除
B14 矩阵优化	按题意多重循环超时	调整乘法/扫描顺序	T111 ¹⁸⁴ / T112 ¹⁸⁴	暴力部分分
B15 时间区间	每分钟逐个标记	时间范围不大或可差分	T103 ¹⁷⁸ /T110 ¹⁸³	数组标记小范围

3.13 C 题快速判断树

C 题规则较多时, 先记下对象、状态、命令和处理顺序, 写到一半再改数据结构会比较费时间。

3.13.1 C 题读题分解

要做什么	在题面中怎么标记	代码中的写法
对象	用户、文件、节点、任务、请求、设备、词条	struct
状态	空闲/占用、有效/过期、允许/拒绝、在线/离线	enum 或 int
命令	create、delete、query、request、release	T602 ²⁰⁶ 命令分发
时间	到期、当前时刻、持续时间、周期	T110 ¹⁸³ 、T514 ²⁰⁴
规则优先级	先判断 A, 再判断 B; 相同时间谁优先	独立函数, 写注释
输出	每次命令输出、最后统一输出、格式化文本	先写输出样例函数

3.13.2 C 题模板选择

题面信号	做法	模板
长题面、多对象、多规则	大模拟拆解	T601 ²⁰⁵
每行一个命令	命令分发	T602 ²⁰⁶
对象状态变化	状态机	T603 ²⁰⁷
按时间发生	事件排序	T110 ¹⁸³
每次取最优任务	优先队列 / 懒删除	T604 ²⁰⁷
路径、URL、配置	字符串解析	T501 ¹⁹⁸ / T606 ²⁰⁹ / T607 ²¹⁰
括号、表达式、方程式	栈 / 递归解析 / 扫描	T608 ²¹¹ / T609 ²¹¹
缓存淘汰	list + map 或队列	T605 ²⁰⁹
地图带状态	状态 BFS	C 题章节
树或目录结构	DFS、父子 map、路径查找	T606 ²⁰⁹ / T208 ¹⁹¹

3.13.3 C 题详细分支表

分支	题面信号	代码结构	易错点	模板
C1 命令分发	每行操作名不同	主循环读 op, 分函数处理	参数数量不同	T602 ²⁰⁶
C2 状态机	对象有状态变化	状态字段 + 转移函数	同一操作多次转移数	T603 ²⁰⁷
C3 事件排序	时间、到期、归还、同时发生	Event 结构体排序	同时事件优先级	T110 ¹⁸³
C4 优先队列	每次取最优任务/对象	heap 或 set	失效元素删除	T604 ²⁰⁷
C5 路径目录	文件、目录、父子、配额	树节点 + path map	创建失败回滚	T606 ²⁰⁹

分支	题面信号	代码结构	易错点	模板
C6 URL/参数	路径、查询参数、匹配规则	split + 参数 map	空参数、类型检查	T607 ²¹⁰
C7 JSON/配置	嵌套键值、查询路径	栈/递归建结构	转义和层级	T501 ¹⁹⁸
C8 Markdown/文本	多行文本、块、行内格式	先分块再处理行	空行分块	T601 ²⁰⁵
C9 表达式/括号	运算、括号、化学式	栈或递归解析	优先级、嵌套倍数	T608 ²¹¹ /T609 ²¹¹
C10 编码解码	十六进制、压缩、校验	指针扫描字符串	长度字段越界	T513 ²⁰³
C11 矩阵图像	固定矩阵、扫描顺序、二维数组 + 顺序表变换		行列、取整	T102 ¹⁷⁷
C12 权限规则	用户、角色、资源、权限	set/map 规则判断	通配符、最大等级	T603 ²⁰⁷
C13 带状态 BFS	地图、钥匙、方向、时间	visited 包含状态	状态维度漏掉	T201 ¹⁸⁵
C14 树/目录模拟	子树、删除、查询、合并	DFS + 父子关系	删除后引用失效	T208 ¹⁹¹
C15 选择子任务	规则太多写不完	实现子任务所需命令	该子任务内仍须遵守全部规则	T601 ²⁰⁵

3.14 逐句读题分支树

考场读题时，建议拿笔在题面旁边标记下面六类词。标完后再决定模板。

标记	看到什么就圈出来	后续判断
对象	学生、用户、文件、节点、仓库、服务器、任务、词条	是否需要 struct? 是否有编号到对象的 map?
属性	权重、时间、状态、坐标、容量、权限、父节点	是否要排序、比较器、状态机?
动作	查询、修改、创建、删除、移动、合并、授权、到期	是否是命令分发或数据结构?
范围	n 、 m 、 q 、值域、字符串总长、矩阵大小	直接排除不可能复杂度。
关系	相邻、包含、依赖、祖先、连通、路径、前缀	图、树、拓扑、字符串、区间?
目标	最大、最小、数量、是否存在、方案数、输出过程	统计、二分、DP、搜索还是模拟?

3.14.1 从句子到模板

题面句子模式	立即追问	模板方向
“给定 q 次询问”	每次询问是否只读不改?	前缀和 / 数据结构

题面句子模式	立即追问	模板方向
“每次修改后回答”	修改是点、区间还是对象状态?	树状数组 / 线段树 / 状态机
“求满足条件的数量”	条件能否拆成前缀、排序或哈希?	计数 / 排序扫描 / map
“求最小的 x 使得……”	x 越大是否越容易满足?	二分答案
“选择若干个”	每个物品选一次还是多次?	01 背包 / 完全背包
“从 A 到 B 的最短……”	边权是否全为 1?	BFS / Dijkstra
“任务必须在前置任务后”	是否有环?	拓扑排序
“字符串由若干规则组成”	是否有括号或嵌套?	split / 栈 / 递归解析
“路径包含. 和..”	是否需要规范化?	T606 ²⁰⁹
“状态在某时刻过期”	过期是在操作前还是操作后处理?	状态机 / 事件模拟

3.15 B 题专项分支树

如果暴力复杂度太高，可以从重复计算的位置找改法。

3.15.1 B 题从暴力到优化

暴力形态	重复在哪里	优化方向
每次询问都扫一段数组	区间和/数量重复计算	一维前缀和 T101 ¹⁷⁷
每次询问都扫一个矩形	子矩阵重复计算	二维前缀和 T102 ¹⁷⁷
每次区间加都逐个改	区间修改重复	差分 T103 ¹⁷⁸ /T104 ¹⁷⁹
每次判断点是否存在都扫所有点	存在性查询重复	set / unordered_set
每次找匹配点都双重循环	坐标或编号匹配重复	map / 双指针 / 排序
每个候选答案都完整模拟	答案范围大	二分答案 T107 ¹⁸¹
每次重新排序	只需维护极值或有序集合	priority_queue / set
每次从头判断依赖任务	依赖结构固定	拓扑排序 T205 ¹⁸⁹

3.15.2 B 题二分答案确认表

问题	是	否
答案是一个整数或实数吗?	继续问单调性	多半不是二分答案。
给定答案 x ，能快速判断不可行吗?	写 <code>check(x)</code>	不适合二分。
x 变大后，可行性只朝一个方向变化吗?	可以二分	不能二分。

问题	是	否
题目问最小时间/最大阈值/最少资源吗?	优先尝试二分	可能是 DP/贪心。

3.16 C 题专项分支树

先把输入中的对象和操作对应到代码，再检查每次操作的复杂度是否合适。

3.16.1 C 题对象建模表

题面对象	常见字段	容器选择
用户 / 角色 / 权限	名称、权限集合、等级、资源类型	<code>map<string, Obj></code> , <code>set<string></code>
文件 / 目录	名称、父节点、子节点、大小、配额	<code>map</code> 路径到节点, 或树节点数组
服务器 / 节点	编号、算力、任务数、可用状态	<code>vector + set/priority_queue</code>
任务 / 请求	时间、优先级、资源需求、状态	<code>struct + sort/heap</code>
词条 / token	字符串、权重、出现位置	<code>map/unordered_map + heap</code>
地图状态	坐标、方向、钥匙、时间、能量	<code>queue + visited</code> 多维数组/ <code>map</code>

3.16.2 C 题规则优先级表

题面出现	必须单独确认	常见错误
“若同时发生”	同时事件的排序规则	先后顺序写反。
“如果不存在则……”	缺省值或失败输出	<code>map</code> 自动创建误判存在。
“到期”	到期时刻是否还有效	<code><</code> 和 <code>≤</code> 写错。
“优先选择”	比较器所有关键字	忘记编号升序等最后规则。
“忽略非法操作”	是否改变状态、是否输出	非法时仍修改了数据。
“回滚/失败”	前面临时改动要撤销	配额、路径创建类常见。

3.16.3 C 题字符串解析分支

字符串形态	判断	模板
固定分隔符，无嵌套	<code>split</code> 即可	T501¹⁹⁸
路径，含 <code>.</code> 、 <code>...</code> 、多个斜杠	栈式规范化	T606²⁰⁹
URL，含路径和查询参数	先按 <code>?</code> ，再按 <code>&</code>	T607²¹⁰
括号嵌套	栈或递归	T608²¹¹
有整数四则优先级	两栈或递归下降	T610²¹²

字符串形态	判断	模板
化学式/表达式含倍数 多行文本块	递归解析返回 map 先分块，再行内解析	C 题表达式扩展 T601 ²⁰⁵

3.17 D/E 题快速判断树

D/E 题可以先看子任务表，找已经会写的范围或特殊情况。

1. 找子任务数据范围： $n \leq 20$ 、链、树、无修改、边权 1、查询少。
2. 写最朴素暴力，估算它能过哪档。
3. 如果暴力只有 0 分，再找一个最接近的模板：BFS、Dijkstra、Kruskal、树状数组、线段树、DP。
4. 优先实现你能调通的版本，不把所有时间都压在复杂满分做法上。

题面信号	满分方向	部分分方向
$n \leq 20$ 子任务	状压 / DFS	枚举子集
树上路径	LCA / 树上差分	每次 DFS 找路径
区间修改查询	线段树	暴力 / 树状数组 / 差分
最小生成代价	Kruskal	小图枚举或直接 Kruskal
字符串匹配	KMP / Trie	find / 暴力匹配
非负权最短路	Dijkstra	小图 Floyd / 无权 BFS
最大化最小值	二分答案	枚举答案小范围
复杂 DP	优化 DP	DFS / 小状态 DP
动态集合取最值	平衡树 / 堆懒删除	每次排序或线性扫描

3.18 D/E 部分分决策树

完整做法暂时不会时，按下面的顺序寻找可做的子任务。

1. 先看子任务表，有没有 $n \leq 20$ 、 $n \leq 100$ 、链、树、无修改、边权 1、查询少。
2. 如果有小 n ：优先 DFS、枚举、Floyd、 $O(n^2)$ DP。
3. 如果有特殊图：边权 1 用 BFS，树用 DFS，DAG 用拓扑。
4. 如果有特殊操作：无修改用前缀，有修改少就每次重算，查询少就每次暴力。
5. 如果只有大数据：确认是否存在可正确处理的特殊输入；没有明确子任务时不能保证部分分，不要仅因模板相近就硬套。

子任务/特殊性质	可写版本	常用模板
$n \leq 20$	枚举子集、DFS、状压	状压 DP、搜索
$n \leq 100$	Floyd、 $O(n^3)$ DP、暴力查询	T206 ¹⁹⁰
树退化成链	数组/前缀处理路径	T101 ¹⁷⁷ / T103 ¹⁷⁸

子任务/特殊性质	可写版本	常用模板
没有修改	预处理后回答查询	前缀、DFS 序
查询次数很少	每次 DFS/BFS/扫描	暴力部分分
边权全为 1	BFS 替代 Dijkstra	T201 ¹⁸⁵
图是 DAG	拓扑 DP	T205 ¹⁸⁹
值域小	计数数组/桶	T106 ¹⁸⁰
参数 k 很小	状态压缩或按 k 枚举	状压入口

3.18.1 D/E 题详细分支表

分支	什么题会用到	满分方向	部分分写法	翻到哪里
DE1 小 n	$n \leq 20$ 或参数小	状压/搜索优化	枚举子集、DFS	第八章状压
DE2 小图	$n \leq 100$	Floyd/DP	Floyd 或暴力建图	T206 ¹⁹⁰
DE3 树路径	树上多次路径	LCA/树剖/树差分	每次 DFS 找路径	T208 ¹⁹¹
DE4 子树查询	子树加、子树查	DFS 序 + 树状数组/线段树	每次遍历子树	T401 ¹⁹⁵ /T402 ¹⁹⁵
DE5 动态区间	区间改查交替	线段树	小范围数组模拟	T402 ¹⁹⁵
DE6 动态集合	插入删除查极值	set/堆懒删除	每次排序/扫描	T604 ²⁰⁷
DE7 最短路	正权图路径	Dijkstra/建图优化	Floyd 小图、BFS 边权 1	T204 ¹⁸⁸
DE8 最小生成	连通最小代价	Kruskal/Prim	直接 Kruskal	T207 ¹⁹⁰
DE9 字符串匹配	长串、多模式、前缀	KMP/Trie/AC 自动机	find 暴力	T502 ¹⁹⁹ /T503 ²⁰⁰
DE10 复杂 DP	最优方案数、限制多	状态设计/优化	记忆化 DFS 小数据	第八章 DP
DE11 数学计数	组合、取模、约数	预处理/数论	枚举小范围	T511 ²⁰² /T512 ²⁰²
DE12 离线处理	操作可重排、查询很多	排序/分治/离线 DSU	按输入暴力	T110 ¹⁸³ /T203 ¹⁸⁷
DE13 几何	点线面、距离、交叉	计算几何	小数据枚举	未收专用模板
DE14 网络流/匹配	分配、匹配、容量流	网络流/二分图	极小规模枚举匹配并验证	放弃满分

3.19 容易误判的分支

看起来像	但如果题面出现	应改判为
简单模拟	q 很大，每次都查区间	前缀和 / 差分 / 数据结构
排序题	要动态插入删除后仍排名	set、树状数组、线段树
字符串题	有括号嵌套、表达式优先级	栈或递归下降
图搜索题	边权不同但均非负	Dijkstra；含负权需另选算法
BFS 题	状态包含钥匙、方向、时间、能量	状态 BFS
贪心题	选或不选、容量限制明显	背包 DP

看起来像	但如果题面出现	应改判为
二分题 树题	check 不单调 只有父子目录和路径字符串	不能二分, 改 DP/贪心/扫描 可能是大模拟, 不一定是 LCA
数学题	有多次询问和大范围	预处理、筛法、快速幂

3.20 无法判断时的暂时没有思路时

如果读完题仍不知道算法:

1. 写出最暴力做法。
2. 估算暴力复杂度。
3. 看数据范围中有没有小数据能过。
4. 找特殊性质: 无修改、链、树、边权 1、参数小。
5. 先实现能过样例和小数据的版本。
6. 再尝试从本章关键词表找优化方向。

3.21 判断树反向测试表

做完一道题后, 可以记下自己最初的判断和实际做法有什么差别, 下次看到相似题面时再翻出来看。

问题	例子	应能定位到
题面主体是什么?	数组、矩阵、图、字符串、对象、事件	题面结构第一判断
是否有多次操作?	q 次查询、每次修改、每次输出	操作模式判断树
暴力复杂度是多少?	$O(nq)$ 、 $O(n^2)$ 、 $O(nm)$	数据范围判断树
关键词是否误导?	看起来像模拟但 q 很大	容易误判分支
满分不会时能拿什么?	小数据、特殊性质、暴力	D/E 部分分决策树
对应模板编号是什么?	T101 ¹⁷⁷ 、T107 ¹⁸¹ 、T603 ²⁰⁷ 等	关键词到模板总索引

3.22 最后 5 分钟判断

情况	处理
A/B 还没过样例	放下后面题, 先修 A/B。
C 写了一半但样例不过	注释掉高风险优化, 保留最基础模拟。
D/E 没思路	找最小子任务, 写暴力或特殊性质。
数组越界风险高	全部下标检查一遍, 必要时数组开大 5 到 10。

情况	处理
输出格式不确定	对照样例逐字符检查空格、换行、小数位。

第二部分

基础与题型

第四章 C++ 语法与 STL 速查

4.1 本章使用方式

这一章放常用的 C++ 写法。忘了容器怎么声明、循环怎么写，或者拿不准整数除法和取模的结果时，可以从这里查：

- 一个程序应该怎么开头。
- 输入输出怎么写。
- 什么时候必须用 `long long`。
- 整数除法、取模、向上取整、向下取整怎么写。
- 数组、`vector`、`string`、结构体怎么用。
- `sort`、`map`、`set`、`queue`、`priority_queue` 怎么写。
- `lambda` 比较器怎么写，常见错误是什么。

写法熟悉就继续用，不必为了把代码写短而临时换一种。

4.2 语言标准与代码拼装

本书代码使用 **GNU C++17**。结构化绑定 `auto [x,y]` 和 `std::gcd` 需要 C++17，编译时可使用 `-std=c++17`。

若只支持旧标准，把 `auto [x,y]=p;` 改成 `auto x=p.first; auto y=p.second;; gcd` 可自行写欧几里得算法。`bits/stdc++.h` 是 GNU 扩展；非 GNU 环境改用对应标准头文件。

模板不是一整份提交代码。先写下面的主函数骨架；结构体、普通函数放在 `main` 外，输入、循环、输出放在 `main` 或 `solve` 内。不要把两个同名 `main`、`Node`、`Edge` 或 `INF` 一起粘贴。带 `check`、`handle...` 和省略号的代码是待按题意补全的框架。

4.3 最小程序骨架

大部分 CSP 题目都可以从这个骨架开始：

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
```

```
7   ios::sync_with_stdio(false);
8   cin.tie(nullptr);
9
10  // write code here
11
12  return 0;
13 }
```

说明：

- `#include <bits/stdc++.h>` 是 GNU C++ 的扩展头文件，包含大多数常用标准库。
- `using namespace std;` 可以少写 `std::`。
- `using ll = long long;` 以后可以用 `ll` 表示 `long long`。
- `ios::sync_with_stdio(false); cin.tie(nullptr);` 用来加速 C++ 输入输出。
- 使用上面两行加速后，不要混用 `scanf/printf` 和 `cin/cout`。

4.4 输入输出

4.4.1 普通输入

```
1 int n;
2 cin >> n;
3
4 int a, b;
5 cin >> a >> b;
6
7 string s;
8 cin >> s; // reads until whitespace
```

4.4.2 读入数组

```
1 int n;
2 cin >> n;
3 vector<int> a(n);
4 for (int i = 0; i < n; i++) {
5     cin >> a[i];
6 }
```

如果题目下标从 1 开始，考场上可以直接开成 `n + 1`：

```
1 int n;
2 cin >> n;
3 vector<int> a(n + 1);
4 for (int i = 1; i <= n; i++) {
5     cin >> a[i];
6 }
```

4.4.3 多组数据

题目明确给出 T :

```
1 int T;
2 cin >> T;
3 while (T--) {
4     solve();
5 }
```

题目没有给 T , 读到文件结束:

```
1 int a, b;
2 while (cin >> a >> b) {
3     cout << a + b << '\n';
4 }
```

多组数据常见坑: 每组开始前要清空数组、图、队列、map、答案变量。

4.4.4 整行输入

`cin >> s` 只能读到空格前。如果要读整行, 用 `getline`:

```
1 string line;
2 getline(cin, line);
```

如果前面刚用了 `cin >> n`, 再用 `getline`, 要先吃掉换行:

```
1 int n;
2 cin >> n;
3 cin.ignore(numeric_limits<streamsize>::max(), '\n');
4
5 string line;
6 getline(cin, line);
```

如果担心行尾有多个字符, 可以写:

```
1 cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

4.4.5 输出

```
1 cout << ans << '\n';
2 cout << a << ' ' << b << '\n';
```

不要输出提示语。 例如不要写 `cout << "answer=" << ans;`, 除非题目明确要求。

4.4.6 小数输出

```
1 double x = 3.1415926;
2 cout << fixed << setprecision(2) << x << '\n'; // 3.14
```

说明:

- `fixed` 表示按小数位输出。
- `setprecision(2)` 表示保留 2 位小数。
- 需要包含 `<iomanip>`，但 `bits/stdc++.h` 已包含。

4.5 数据类型

4.5.1 整数类型

类型	大约范围	使用场景
<code>int</code>	-2.1×10^9 到 2.1×10^9	下标、普通计数、小范围整数。
<code>long long</code>	-9.2×10^{18} 到 9.2×10^{18}	求和、乘法、答案、距离、方案数。
<code>unsigned long long</code>	0 到约 1.8×10^{19}	很少必须用，新手慎用。

必须用 `long long` 的常见情况：

- n 个数求和，每个数可达 10^9 。
- 两个最大可达 10^9 的数相乘。
- 最短路边权和可能很大。
- 方案数、组合数、累计时间、累计代价。

乘法防溢出：

```
1 int a = 1000000000;
2 int b = 1000000000;
3 long long x = 1LL * a * b;
```

错误写法：

```
1 long long x = a * b; // a*b already overflowed as int
```

4.5.2 浮点类型

- `double` 是常用浮点数。
- 判断浮点相等时不要直接用 `==`，通常用误差。

```
1 const double EPS = 1e-9;
2 if (abs(a - b) < EPS) {
3     // a and b are almost equal
4 }
```

4.5.3 字符和布尔

```
1 char c = 'A';
2 bool ok = true;
```

字符判断：

```
1 isdigit((unsigned char)c); // is digit
2 isalpha((unsigned char)c); // is letter
3 islower((unsigned char)c);
4 isupper((unsigned char)c);
5 tolower((unsigned char)c);
6 toupper((unsigned char)c);
```

注意： `isdigit((unsigned char)c)` 返回非零表示真，不一定返回 1。

4.6 表达式、整除和取整

4.6.1 基本运算符

C++ 中常用算术运算符如下：

运算符	含义	注意点
<code>+</code> <code>-</code> <code>*</code>	加、减、乘	两个 <code>int</code> 相乘仍先按 <code>int</code> 计算。
<code>/</code>	除法	两边都是整数时做整数除法，结果向 0 截断。
<code>%</code>	取余	只能用于整数；负数取余的结果可能为负。
<code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>	复合赋值	<code>x += y</code> 等价于 <code>x = x + y</code> 。
<code>++</code> <code>--</code>	自增、自减	简单循环中优先写 <code>i++</code> 或 <code>++i</code> ，不要嵌进复杂表达式。

整数除法不是数学里的向下取整。在 C++ 中，`-3 / 2` 的结果是 `-1`，因为整数除法向 0 截断；数学意义上的 $\lfloor -3/2 \rfloor$ 是 `-2`。

4.6.2 整数除法和类型转换

如果两边都是整数，`/` 会丢掉小数部分：

```
1 int a = 5, b = 2;
2 cout << a / b << '\n'; // 2
```

如果要小数结果，至少把一个操作数转成浮点数：

```
1 cout << 1.0 * a / b << '\n'; // 2.5
2 cout << (double)a / b << '\n'; // 2.5
```

整数答案尽量用整数运算求。大整数转成 `double` 后可能丢失低位，取整时尤其要留意。

4.6.3 正整数向上取整

最常见场景： a 个物品，每组最多放 b 个，问需要几组。若 $a \geq 0, b > 0$ ：

```
1 long long ceil_div_positive(long long a, long long b) {
2     return a / b + (a % b != 0);
3 }
```

常见特例：

```
1 long long half_up = (n + 1) / 2; // ceil(n / 2), n >= 0
2 long long blocks = (n + block - 1) / block;
```

这个简式只适合 $a \geq 0, b > 0$ 。如果 a 可能为负，或者 b 可能为负，不能直接套 $a/b+(a\%b!=0)$ 。

4.6.4 有符号整数向下和向上取整

如果题目涉及负坐标、负偏移、时间差、数学分段，建议使用下面的通用函数。它们要求 $b \neq 0$ ，并且遵循数学意义上的 floor 和 ceil。

```
1 // Require b != 0; exclude LLONG_MIN / -1.
2 long long floor_div(long long a, long long b) {
3     long long q = a / b;
4     long long r = a % b;
5     if (r != 0 && ((r > 0) != (b > 0))) q--;
6     return q;
7 }
8
9 long long ceil_div(long long a, long long b) {
10    long long q = a / b;
11    long long r = a % b;
12    if (r != 0 && ((r > 0) == (b > 0))) q++;
13    return q;
14 }
```

快速自测：

表达式	floor_div	ceil_div
$3/2$	1	2
$-3/2$	-2	-1
$3/-2$	-2	-1
$-3/-2$	1	2

4.6.5 负数取模

C++ 中 $x \% m$ 的符号跟 x 相同。例如 $-1 \% 5$ 是 -1 ，不是 4 。如果题目需要 0 到 $m-1$ 的标准余数，写：

```
1 long long norm_mod(long long x, long long m) {
2     long long r = x % m;
```

```

3   if (r < 0) r += m;
4   return r;
5 }

```

适用： $m > 0$ 。若 x 可能小于 $-m$ 很多，上面仍然正确，因为 C++ 的一次取余后 r 已经在 $(-m, m)$ 内。

4.6.6 运算优先级和括号

常用的优先级关系如下。混合运算拿不准时，可以多加一层括号：

- 乘、除、取模优先于加减。
- 比较运算优先级低于加减乘除。
- `&&` 优先于 `||`。
- 位运算和比较混在一起时必须加括号。

```

1 if ((mask & (1 << k)) != 0) {
2     // bit k is 1
3 }

```

有疑问就加括号。 括号不会让程序变慢，却能显著减少读错表达式的风险。

4.6.7 常量和初始化

常用写法：

```

1 const int N = 100000 + 5;
2 const long long INF = (long long)4e18;
3 const int MOD = 1000000007;
4
5 int cnt = 0;
6 long long sum = 0;
7 bool ok = true;

```

提醒：

- 局部普通变量不会自动清零，定义后要赋初值。
- 全局数组会默认清零，但多组数据不会自动恢复初始状态。
- `const` 常量不能被修改，适合写数组上限、模数、无穷大。

4.6.8 四舍五入与银行家舍入不能混用

`llround` 的半整数规则是远离 0；题面要求“恰好一半时取偶数”时应自行实现。若输入为非负数、恰有一位小数，且整数部分不超过 10^5 ，可以完全避免浮点：

```

1 pair<long long, long long> roundOneDecimal(const string& s) {
2     size_t dot = s.find('.');
3     long long a = stoll(s.substr(0, dot));
4     int b = s[dot + 1] - '0';

```

```
5   long long halfUp = a + (b >= 5);
6   long long halfEven = a + (b > 5 || (b == 5 && a % 2 == 1));
7   return {halfUp, halfEven};
8 }
```

例如 2.5 分别得到 3 与 2, 3.5 两者均为 4。此代码仅针对上述输入格式; 不能直接用于负数、多位小数或超大整数。setprecision 负责显示格式, 不替代题目定义的舍入算法。

4.7 控制流

4.7.1 if

```
1 if (x > 0) {
2     cout << "positive\n";
3 } else if (x == 0) {
4     cout << "zero\n";
5 } else {
6     cout << "negative\n";
7 }
```

4.7.2 for

```
1 for (int i = 0; i < n; i++) {
2     // 0, 1, ..., n-1
3 }
4
5 for (int i = 1; i <= n; i++) {
6     // 1, 2, ..., n
7 }
```

倒序:

```
1 for (int i = n - 1; i >= 0; i--) {
2     // be careful when i is unsigned
3 }
```

4.7.3 while

```
1 while (condition) {
2     // loop
3 }
```

读到结束:

```
1 int x;
2 while (cin >> x) {
3     // process x
4 }
```

4.7.4 break 和 continue

```

1 for (int i = 0; i < n; i++) {
2     if (a[i] < 0) continue; // skip this round
3     if (a[i] == target) break; // exit loop
4 }

```

4.8 auto、引用和范围 for

4.8.1 auto

auto 让编译器根据右侧表达式推断类型，常用于迭代器、pair、复杂容器遍历。

```

1 auto it = lower_bound(a.begin(), a.end(), x);
2 auto p = make_pair(3, 5);

```

遍历 map:

```

1 map<string, int> cnt;
2 for (auto [key, value] : cnt) {
3     cout << key << ' ' << value << '\n';
4 }

```

注意复制。 auto [key, value] 会复制键和值；如果元素很大，写 const auto& 更合适。

```

1 for (const auto& item : records) {
2     // read item without copying
3 }

```

4.8.2 引用

引用相当于变量的别名。函数传大对象时常用引用避免复制。

```

1 void printVector(const vector<int>& a) {
2     for (int x : a) cout << x << ' ';
3 }
4
5 void addOne(vector<int>& a) {
6     for (int& x : a) x++;
7 }

```

区别:

- const vector<int>& a: 不复制，也不允许函数内修改。
- vector<int>& a: 不复制，允许函数内修改原数组。
- vector<int> a: 会复制整个数组，数据大时慢。

4.8.3 范围 for

只读遍历：

```
1 for (int x : a) {
2     cout << x << '\n';
3 }
```

修改原元素：

```
1 for (int& x : a) {
2     x++;
3 }
```

如果忘了写引用，修改不会回到原数组。

```
1 for (int x : a) {
2     x++; // only modifies the copy
3 }
```

4.8.4 size 的类型

`a.size()` 的返回类型是无符号整数。考场上为了减少类型麻烦，常写成：

```
1 int n = (int)a.size();
2 for (int i = 0; i < n; i++) {
3     // use a[i]
4 }
```

倒序遍历不要写下面这种：

```
1 for (int i = a.size() - 1; i >= 0; i--) {
2     // dangerous when a is empty
3 }
```

更稳的写法：

```
1 for (int i = (int)a.size() - 1; i >= 0; i--) {
2     // ok even when a is empty: i starts at -1
3 }
```

4.9 函数

把重复逻辑写成函数，能减少大模拟和算法题的错误。

```
1 int add(int a, int b) {
2     return a + b;
3 }
4
5 bool isEven(int x) {
6     return x % 2 == 0;
7 }
```

传 vector:

```
1 long long sumVector(const vector<int>& a) {  
2     long long sum = 0;  
3     for (int x : a) sum += x;  
4     return sum;  
5 }
```

说明:

- `const vector<int>& a` 表示不复制整个数组, 并且函数内部不修改它。
- 如果函数需要修改数组, 去掉 `const`。

```
1 void addOne(vector<int>& a) {  
2     for (int& x : a) x++;  
3 }
```

4.10 数组

4.10.1 普通数组

全局数组适合大数组:

```
1 const int N = 100000 + 5;  
2 int a[N];  
3 long long pre[N];
```

说明:

- 全局数组默认初始化为 0。
- 局部大数组可能爆栈, 尽量放全局或用 `vector`。

4.10.2 二维数组

```
1 const int N = 1005;  
2 int grid[N][N];
```

如果规模不确定:

```
1 int n, m;  
2 cin >> n >> m;  
3 vector<vector<int>> grid(n, vector<int>(m, 0));
```

内存估算: `int a[10000][10000]` 大约 400MB, 通常危险。

4.10.3 初始化

全局普通数组默认是 0。如果要重置数组:

```
1 int a[1005];
2 memset(a, 0, sizeof a);
3 memset(a, -1, sizeof a);
```

注意：memset 适合把数组设为 0 或 -1，不适合把 int 数组设成 1、2、无穷大。
设置成大数：

```
1 const int INF = 0x3f3f3f3f;
2 int dist[1005];
3 memset(dist, 0x3f, sizeof dist);
```

vector 初始化：

```
1 vector<int> a(n, 0);
2 vector<long long> dist(n + 1, (long long)4e18);
3 vector<vector<int>> grid(n, vector<int>(m, 0));
```

4.11 vector

4.11.1 创建

```
1 vector<int> a; // empty
2 vector<int> b(10); // 10 zeros
3 vector<int> c(10, -1); // 10 values, all -1
```

4.11.2 常用操作

```
1 a.push_back(5);
2 a.pop_back();
3 a.clear();
4 int n = (int)a.size();
5 bool empty = a.empty();
```

访问：

```
1 cout << a[0] << '\n';
2 cout << a.back() << '\n';
```

访问前检查非空。a.back()、a[0] 在空数组上会出错。

4.11.3 遍历

```
1 for (int i = 0; i < (int)a.size(); i++) {
2     cout << a[i] << '\n';
3 }
4
5 for (int x : a) {
6     cout << x << '\n';
7 }
```

```

7 }
8
9 for (int& x : a) {
10     x++; // modify original element
11 }

```

4.11.4 删除元素

删除最后一个：

```
1 a.pop_back();
```

删除某个位置：

```
1 a.erase(a.begin() + pos);
```

注意：erase 是 $O(n)$ ，大量删除时可能超时。

删除所有等于 x 的元素：

```
1 a.erase(remove(a.begin(), a.end(), x), a.end());
```

4.11.5 二维 vector

二维网格：

```

1 int n, m;
2 cin >> n >> m;
3 vector<vector<int>> a(n, vector<int>(m));
4 for (int i = 0; i < n; i++) {
5     for (int j = 0; j < m; j++) {
6         cin >> a[i][j];
7     }
8 }

```

二维字符图：

```

1 int n, m;
2 cin >> n >> m;
3 vector<string> g(n);
4 for (int i = 0; i < n; i++) {
5     cin >> g[i];
6 }
7 cout << g[0][0] << '\n';

```

图的邻接表：

```

1 int n, m;
2 cin >> n >> m;
3 vector<vector<int>> g(n + 1);
4 for (int i = 0; i < m; i++) {
5     int u, v;

```

```
6   cin >> u >> v;
7   g[u].push_back(v);
8   g[v].push_back(u);
9 }
```

4.12 string

4.12.1 基本操作

```
1 string s = "abc";
2 int n = (int)s.size();
3 char c = s[0];
4 s.push_back('d');
5 s += "ef";
```

4.12.2 子串

```
1 string s = "abcdef";
2 string t = s.substr(2, 3); // "cde"
```

`substr(pos, len)` 从 `pos` 开始取 `len` 个字符，下标从 0 开始。

4.12.3 查找

```
1 string s = "abcdef";
2 if (s.find("cd") != string::npos) {
3     cout << "found\n";
4 }
```

不要写：

```
1 if (s.find("cd")) {
2     // wrong when position is 0
3 }
```

4.12.4 字符串转数字

```
1 int x = stoi("123");
2 long long y = stoll("1234567890123");
3 double z = stod("3.14");
```

数字转字符串：

```
1 string s = to_string(123);
```

如果数字很长，`stoi/stoll` 会溢出或异常，要用字符串模拟。

4.12.5 按空格分割

```
1 string line;
2 getline(cin, line);
3 stringstream ss(line);
4
5 string word;
6 while (ss >> word) {
7     cout << word << '\n';
8 }
```

4.13 pair 和 tuple

4.13.1 pair

```
1 pair<int, int> p = {3, 5};
2 cout << p.first << ' ' << p.second << '\n';
```

常用于坐标、边、距离和编号：

```
1 vector<pair<int, int>> points;
2 points.push_back({x, y});
```

默认排序：

```
1 sort(points.begin(), points.end());
```

默认先按 first 升序，再按 second 升序。

4.13.2 tuple

字段超过两个时可以用 tuple，但新手更推荐结构体。

```
1 tuple<int, int, int> t = {1, 2, 3};
2 auto [a, b, c] = t;
```

4.14 结构体 struct

结构体适合表示学生、任务、边、事件、状态。

```
1 struct Student {
2     int id;
3     string name;
4     int score;
5 };
6
7 Student s;
8 s.id = 1;
9 s.name = "Alice";
10 s.score = 90;
```

数组：

```
1 vector<Student> students;
2 students.push_back({1, "Alice", 90});
```

图的边：

```
1 struct Edge {
2     int to;
3     int w;
4 };
5
6 vector<vector<Edge>> g(n + 1);
7 g[u].push_back({v, w});
```

4.15 排序 sort

4.15.1 普通排序

```
1 sort(a.begin(), a.end()); // ascending
2 sort(a.rbegin(), a.rend()); // descending
```

数组排序：

```
1 sort(a, a + n);
```

4.15.2 自定义排序：从大到小

```
1 sort(a.begin(), a.end(), [](int x, int y) {
2     return x > y;
3 });
```

4.15.3 结构体排序

成绩高的在前；成绩相同，编号小的在前：

```
1 struct Student {
2     int id;
3     int score;
4 };
5
6 vector<Student> v;
7
8 sort(v.begin(), v.end(), [](const Student& a, const Student& b) {
9     if (a.score != b.score) return a.score > b.score;
10    return a.id < b.id;
11 });
```

4.15.4 按 vector 的某一行排序

```
1 vector<vector<int>> a;  
2  
3 sort(a.begin(), a.end(), [](const vector<int>& x, const vector<int>& y) {  
4     return x[0] < y[0]; // sort by column 0 ascending  
5 });
```

先按第 0 列升序，再按第 1 列降序：

```
1 sort(a.begin(), a.end(), [](const vector<int>& x, const vector<int>& y) {  
2     if (x[0] != y[0]) return x[0] < y[0];  
3     return x[1] > y[1];  
4 });
```

4.15.5 stable_sort

如果题目要求相同关键字保持原输入顺序：

```
1 stable_sort(v.begin(), v.end(), [](const Student& a, const Student& b) {  
2     return a.score > b.score;  
3 });
```

4.15.6 比较器高危错误

比较器必须表达“谁应该排在前面”。

正确：

```
1 sort(v.begin(), v.end(), [](const Student& a, const Student& b) {  
2     if (a.score != b.score) return a.score > b.score;  
3     return a.id < b.id;  
4 });
```

错误：

```
1 sort(v.begin(), v.end(), [](const Student& a, const Student& b) {  
2     return a.score >= b.score; // wrong: equality should return false  
3 });
```

相等时返回 true 可能导致排序行为错误。

4.16 二分相关函数

4.16.1 binary_search

```
1 sort(a.begin(), a.end());  
2 bool ok = binary_search(a.begin(), a.end(), x);
```

4.16.2 lower_bound 和 upper_bound

```
1 sort(a.begin(), a.end());
2
3 auto it1 = lower_bound(a.begin(), a.end(), x); // first >= x
4 auto it2 = upper_bound(a.begin(), a.end(), x); // first > x
```

下标:

```
1 int pos = lower_bound(a.begin(), a.end(), x) - a.begin();
```

解引用前检查:

```
1 auto it = lower_bound(a.begin(), a.end(), x);
2 if (it != a.end()) {
3     cout << *it << '\n';
4 }
```

4.17 map 和 unordered_map

4.17.1 map

map 按 key 自动升序排列，常用于字符串计数、编号映射、需要有序遍历的场景。

```
1 map<string, int> cnt;
2 cnt["apple"]++;
3 cnt["banana"] += 2;
4
5 cout << cnt["apple"] << '\n';
```

判断是否存在:

```
1 if (cnt.count("apple")) {
2     cout << "exists\n";
3 }
4
5 auto it = cnt.find("apple");
6 if (it != cnt.end()) {
7     cout << it->second << '\n';
8 }
```

注意: cnt[key] 会在 key 不存在时自动创建一个默认值。

遍历:

```
1 for (auto [key, value] : cnt) {
2     cout << key << ' ' << value << '\n';
3 }
```

4.17.2 unordered_map

unordered_map 平均更快，但没有顺序。

```
1 unordered_map<string, int> cnt;  
2 cnt["apple"]++;
```

考场建议：

- 需要按 key 排序输出，用 map。
- 只做计数且数据量大，用 unordered_map。
- 自定义 key 时新手优先考虑转成字符串或 long long 编码，不要现场写复杂 hash。

4.18 unordered_set

unordered_set 用来快速判断一个值是否出现过，不关心顺序。

```
1 unordered_set<int> seen;  
2 seen.insert(5);  
3  
4 if (seen.count(5)) {  
5     cout << "yes\n";  
6 }
```

字符串集合：

```
1 unordered_set<string> words;  
2 words.insert("abc");  
3 if (words.count("abc")) {  
4     cout << "exists\n";  
5 }
```

如果需要按升序遍历，用 set，不要用 unordered_set。

4.19 set 和 multiset

4.19.1 set

set 自动排序且不重复。

```
1 set<int> s;  
2 s.insert(3);  
3 s.insert(1);  
4 s.erase(3);  
5  
6 if (s.count(1)) {  
7     cout << "exists\n";  
8 }
```

查找第一个大于等于 x 的元素：

```
1 auto it = s.lower_bound(x);
2 if (it != s.end()) {
3     cout << *it << '\n';
4 }
```

4.19.2 multiset

multiset 允许重复元素。

```
1 multiset<int> s;
2 s.insert(5);
3 s.insert(5);
4 cout << s.count(5) << '\n'; // 2
```

删除一个 5:

```
1 auto it = s.find(5);
2 if (it != s.end()) s.erase(it);
```

删除所有 5:

```
1 s.erase(5);
```

4.20 queue、stack、deque

4.20.1 queue

BFS 常用队列。

```
1 queue<int> q;
2 q.push(1);
3 int x = q.front();
4 q.pop();
5 bool empty = q.empty();
```

4.20.2 stack

括号匹配、表达式、DFS 模拟可用栈。

```
1 stack<char> st;
2 st.push('(');
3 char c = st.top();
4 st.pop();
```

调用 front/top 前必须检查非空。

4.20.3 deque

双端队列，两边都能进出。

```
1 deque<int> dq;  
2 dq.push_back(1);  
3 dq.push_front(2);  
4 dq.pop_back();  
5 dq.pop_front();
```

单调队列会用到 `deque`。

4.21 priority_queue

4.21.1 大根堆

默认是大根堆，最大值在顶部。

```
1 priority_queue<int> pq;  
2 pq.push(3);  
3 pq.push(10);  
4 cout << pq.top() << '\n'; // 10  
5 pq.pop();
```

4.21.2 小根堆

```
1 priority_queue<int, vector<int>, greater<int>> pq;  
2 pq.push(3);  
3 pq.push(10);  
4 cout << pq.top() << '\n'; // 3
```

4.21.3 pair 小根堆

Dijkstra 常用：

```
1 using P = pair<long long, int>; // distance, node  
2 priority_queue<P, vector<P>, greater<P>> pq;  
3 pq.push({0, 1});
```

`pair` 的默认比较先看 `first`，再看 `second`。

4.22 常用算法函数

4.22.1 min、max、swap

```
1 int x = min(a, b);  
2 int y = max(a, b);  
3 swap(a, b);
```

4.22.2 reverse

```
1 reverse(a.begin(), a.end());
2 reverse(s.begin(), s.end());
```

4.22.3 unique 去重

若要去掉所有重复值且允许改变原顺序，先排序。`unique` 只消除相邻重复；要保留首次出现顺序，用集合配合一次扫描：

```
1 sort(a.begin(), a.end());
2 a.erase(unique(a.begin(), a.end()), a.end());
```

4.22.4 accumulate

```
1 long long sum = accumulate(a.begin(), a.end(), 0LL);
```

第三个参数写 0LL，否则可能按 int 求和。

4.22.5 gcd

```
1 // Require a,b >= 0 and lcm(a,b) <= LLONG_MAX.
2 long long g = gcd(a, b);
3 long long l = (a == 0 || b == 0) ? 0 : a / g * b;
```

最小公倍数先除后乘。这样比 $a * b / \text{gcd}(a, b)$ 更不容易溢出。

4.22.6 常用数学函数

```
1 double r = sqrt(x); // square root
2 double y = pow(a, b); // a^b, floating point
3 double f = floor(v); // round down as floating point
4 double c = ceil(v); // round up as floating point
5 long long z = llround(v); // nearest integer
```

提醒：

- `sqrt`、`pow` 返回浮点数，不适合直接处理大整数精确幂。
- 整数平方尽量写 `1LL * x * x`，不要写 `pow(x, 2)`。
- 判断一个整数是否为完全平方数时，先取 `sqrt`，再用整数乘法校验。
- 整数向上取整不要用 `ceil(1.0*a/b)` 代替整数公式。

```
1 bool isSquare(long long x) {
2     if (x < 0) return false;
3     long long r = sqrt((long double)x);
4     while (r > 0 && r > x / r) r--;
5     while ((r + 1) <= x / (r + 1)) r++;
6 }
```

```

6   return r * r == x;
7 }

```

4.23 lambda 入门

lambda 是临时函数，排序时非常常用。

最基本写法：

```

1 [](int a, int b) {
2     return a < b;
3 }

```

用于排序：

```

1 sort(a.begin(), a.end(), [](int x, int y) {
2     return x > y;
3 });

```

使用外部变量：

```

1 int mod = 10;
2 sort(a.begin(), a.end(), [mod](int x, int y) {
3     return x % mod < y % mod;
4 });

```

引用捕获：

```

1 int cnt = 0;
2 auto add = [&](int x) {
3     cnt += x;
4 };
5 add(5);

```

考场建议：

- 排序比较器中优先使用 `const Type&`，避免复制。
- 不熟悉捕获时，尽量不要写复杂 lambda。
- 复杂逻辑可以改成普通函数或先计算辅助字段。

4.24 常见编译错误和原因

报错或现象	常见原因	处理方法
expected ';'	少分号	看上一行结构体、变量定义、语句末尾。
was not declared	变量未定义或作用域不对	检查拼写和定义位置。
no matching function	函数参数类型不匹配	检查传参类型、引用、const。

报错或现象	常见原因	处理方法
segmentation fault	越界、空容器取值、递归爆栈	查数组下标、front/top/back 前是否非空。
样例正确但大数据错	溢出或复杂度过高	检查 long long 和复杂度。
输出格式错误	多输出、少换行、空格不对	严格对照题目输出格式。

4.25 C 和 C++ 混用提醒

如果你使用 C++，建议全程使用 cin/cout 或全程使用 scanf/printf，不要混用。

如果确实使用 C 风格输入输出：

```

1 int x;
2 scanf("%d", &x);
3
4 long long y;
5 scanf("%lld", &y);
6
7 printf("%d\n", x);
8 printf("%lld\n", y);

```

字符串：

```

1 char s[1005];
2 scanf("%s", s);

```

初学时可以先熟悉 string、vector 和 map，字符串和动态数组的存储会方便一些。

4.26 考场 C++ 快速检查

提交前快速看一遍：

- 是否包含了头文件和 using namespace std;。
- 是否开了快速输入输出。
- 是否混用了 cin/cout 和 scanf/printf。
- 求和、乘法、答案是否用了 long long。
- vector 是否按正确大小初始化。
- 访问 front/top/back 前是否判断非空。
- lower_bound 返回 end 时是否处理。
- 排序比较器相等时是否返回 false 或继续比较。
- 多组数据是否清空容器。
- 输出是否完全符合题意。

第五章 A 题决策树与高频模板

5.1 A 题定位

A 题多是扫描、统计或按规则模拟。做法想清楚后，花一点时间检查输入、下标和输出格式，这些地方很容易漏。

- 建议目标时间：20–35 分钟。
- 最大容忍时间：45 分钟。超过后仍无法通过样例，应先转下一题。
- 目标分数：100 分。
- 完成样例后，再试最小输入、重复值和边界。

常见失分点是没读完输入、整数溢出、输出格式不符，或把“包含端点”看成“不包含”。下面按题型列出需要留意的细节。

5.2 A 题开题流程

动手前把输入、输出和范围看完，再写代码：

1. 看输入格式：有几个数、几行、是否有字符串、是否有多组。
2. 看输出格式：输出一个数、一行多个数、字符串、小数、是否有特殊提示。
3. 看数据范围：是否需要 `long long`，数组要开多大。
4. 判断题型：统计、公式、排序、字符串、日期、进制、简单模拟。
5. 在草稿纸上写 1 到 2 个极端样例。
6. 写代码。
7. 过样例后，测试边界。

5.3 A 题读题标注法

读题时可以在草稿纸上简记输入数量、下标和边界规则，写代码时随手对照。

5.3.1 读题时要记下的信息

要圈的信息	为什么重要	例子
输入数量	决定循环次数	读 n 个数、读 n 行、读到 EOF。
数据范围	决定数组大小和复杂度	$n \leq 10^5$ 时不要双重循环。
数值范围	决定是否用 long long	每个数 10^9 ，总和可能溢出。
下标规则	决定 0 下标或 1 下标	第 1 个、第 i 个、编号从 0 开始。
区间端点	决定是否包含端点	$[l, r]$ 、 (l, r) 、从 l 到 r 。
输出格式	决定空格、换行、小数位	输出 YES，保留 2 位小数。
特殊情况	决定是否特判	无解、相等、为空、越界、最后一天。

5.3.2 草稿纸模板

做题前写 6 行：

```
1 Input:
2 Output:
3 n range:
4 value range:
5 Need long long? yes/no
6 Type: statistic / formula / sort / string / date / base / simulate
```

示例：

```
1 Input: n, then n integers
2 Output: sum and max
3 n range: 1e5
4 value range: 1e9
5 Need long long? yes
6 Type: statistic
```

5.3.3 样例手算方法

样例不是只拿来跑程序的。写代码前先手算一遍：

- 1. 把输入拆成变量。
- 2. 按题目规则一步一步算。
- 3. 写出中间结果。
- 4. 对比样例输出。

如果手算结果对不上样例，先回题面找漏掉的条件，再动手实现。

5.4 A 题总决策树

题面信号	优先判断为	去看
求和、平均、最大、最小、计数	简单统计	本章“统计模板”
根据公式算一个值	公式题	本章“公式题检查”
要求排序、排名、第几名	排序题	第4章排序；本章“排序排名”
出现字符、字符串、单词、大小写	字符串题	第4章字符串；本章“字符串处理”
出现二进制、十六进制、进制转换	进制题	本章“进制处理”
出现日期、时间、星期、天数	日期时间题	本章“日期时间”
出现坐标、网格、方向移动	简单模拟或网格模拟	本章“简单模拟”
出现百分比、保留小数、四舍五入	小数输出题	第4章小数输出
出现区间但数据很小	暴力枚举或前缀和	本章“区间入门”
出现重复统计、出现次数	计数数组或 map	第4章 map；本章“频率统计”

5.5 A 题关键词到动作

下表列出一些常见问法和对应写法。

你看到	立刻做什么	小心什么
“共 n 个”	开数组或 <code>vector</code> ，循环 n 次	循环是 $i < n$ 还是 $i \leq n$ 。
“编号从 1 到 n ”	优先开 $n+1$ ，按 1 下标写	不要访问 <code>a[0]</code> 。
“第 k 个”	注意 k 是 1 下标	<code>a[k-1]</code> 或排序后 <code>a[k-1]</code> 。
“最大/最小”	一遍扫描或排序	全负数时最大值不能初始化 0。
“总和/累计”	答案用 <code>long long</code>	<code>int</code> 乘法先溢出。
“平均”	判断输出整数还是小数	整数除法会截断。
“每 k 个一组” “分成若干页”	正整数向上取整	只在 $a \geq 0, b > 0$ 时用 $a/b + (a\%b != 0)$ 。
“保留 x 位”	用 <code>fixed << setprecision(x)</code>	显示位数由输出库处理；特殊舍入规则须自行实现。
“升序/降序”	使用 <code>sort</code>	降序可用 <code>greater<int>()</code> 。
“相同则按...”	写完整比较器	相等时不能返回 <code>true</code> 。
“字符串可能含空格”	使用 <code>getline</code>	前面用过 <code>cin</code> 要 <code>ignore</code> 。
“统计出现次数”	值域小用数组，值域大用 <code>map</code>	是否要求有序输出。

你看到	立刻做什么	小心什么
“判断是否存在”	用 <code>set/count</code> 或排序后二分	找不到时不要解引用迭代器。
“日期”	写闰年、每月天数	2 月和跨年。
“进制”	先判断是否超过 <code>long long</code>	大数不能 <code>stoll</code> 。
“区间 <code>[l,r]</code> ”	看有多少次查询	多次查询考虑前缀和。

5.6 A 题数据范围判断

看到的数据范围	含义	建议
$n \leq 100$	很小	直接模拟或双重循环通常可行。
$n \leq 1000$	小到中等	$O(n^2)$ 通常可试。
$n \leq 10^5$	常见大数组	需要 $O(n)$ 或 $O(n \log n)$, 不要双重循环。
数值 $\leq 10^9$	单个数可放 <code>int</code>	但加法和乘法可能要 <code>long long</code> 。
乘积、总和、答案可能很大	有溢出风险	默认用 <code>long long</code> 。
字符串长度很大	不能频繁插入删除	用线性扫描。

5.7 A 题常见输入形态

5.7.1 输入一个数

```

1 int x;
2 cin >> x;
3 cout << x << '\n';

```

5.7.2 输入两个或多个数

```

1 int a, b, c;
2 cin >> a >> b >> c;
3 cout << a + b + c << '\n';

```

5.7.3 输入 n 个数

```

1 int n;
2 cin >> n;
3 vector<int> a(n);

```

```
4 for (int i = 0; i < n; i++) {  
5     cin >> a[i];  
6 }
```

5.7.4 输入 n 行记录

例如每行一个学生的编号和成绩：

```
1 struct Student {  
2     int id;  
3     int score;  
4 };  
5  
6 int n;  
7 cin >> n;  
8 vector<Student> a(n);  
9 for (int i = 0; i < n; i++) {  
10     cin >> a[i].id >> a[i].score;  
11 }
```

5.7.5 输入 n 行字符串

```
1 int n;  
2 cin >> n;  
3 vector<string> s(n);  
4 for (int i = 0; i < n; i++) {  
5     cin >> s[i];  
6 }
```

5.7.6 读到文件结束

```
1 int x;  
2 while (cin >> x) {  
3     // process x  
4 }
```

题目没有说有 T 组数据时，不要自己加 T 。

5.8 简单统计

5.8.1 什么题会用到

题面出现：

- 求总和、平均数。
- 统计满足条件的个数。

- 找最大值、最小值。
- 统计正数、负数、奇数、偶数。
- 统计超过某个阈值的元素。

5.8.2 一遍扫描模板

```
1 int n;
2 cin >> n;
3
4 long long sum = 0;
5 int cnt = 0;
6 int mx = numeric_limits<int>::lowest();
7 int mn = numeric_limits<int>::max();
8
9 for (int i = 0; i < n; i++) {
10     int x;
11     cin >> x;
12     sum += x;
13     if (x > mx) mx = x;
14     if (x < mn) mn = x;
15     if (x % 2 == 0) cnt++;
16 }
17
18 cout << sum << ' ' << mx << ' ' << mn << ' ' << cnt << '\n';
```

5.8.3 注意事项

- 求和变量用 long long。
- 最大值、最小值初始值要足够小或足够大。
- 如果数据可能全是负数，最大值不能初始化为 0。
- 平均数如果要小数，使用 double 或输出时转换。

5.8.4 统计满足条件的个数

```
1 int n;
2 cin >> n;
3 int cnt = 0;
4 for (int i = 0; i < n; i++) {
5     int x;
6     cin >> x;
7     if (x >= 60) {
8         cnt++;
9     }
10 }
11 cout << cnt << '\n';
```

5.8.5 同时统计多个类别

例如统计正数、负数、零：

```
1 int n;
2 cin >> n;
3 int pos = 0, neg = 0, zero = 0;
4 for (int i = 0; i < n; i++) {
5     int x;
6     cin >> x;
7     if (x > 0) pos++;
8     else if (x < 0) neg++;
9     else zero++;
10 }
11 cout << pos << ' ' << neg << ' ' << zero << '\n';
```

5.8.6 找最大值的位置

```
1 int n;
2 cin >> n;
3 vector<int> a(n);
4 for (int i = 0; i < n; i++) cin >> a[i];
5
6 int bestPos = 0;
7 for (int i = 1; i < n; i++) {
8     if (a[i] > a[bestPos]) {
9         bestPos = i;
10    }
11 }
12 cout << bestPos << ' ' << a[bestPos] << '\n';
```

如果题目说“第几个”，通常要输出 `bestPos + 1`。

5.8.7 计数数组

当值域很小，例如分数在 0 到 100：

```
1 int n;
2 cin >> n;
3 vector<int> cnt(101, 0);
4 for (int i = 0; i < n; i++) {
5     int score;
6     cin >> score;
7     cnt[score]++;
8 }
9
10 for (int s = 0; s <= 100; s++) {
11     if (cnt[s] > 0) {
12         cout << s << ' ' << cnt[s] << '\n';
```

```
13     }  
14 }
```

计数数组前提：数值不能是很大的 10^9 ，也不能是负数，除非你做了偏移。

5.9 存在、全部、任意判断

A 题常见问法：

- 是否存在一个数满足条件。
- 是否所有数都满足条件。
- 是否至少有一个字符串包含某内容。
- 是否没有任何违规情况。

5.9.1 判断是否存在

```
1 int n;  
2 cin >> n;  
3 bool found = false;  
4 for (int i = 0; i < n; i++) {  
5     int x;  
6     cin >> x;  
7     if (x == 0) {  
8         found = true;  
9     }  
10 }  
11  
12 cout << (found ? "YES" : "NO") << '\n';
```

5.9.2 判断是否全部满足

```
1 int n;  
2 cin >> n;  
3 bool ok = true;  
4 for (int i = 0; i < n; i++) {  
5     int x;  
6     cin >> x;  
7     if (x < 60) {  
8         ok = false;  
9     }  
10 }  
11  
12 cout << (ok ? "YES" : "NO") << '\n';
```

5.9.3 提前结束

如果后面的输入还必须读完，不要随便 `break`。除非你确定后续数据不需要再读，或者已经读完本组数据。

```

1 bool found = false;
2 for (int i = 0; i < n; i++) {
3     int x;
4     cin >> x;
5     if (x == target) {
6         found = true;
7     }
8 }
```

判断已经成立时仍可继续读入，只把结果记下来；这样不容易把下一组输入读错。

5.10 公式题

5.10.1 什么题会用到

题面给出明确计算规则，例如：

- 根据若干输入计算费用、得分、距离、面积。
- 分段收费。
- 按比例、百分比、折扣计算。
- 对结果取整、向上取整、向下取整。

5.10.2 公式题流程

1. 先在草稿纸上写出数学式。
2. 判断每个中间变量是否可能溢出。
3. 判断除法是整数除法还是小数除法。
4. 判断是否需要四舍五入、向上取整、向下取整。
5. 用样例手算一遍。

5.10.3 整数除法和取整

公式题中最容易错的是“除法到底按什么规则取整”。先分清三种情况：

题目要求	常用写法	适用条件
普通整数除法，向 0 截断	a / b	C++ 默认规则，负数时不是向下取整。
正整数向上取整	$a/b + (a \% b != 0)$	只适合 $a \geq 0, b > 0$ 。

题目要求	常用写法	适用条件
数学意义向下/向上取整	使用 <code>floor_div</code> / <code>ceil_div</code>	适合可能出现负数的公式。
整		

正整数向上取整最常见，例如 a 个物品每组放 b 个：

```
1 long long ceil_div_positive(long long a, long long b) {
2     return a / b + (a % b != 0);
3 }
```

若 a 可能为负，或题目明确要求数学意义的向下/向上取整，使用通用写法：

```
1 // Require b != 0; exclude LLONG_MIN / -1.
2 long long floor_div(long long a, long long b) {
3     long long q = a / b;
4     long long r = a % b;
5     if (r != 0 && ((r > 0) != (b > 0))) q--;
6     return q;
7 }
8
9 long long ceil_div(long long a, long long b) {
10    long long q = a / b;
11    long long r = a % b;
12    if (r != 0 && ((r > 0) == (b > 0))) q++;
13    return q;
14 }
```

不要随使用浮点数做整数取整题。如果题目全是整数，通常可以用整数公式避免误差；特别是数值接近 10^{18} 时，`double` 可能无法精确表示每一个整数。

5.10.4 分段计算模板

```
1 int x;
2 cin >> x;
3
4 int ans = 0;
5 if (x <= 10) {
6     ans = x * 2;
7 } else if (x <= 20) {
8     ans = 10 * 2 + (x - 10) * 3;
9 } else {
10    ans = 10 * 2 + 10 * 3 + (x - 20) * 5;
11 }
12
13 cout << ans << '\n';
```

注意：

- 边界是 `<=` 还是 `<`。

- 每一段是否包含端点。
- 前面几段的费用是否已经累计。

5.10.5 分类讨论模板

A 题经常给出若干规则，按条件输出不同结果。

```

1 int x;
2 cin >> x;
3
4 if (x < 0) {
5     cout << "negative\n";
6 } else if (x == 0) {
7     cout << "zero\n";
8 } else {
9     cout << "positive\n";
10 }
```

分类讨论检查：

- 条件是否覆盖所有情况。
- 条件顺序是否正确。
- 边界值属于哪一类。
- 是否存在重叠条件。

5.10.6 绝对值和距离

```

1 int a, b;
2 cin >> a >> b;
3 cout << labs(1LL * a - b) << '\n';
```

如果 a, b 很大：

```

1 long long a, b;
2 cin >> a >> b;
3 cout << llabs(a - b) << '\n';
```

5.10.7 取模

```

1 long long ans = 0;
2 const long long MOD = 1000000007;
3 ans = (ans + x) % MOD;
4 ans = ans * y % MOD;
```

如果可能出现负数：

```

1 long long r = x % MOD;
2 if (r < 0) r += MOD;
```

5.10.8 小数和百分比

保留小数：

```
1 double x;
2 cin >> x;
3 cout << fixed << setprecision(2) << x << '\n';
```

整数转百分比：

```
1 int good, total;
2 cin >> good >> total;
3 double rate = 100.0 * good / total;
4 cout << fixed << setprecision(2) << rate << "%\n";
```

平均数：

```
1 long long sum = 0;
2 int n;
3 cin >> n;
4 for (int i = 0; i < n; i++) {
5     int x;
6     cin >> x;
7     sum += x;
8 }
9 double avg = 1.0 * sum / n;
10 cout << fixed << setprecision(1) << avg << '\n';
```

注意： sum / n 如果两边都是整数，会做整数除法。写成 $1.0 * \text{sum} / n$ 。

5.11 排序和排名

5.11.1 什么题会用到

题面出现：

- 按成绩排序、按编号排序、按时间排序。
- 输出第 k 个。
- 排名、名次、优先级。
- 相同分数时按另一个条件排序。

5.11.2 单个数组排序

```
1 int n;
2 cin >> n;
3 vector<int> a(n);
4 for (int i = 0; i < n; i++) cin >> a[i];
5
6 sort(a.begin(), a.end());
```

```
7
8 for (int x : a) {
9     cout << x << ' ';
10 }
11 cout << '\n';
```

降序:

```
1 sort(a.begin(), a.end(), greater<int>());
```

5.11.3 结构体排序

```
1 struct Student {
2     int id;
3     int score;
4 };
5
6 int n;
7 cin >> n;
8 vector<Student> a(n);
9 for (int i = 0; i < n; i++) {
10     cin >> a[i].id >> a[i].score;
11 }
12
13 sort(a.begin(), a.end(), [](const Student& x, const Student& y) {
14     if (x.score != y.score) return x.score > y.score;
15     return x.id < y.id;
16 });
17
18 for (const auto& s : a) {
19     cout << s.id << ' ' << s.score << '\n';
20 }
```

5.11.4 排名常见规则

排名题一定要看清楚“并列名次”。

普通排序后第 k 个:

```
1 sort(a.begin(), a.end(), greater<int>());
2 cout << a[k - 1] << '\n';
```

并列排名示例:

```
1 vector<int> score = {100, 90, 90, 80};
2 int rank = 1;
3 for (int i = 0; i < (int)score.size(); i++) {
4     if (i > 0 && score[i] != score[i - 1]) {
5         rank = i + 1;
6     }
7 }
```

```
7     cout << score[i] << ' ' << rank << '\n';
8 }
```

排序题高危点：比较器里相等时不能返回 `true`，需要继续比较或返回 `false`。

5.11.5 按字符串字典序排序

```
1 int n;
2 cin >> n;
3 vector<string> s(n);
4 for (int i = 0; i < n; i++) cin >> s[i];
5
6 sort(s.begin(), s.end());
7
8 for (string x : s) {
9     cout << x << '\n';
10 }
```

5.11.6 先排序再去重

```
1 sort(a.begin(), a.end());
2 a.erase(unique(a.begin(), a.end()), a.end());
```

5.11.7 找第 k 小和第 k 大

```
1 sort(a.begin(), a.end());
2 cout << a[k - 1] << '\n'; // kth smallest
3
4 sort(a.begin(), a.end(), greater<int>());
5 cout << a[k - 1] << '\n'; // kth largest
```

第 k 个通常是 1 下标，所以用 $k - 1$ 。

5.12 字符串处理

5.12.1 什么题会用到

题面出现：

- 字符、单词、大小写。
- 判断前缀、后缀、是否包含。
- 统计某字符出现次数。
- 按空格、逗号、冒号分割。
- 文件路径、命令、表达式。

5.12.2 逐字符扫描

```
1 string s;
2 cin >> s;
3
4 int digits = 0;
5 int letters = 0;
6 for (char c : s) {
7     if (isdigit((unsigned char)c)) digits++;
8     if (isalpha((unsigned char)c)) letters++;
9 }
10
11 cout << digits << ' ' << letters << '\n';
```

5.12.3 大小写转换

```
1 string s;
2 cin >> s;
3 for (char& c : s) {
4     c = tolower((unsigned char)c);
5 }
6 cout << s << '\n';
```

5.12.4 查找子串

```
1 string s, t;
2 cin >> s >> t;
3 if (s.find(t) != string::npos) {
4     cout << "YES\n";
5 } else {
6     cout << "NO\n";
7 }
```

5.12.5 按空格分割一整行

```
1 string line;
2 getline(cin, line);
3
4 stringstream ss(line);
5 string word;
6 while (ss >> word) {
7     cout << word << '\n';
8 }
```

如果前面读过整数：

```
1 int n;
2 cin >> n;
3 cin.ignore(numeric_limits<streamsize>::max(), '\n');
4
5 string line;
6 getline(cin, line);
```

5.12.6 按指定字符分割

```
1 vector<string> split(const string& s, char sep) {
2     vector<string> res;
3     string cur;
4     for (char c : s) {
5         if (c == sep) {
6             res.push_back(cur);
7             cur.clear();
8         } else {
9             cur.push_back(c);
10        }
11    }
12    res.push_back(cur);
13    return res;
14 }
```

字符串题常见坑：空字符串、连续分隔符、行尾换行、下标从 0 开始。

5.12.7 判断前缀和后缀

前缀：

```
1 bool startsWith(const string& s, const string& p) {
2     if (p.size() > s.size()) return false;
3     for (int i = 0; i < (int)p.size(); i++) {
4         if (s[i] != p[i]) return false;
5     }
6     return true;
7 }
```

后缀：

```
1 bool endsWith(const string& s, const string& p) {
2     if (p.size() > s.size()) return false;
3     int offset = (int)s.size() - (int)p.size();
4     for (int i = 0; i < (int)p.size(); i++) {
5         if (s[offset + i] != p[i]) return false;
6     }
7     return true;
8 }
```

5.12.8 统计每个小写字母

```
1 string s;  
2 cin >> s;  
3 vector<int> cnt(26, 0);  
4 for (char c : s) {  
5     if ('a' <= c && c <= 'z') {  
6         cnt[c - 'a']++;  
7     }  
8 }
```

5.12.9 反转字符串

```
1 string s;  
2 cin >> s;  
3 reverse(s.begin(), s.end());  
4 cout << s << '\n';
```

5.12.10 回文判断

```
1 bool isPalindrome(const string& s) {  
2     int l = 0, r = (int)s.size() - 1;  
3     while (l < r) {  
4         if (s[l] != s[r]) return false;  
5         l++;  
6         r--;  
7     }  
8     return true;  
9 }
```

5.13 进制处理

5.13.1 什么题会用到

题面出现：

- 二进制、八进制、十六进制。
- 把某进制转为十进制。
- 把十进制转为某进制。
- 位、编码、补码、掩码。

5.13.2 任意进制转十进制

```

1 // Valid nonempty digits; 2 <= base <= 16; result <= LLONG_MAX.
2 long long toDecimal(const string& s, int base) {
3     long long ans = 0;
4     for (char c : s) {
5         int v;
6         if ('0' <= c && c <= '9') v = c - '0';
7         else if ('A' <= c && c <= 'F') v = c - 'A' + 10;
8         else v = c - 'a' + 10;
9         ans = ans * base + v;
10    }
11    return ans;
12 }

```

5.13.3 十进制转任意进制

```

1 // Require x >= 0 and 2 <= base <= 16.
2 string fromDecimal(long long x, int base) {
3     if (x == 0) return "0";
4     string digits = "0123456789ABCDEF";
5     string res;
6     while (x > 0) {
7         res.push_back(digits[x % base]);
8         x /= base;
9     }
10    reverse(res.begin(), res.end());
11    return res;
12 }

```

5.13.4 位运算入门

```

1 if (x & 1) {
2     // x is odd
3 }
4
5 if (x & (1 << k)) {
6     // bit k is 1
7 }
8
9 x |= (1 << k); // set bit k to 1
10 x &= ~(1 << k); // set bit k to 0

```

如果 $k \geq 31$, 写 $1LL \ll k$, 不要写 $1 \ll k$ 。

5.13.5 进制题检查

- 输入数字是否可能很长。

- 转成十进制是否超过 `long long`。
- 十六进制字母是大写还是小写。
- 输出是否要求补前导零。
- 二进制位编号从 0 还是 1 开始。

5.14 日期时间

5.14.1 什么题会用到

题面出现：

- 年、月、日、星期。
- 给定日期求过几天。
- 两个日期之间的天数。
- 时间格式如 HH:MM:SS。

5.14.2 闰年

```
1 bool leap(int y) {
2     return (y % 400 == 0) || (y % 4 == 0 && y % 100 != 0);
3 }
```

5.14.3 每月天数

```
1 int mdays(int y, int m) {
2     int d[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};
3     if (m == 2 && leap(y)) return 29;
4     return d[m];
5 }
```

5.14.4 下一天

```
1 void nextDay(int& y, int& m, int& d) {
2     d++;
3     if (d > mdays(y, m)) {
4         d = 1;
5         m++;
6         if (m > 12) {
7             m = 1;
8             y++;
9         }
10    }
11 }
```

5.14.5 时间转秒

```
1 int toSeconds(int h, int m, int s) {  
2     return h * 3600 + m * 60 + s;  
3 }
```

日期题先看范围。如果只跨几年，可以逐日模拟；如果年份很大，要转为天数公式或按周期处理。

5.15 简单模拟

5.15.1 什么题会用到

题面出现：

- 按规则一步一步执行。
- 有当前位置、方向、状态。
- 有若干操作命令。
- 有网格移动，但不要求最短路。
- 有游戏规则、计分规则。

5.15.2 方向数组

四方向：

```
1 int dx[4] = {-1, 0, 1, 0};  
2 int dy[4] = {0, 1, 0, -1};
```

八方向：

```
1 int dx[8] = {-1,-1,-1,0,0,1,1,1};  
2 int dy[8] = {-1,0,1,-1,1,-1,0,1};
```

判断是否在网格内：

```
1 bool inside(int x, int y, int n, int m) {  
2     return 0 <= x && x < n && 0 <= y && y < m;  
3 }
```

5.15.3 按命令移动

```
1 int x = 0, y = 0;  
2 string ops;  
3 cin >> ops;  
4  
5 for (char op : ops) {  
6     if (op == 'U') x--;  
7     else if (op == 'D') x++;  
8     else if (op == 'L') y--;
```

```
9     else if (op == 'R') y++;
10 }
11
12 cout << x << ' ' << y << '\n';
```

5.15.4 模拟题写法建议

- 状态变量先列出来，例如位置、方向、分数、时间。
- 每个操作只改变少量状态。
- 复杂规则拆成函数。
- 样例过后自己造最小边界和最大边界。

5.16 坐标和距离入门

A 题中若出现坐标，通常是简单计算，不一定是图搜索。

5.16.1 二维点

```
1 struct Point {
2     long long x;
3     long long y;
4 };
5
6 Point a, b;
7 cin >> a.x >> a.y >> b.x >> b.y;
```

5.16.2 曼哈顿距离

只能上下左右走时，常见距离：

```
1 long long dist = llabs(a.x - b.x) + llabs(a.y - b.y);
2 cout << dist << '\n';
```

5.16.3 欧几里得距离

直线距离：

```
1 double dx = a.x - b.x;
2 double dy = a.y - b.y;
3 double dist = sqrt(dx * dx + dy * dy);
4 cout << fixed << setprecision(6) << dist << '\n';
```

5.16.4 坐标题检查

- 坐标可能为负数。
- 坐标差的平方可能溢出，必要时用 long long 或 double。
- 题目要求的是曼哈顿距离还是直线距离。
- 输出是否需要小数。

5.17 二维表格和矩阵入门

A 题可能出现很简单的二维表格，例如统计每行和、每列和、找最大值。

5.17.1 读入 n 行 m 列

```
1 int n, m;
2 cin >> n >> m;
3 vector<vector<int>> a(n, vector<int>(m));
4 for (int i = 0; i < n; i++) {
5     for (int j = 0; j < m; j++) {
6         cin >> a[i][j];
7     }
8 }
```

5.17.2 每行求和

```
1 for (int i = 0; i < n; i++) {
2     long long sum = 0;
3     for (int j = 0; j < m; j++) {
4         sum += a[i][j];
5     }
6     cout << sum << '\n';
7 }
```

5.17.3 每列求和

```
1 for (int j = 0; j < m; j++) {
2     long long sum = 0;
3     for (int i = 0; i < n; i++) {
4         sum += a[i][j];
5     }
6     cout << sum << '\n';
7 }
```

5.17.4 找矩阵最大值

```
1 int bx = 0, by = 0;
2 for (int i = 0; i < n; i++) {
3     for (int j = 0; j < m; j++) {
4         if (a[i][j] > a[bx][by]) {
5             bx = i;
6             by = j;
7         }
8     }
9 }
10 cout << bx << ' ' << by << ' ' << a[bx][by] << '\n';
```

如果题目要求第几行第几列，通常输出 $bx + 1$ 和 $by + 1$ 。

5.18 频率统计

5.18.1 值域小：用数组

如果数值范围是 0 到 1000：

```
1 int cnt[1001] = {};
2 int n;
3 cin >> n;
4 for (int i = 0; i < n; i++) {
5     int x;
6     cin >> x;
7     cnt[x]++;
8 }
```

5.18.2 值域大或 key 是字符串：用 map

```
1 int n;
2 cin >> n;
3 map<string, int> cnt;
4 for (int i = 0; i < n; i++) {
5     string s;
6     cin >> s;
7     cnt[s]++;
8 }
9
10 for (auto [s, c] : cnt) {
11     cout << s << ' ' << c << '\n';
12 }
```

5.18.3 只需要快速统计：unordered_map

```
1 unordered_map<int, int> cnt;
2 for (int x : a) {
3     cnt[x]++;
4 }
```

如果题目要求按字典序输出，不能用 `unordered_map` 直接遍历。

5.19 区间入门

A 题中也可能出现简单区间。先看数据范围。

5.19.1 数据小：直接循环

```
1 int l, r;
2 cin >> l >> r;
3 long long sum = 0;
4 for (int i = l; i <= r; i++) {
5     sum += a[i];
6 }
```

5.19.2 多次区间和：前缀和

```
1 int n, q;
2 cin >> n >> q;
3 vector<long long> pre(n + 1, 0);
4 for (int i = 1; i <= n; i++) {
5     long long x;
6     cin >> x;
7     pre[i] = pre[i - 1] + x;
8 }
9
10 while (q--) {
11     int l, r;
12     cin >> l >> r;
13     cout << pre[r] - pre[l - 1] << '\n';
14 }
```

注意：

- 上面模板假设下标从 1 开始。
- 如果题目给的是 0 下标，要改公式。
- 前缀和数组用 `long long`。

5.20 A 题输出格式专项

提交前逐项对照：

- 题目要求输出 YES/NO 还是 Yes/No。
- 是否每个答案一行。
- 多个数之间是空格还是换行。
- 最后一项后面是否允许空格。一般允许，但不依赖。
- 小数保留几位。
- 是否要求百分号、冒号、逗号等符号。

输出数组推荐写法：

```
1 for (int i = 0; i < n; i++) {
2     if (i) cout << ' ';
3     cout << a[i];
4 }
5 cout << '\n';
```

5.21 A 题新手错误对照表

现象	可能原因	快速检查
样例输出差一个数	循环次数错	检查 $i < n$ 和 $i \leq n$ 。
样例数字对但格式错	空格、换行、大小写错误	对照输出格式逐字符看。
小数据对，大数据错	int 溢出	把和、乘积、答案改为 long long。
排序后顺序怪	比较器错误	相等时返回 false 或继续比较。
读入字符串为空	cin 后直接 getline	加 cin.ignore()。
程序崩溃	数组越界或空容器取值	查 a[i]、front/top/back。
答案少算最后一项	区间端点没包含	检查 $\leq r$ 还是 $< r$ 。
答案多算一项	下标起点错	检查 0 下标和 1 下标是否混用。
找子串时明明在开头却判断失败	错用 if (s.find(t))	改为和 string::npos 比较。
多组数据第二组错	没清空状态	每组重置数组、map、答案。

5.22 A 题推荐代码组织

简单题可以直接写在 main，但如果题目有多组数据，推荐写 solve()。

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 void solve() {
```

```
6   int n;
7   cin >> n;
8   vector<int> a(n);
9   for (int i = 0; i < n; i++) {
10      cin >> a[i];
11  }
12
13  ll sum = 0;
14  for (int x : a) {
15      sum += x;
16  }
17  cout << sum << '\n';
18 }
19
20 int main() {
21     ios::sync_with_stdio(false);
22     cin.tie(nullptr);
23
24     solve();
25     return 0;
26 }
```

如果题目有 T 组：

```
1  int main() {
2      ios::sync_with_stdio(false);
3      cin.tie(nullptr);
4
5      int T;
6      cin >> T;
7      while (T--) {
8          solve();
9      }
10     return 0;
11 }
```

如果题目没有多组数据，不要自己读一个 T 。

5.23 A 题自造边界样例

样例通过后，至少测试：

- 最小输入： $n = 0$ 或 $n = 1$ ，如果题目允许。
- 最大输入附近：最大数值、最大长度。
- 全相同数据。
- 全为 0。
- 全为负数，如果题目允许。

- 排序中有相同关键字。
- 字符串为空或只有一个字符，如果题目允许。
- 日期为 2 月 28 日、2 月 29 日、12 月 31 日。

5.24 A 题本地调试流程

样例不过时，可以先检查读入，再检查中间结果和边界。

5.24.1 第一步：确认读入

临时输出读到的变量：

```
1 cerr << "n=" << n << '\n';
2 for (int i = 0; i < n; i++) {
3     cerr << "a[" << i << "]= " << a[i] << '\n';
4 }
```

提交前必须删除或注释调试输出。虽然 `cerr` 通常不进入标准输出，但考场上不要依赖这一点，最终代码保持干净。

5.24.2 第二步：确认中间结果

例如统计题：

```
1 cerr << "sum=" << sum << " cnt=" << cnt << '\n';
```

排序题：

```
1 for (int x : a) cerr << x << ' ';
2 cerr << '\n';
```

5.24.3 第三步：用最小样例

把输入改成最小情况：

```
1 1
2 5
```

如果最小样例都错，通常是读入、初始化或输出格式错。

5.24.4 第四步：检查边界

- 循环是否少跑一次。
- 数组是否越界。
- 是否把 n 当成最大下标。
- 是否忘记处理最后一个元素。
- 是否忘记初始化答案变量。

5.25 A 题常见边界库

做完样例后，根据题型选 2 到 3 个边界测。

题型	建议边界	目的
统计	$n = 1$, 全 0, 全负数, 全相同	检查初始化和条件判断。
最大最小	全负数、最大值在第一项、最大值在最后一项	检查初始值和循环。
排序	已有序、逆序、全相等、有重复关键字	检查比较器和稳定性。
字符串	长度 1、全数字、全字母、含空格、子串在开头	检查读取和 <code>find</code> 。
日期	2 月 28 日、闰年 2 月 29 日、12 月 31 日	检查闰年和跨年。
进制	0、最大位、含 A-F、小写字母	检查特殊值和字符转换。
区间	$l = r$, $l = 1$, $r = n$	检查前缀和端点。
二维表	1×1 , 单行, 单列	检查双重循环。

5.26 A 题模板练习清单

这些模板建议单独练到不用看也能写。

模板	熟练程度	关联章节
读入 n 个数	建议熟练掌握	输入形态、vector
一遍求和、最大、最小	建议熟练掌握	简单统计
统计满足条件的个数	建议熟练掌握	简单统计
结构体排序	能对照写出	排序和排名
字符串逐字符扫描	建议熟练掌握	字符串处理
<code>getline</code> + <code>stringstream</code>	能对照写出	字符串处理
进制转十进制	照着能写	进制处理
闰年和每月天数	照着能写	日期时间
方向数组	照着能写	简单模拟
二维矩阵读入	建议熟练掌握	二维表格
前缀和查询	照着能写	区间入门

5.27 A 题实现顺序

一时拿不准从哪里开始时，可以先做下面几步：

1. 先保证读入正确。
2. 再保证样例能手算清楚。

3. 写出范围允许的直接做法。
4. 再处理边界。
5. 运行后再检查输出和边界。

两种做法都能满足时限，选自己更熟悉、也更容易查错的那一种即可。

5.28 A 题提交前总检查

1. 输入是否完整读入。
2. 输出是否完全符合题目。
3. 是否有调试输出。
4. 是否需要 `long long`。
5. 数组是否越界。
6. 下标从 0 还是 1 开始是否统一。
7. 排序比较器是否正确。
8. 字符串读取是否受空格影响。
9. 小数精度是否正确。
10. 边界样例是否测试过。

5.29 A 题卡住时怎么办

- 读不懂题：重看输入输出样例，手算样例。
- 样例不过：打印中间变量在本地查错，提交前删掉。
- 怀疑溢出：把答案和中间量改成 `long long`。
- 怀疑下标：画出长度为 3 的小数组，手动走一遍。
- 超过 45 分钟仍未解决：保存代码，先做 B 或 C，之后回来。

第六章 B 题决策树与高频算法入口

6.1 B 题定位

B 题常用到前缀和、差分、排序、查找或简单 DP。直接照题意模拟之前，先算一下操作次数：数据小时暴力就够，数据大时再找重复计算。

- 建议目标时间：40–70 分钟。
- 目标分数：尽量 100 分。
- 高频主题：排序、哈希统计、前缀和、差分、二分、简单 BFS、简单图、简单 DP。
- 留意暴力的最坏复杂度，样例规模通常很小。

同样是双重循环， $n=100$ 和 $n=100000$ 差别很大。先代入最大范围估算，再决定是否需要优化。

6.2 B 题开题流程

1. 圈出数据范围： n, m, q 分别多大。
2. 圈出操作次数：是否有很多查询、很多修改。
3. 判断暴力复杂度：如果每次操作都扫描一遍，总次数是多少。
4. 找题面信号：区间、排序、统计、最短、连通、依赖、答案范围。
5. 查 B 题决策树，定位高频模板。
6. 如果满分做法明确，直接写；如果不明确，先写可拿部分分的暴力。

6.3 B 题总决策树

题面信号	优先怀疑	首选模板
多次静态区间和 / 区间最值	前缀和 / 最值数据结构	和用 T101¹⁷⁷ / T102¹⁷⁷ ；最值需另建结构，不能相减
多次区间加，最后查询结果	差分	一维/二维差分
区间修改和区间查询交替出现	树状数组或线段树	数据结构章节，B 题先看是否可简化
统计出现次数、找重复、频率	哈希统计	数组计数、map、unordered_map

题面信号	优先怀疑	首选模板
排序后选择、排名、贪心安排	排序与扫描	sort + 自定义比较器
前后两边信息、删除一个元素后的最优值	前后缀预处理	前缀最大/最小、后缀最大/最小
余数、整除、模 k 相同	余数统计	cnt[mod]、map 统计
第一个满足条件、答案具有单调性	二分或二分答案	lower_bound / check
最少步数、网格移动、迷宫	BFS	队列、距离数组
合并集合、判断是否同组	并查集	DSU
依赖关系、先后顺序	拓扑排序	入度 + 队列
小规模任意两点距离	Floyd	三重循环
非负权图最短路	Dijkstra	堆优化 Dijkstra
选择若干、容量限制、最优值	简单 DP	背包或线性 DP

6.4 B 题数据范围判断

数据范围	暴力是否可行	常见替代
$n \leq 1000$	$O(n^2)$ 常常可行	双重循环、简单 DP
$n \leq 10^5$	$O(n^2)$ 通常不可行	排序、哈希、前缀和、二分
$q \leq 10^5$ 且每次查区间	每次扫描不可行	前缀和、树状数组、线段树
二维 $n, m \leq 1000$	$O(nm)$ 可行, $O(n^2m^2)$ 不可行	二维前缀和
图点数 ≤ 500	Floyd 可能可行	Floyd
图点数、边数 $\leq 10^5$	Floyd 不可行	BFS、Dijkstra、并查集
值域 $\leq 10^6$	可用计数数组	数组计数
值域很大或字符串 key	不能开值域数组	map / unordered_map

6.5 B 题关键词到动作

看到	立刻想	小心
“多次询问”	预处理，不能每次从头算	查询次数 q 是否很大。
“区间 $[l, r]$ 的和”	前缀和	下标和 <code>pre[r]-pre[l-1]</code> 。
“子矩阵和”	二维前缀和	公式四项别写错。
“区间加”	差分	右端点后一位是否存在。
“出现次数”	计数数组或 map	值域是否能开数组。
“去重后数量”	set 或 sort+unique	是否需要保持顺序。

看到	立刻想	小心
“第一个不小于”	lower_bound	容器必须有序。
“最小的最大值”	二分答案	必须能写 check。
“删除一个后”“左边右边”	前后缀预处理	注意边界位置。
“除以 k 的余数”“能否整除”	余数统计	负数取模要修正。
“尽量多选”“最早结束”“最少次数覆盖”	贪心	必须按正确关键字排序。
“最短步数”	BFS	BFS 适合每步代价相同。
“连通、合并”	并查集	初始化每个点的父亲。
“前置条件、依赖”	拓扑排序	入度更新。

6.6 从暴力到优化的转换表

已有暴力思路时，可以对照下表，看看重复扫描、求和或查找能否省掉。

暴力想法	为什么危险	常见优化
每次查询都遍历 $[l, r]$	$O(nq)$, q 大时超时	前缀和、树状数组
每次区间加都逐个元素修改	$O(nq)$	差分、线段树
双重循环找两数关系	$O(n^2)$	排序、哈希、双指针
每次判断是否出现都扫描数组	$O(nq)$	set、unordered_set、排序后二分
每次找最大值都扫描	$O(nq)$	priority_queue、set、预处理
逐格计算每个子矩阵的和	枚举本身仍有 $O(n^2m^2)$ 个	T102¹⁷⁷ 仅将每次求和降为 $O(1)$
图上每对点都 BFS	点数大时超时	看是否只需单源；正权用 Dijkstra
字符串反复拼接或插入	长字符串会很慢	线性扫描、vector 存片段

6.7 前缀和

6.7.1 什么题会用到

题面出现：

- 多次查询区间和。
- 查询前 k 项和。
- 子数组和、某段累计值。
- 数组不修改，或者所有查询都在修改之前/之后。

6.7.2 一维前缀和模板

使用 1 下标最稳：

```
1 int n, q;
2 cin >> n >> q;
3 vector<long long> pre(n + 1, 0);
4
5 for (int i = 1; i <= n; i++) {
6     long long x;
7     cin >> x;
8     pre[i] = pre[i - 1] + x;
9 }
10
11 while (q--) {
12     int l, r;
13     cin >> l >> r;
14     cout << pre[r] - pre[l - 1] << '\n';
15 }
```

6.7.3 0 下标前缀和

如果原数组是 0 下标，也可以让 $\text{pre}[i+1]$ 表示前 $i+1$ 个数之和：

```
1 vector<int> a(n);
2 vector<long long> pre(n + 1, 0);
3 for (int i = 0; i < n; i++) {
4     cin >> a[i];
5     pre[i + 1] = pre[i] + a[i];
6 }
7
8 // sum of a[l..r], 0-indexed
9 long long ans = pre[r + 1] - pre[l];
```

6.7.4 二维前缀和

适合多次查询子矩阵和。

```
1 int n, m, q;
2 cin >> n >> m >> q;
3 vector<vector<long long>> pre(n + 1, vector<long long>(m + 1, 0));
4
5 for (int i = 1; i <= n; i++) {
6     for (int j = 1; j <= m; j++) {
7         long long x;
8         cin >> x;
9         pre[i][j] = pre[i - 1][j] + pre[i][j - 1]
10             - pre[i - 1][j - 1] + x;
11     }
```

```
12 }
13
14 while (q--) {
15     int x1, y1, x2, y2;
16     cin >> x1 >> y1 >> x2 >> y2;
17     long long ans = pre[x2][y2] - pre[x1 - 1][y2]
18                     - pre[x2][y1 - 1] + pre[x1 - 1][y1 - 1];
19     cout << ans << '\n';
20 }
```

6.7.5 前缀和易错点

- 查询端点是否从 1 开始。
- `pre[0]` 必须是 0。
- 区间公式是一维两项、二维四项。
- 和可能溢出，使用 `long long`。
- 如果中途有修改，普通前缀和通常不适用。

6.8 前后缀预处理

6.8.1 什么题会用到

题面出现：

- 删除一个元素后，问剩余部分的最大、最小、和。
- 对每个位置，问它左边和右边的信息。
- 需要快速知道前 i 个的最大值、后 i 个的最大值。

6.8.2 前缀最大和后缀最大

```
1 int n;
2 cin >> n;
3 vector<int> a(n + 1);
4 for (int i = 1; i <= n; i++) cin >> a[i];
5
6 vector<int> preMax(n + 2, numeric_limits<int>::lowest());
7 vector<int> sufMax(n + 2, numeric_limits<int>::lowest());
8
9 for (int i = 1; i <= n; i++) {
10     preMax[i] = max(preMax[i - 1], a[i]);
11 }
12 for (int i = n; i >= 1; i--) {
13     sufMax[i] = max(sufMax[i + 1], a[i]);
14 }
```

```
15
16 // max value except position i
17 for (int i = 1; i <= n; i++) {
18     int best = max(preMax[i - 1], sufMax[i + 1]);
19     cout << best << '\n';
20 }
```

本例要求 $n \geq 2$ 。若 $n = 1$ ，删除后为空，没有最大值，应按题意单独输出。

6.8.3 前后缀易错点

- 数组多开两位，方便处理 $i-1$ 和 $i+1$ 。
- 初始值要根据题意设置，最大值用很小值，最小值用很大值。
- 如果是求和，使用 `long long`。
- 删除第一个或最后一个元素时，一边为空，要提前定义空边的值。

6.9 差分

6.9.1 什么题会用到

题面出现：

- 很多次区间加。
- 最后统一输出每个位置的值。
- 修改多，查询少或只在最后查询。

6.9.2 一维差分模板

区间 $[l, r]$ 加 v ：

```
1 int n, q;
2 cin >> n >> q;
3 vector<long long> diff(n + 2, 0);
4
5 while (q--) {
6     int l, r;
7     long long v;
8     cin >> l >> r >> v;
9     diff[l] += v;
10    diff[r + 1] -= v;
11 }
12
13 long long cur = 0;
14 for (int i = 1; i <= n; i++) {
15     cur += diff[i];
16     cout << cur << '\n';
17 }
```

如果原数组已有初值：

```
1 vector<long long> a(n + 1), diff(n + 2, 0);
2 for (int i = 1; i <= n; i++) cin >> a[i];
3 for (int i = 1; i <= n; i++) {
4     diff[i] += a[i] - a[i - 1];
5 }
```

6.9.3 差分易错点

- 数组要开到 $n+2$ ，因为会访问 $r+1$ 。
- 差分适合“多次修改后统一求结果”。
- 如果每次修改后立刻查询区间和，普通差分不够。
- 修改值可能很大，用 `long long`。

6.9.4 二维差分入口

适合很多次对子矩阵加值，最后输出整个矩阵。

对矩形 $(x1, y1)$ 到 $(x2, y2)$ 加 v ：

```
1 int n, m, q;
2 cin >> n >> m >> q;
3 vector<vector<long long>>> diff(n + 2, vector<long long>(m + 2, 0));
4
5 while (q--) {
6     int x1, y1, x2, y2;
7     long long v;
8     cin >> x1 >> y1 >> x2 >> y2 >> v;
9     diff[x1][y1] += v;
10    diff[x2 + 1][y1] -= v;
11    diff[x1][y2 + 1] -= v;
12    diff[x2 + 1][y2 + 1] += v;
13 }
14
15 for (int i = 1; i <= n; i++) {
16     for (int j = 1; j <= m; j++) {
17         diff[i][j] += diff[i - 1][j] + diff[i][j - 1]
18             - diff[i - 1][j - 1];
19         cout << diff[i][j] << (j == m ? '\n' : ' ');
20     }
21 }
```

二维差分只适合修改后统一求结果。如果修改和查询交替出现，要考虑二维树状数组或其他方法，B 题中通常会有更简单性质。

6.10 哈希统计

6.10.1 什么题会用到

题面出现：

- 统计每个数或字符串出现次数。
- 判断重复。
- 找出现最多的元素。
- 判断两个集合是否有交集。

6.10.2 值域小：计数数组

```
1 int n;  
2 cin >> n;  
3 vector<int> cnt(100001, 0);  
4 for (int i = 0; i < n; i++) {  
5     int x;  
6     cin >> x;  
7     cnt[x]++;  
8 }
```

6.10.3 值域大：map

```
1 int n;  
2 cin >> n;  
3 map<long long, int> cnt;  
4 for (int i = 0; i < n; i++) {  
5     long long x;  
6     cin >> x;  
7     cnt[x]++;  
8 }  
9  
10 for (auto [x, c] : cnt) {  
11     cout << x << ' ' << c << '\n';  
12 }
```

6.10.4 字符串统计

```
1 int n;  
2 cin >> n;  
3 unordered_map<string, int> cnt;  
4 for (int i = 0; i < n; i++) {  
5     string s;  
6     cin >> s;  
7     cnt[s]++;  
}
```

```
8 }
```

6.10.5 找出现次数最多

```
1 map<string, int> cnt;
2 // fill cnt
3
4 string best;
5 int bestCnt = -1;
6 for (auto [s, c] : cnt) {
7     if (c > bestCnt) {
8         bestCnt = c;
9         best = s;
10    }
11 }
12 cout << best << ' ' << bestCnt << '\n';
```

如果有并列规则，例如字典序最小，由 map 遍历天然按字典序升序。

6.11 余数和同余统计

6.11.1 什么题会用到

题面出现：

- 能否被 k 整除。
- 两个数相加后是 k 的倍数。
- 前缀和模 k 相同。
- 按余数分类统计。

6.11.2 统计每个余数

```
1 int n, k;
2 cin >> n >> k;
3 vector<long long> cnt(k, 0);
4
5 for (int i = 0; i < n; i++) {
6     long long x;
7     cin >> x;
8     int r = (int)(x % k);
9     if (r < 0) r += k;
10    cnt[r]++;
11 }
```

6.11.3 两数和能被 k 整除

```
1 long long ans = 0;
2 for (int r = 0; r < k; r++) {
3     int need = (k - r) % k;
4     if (r < need) {
5         ans += cnt[r] * cnt[need];
6     } else if (r == need) {
7         ans += cnt[r] * (cnt[r] - 1) / 2;
8     }
9 }
10 cout << ans << '\n';
```

6.11.4 前缀和模 k

如果两个前缀和模 k 相同，它们之间的区间和能被 k 整除。

```
1 int n, k;
2 cin >> n >> k;
3 vector<long long> cnt(k, 0);
4 cnt[0] = 1;
5
6 long long sum = 0;
7 long long ans = 0;
8 for (int i = 0; i < n; i++) {
9     long long x;
10    cin >> x;
11    sum += x;
12    int r = (int)(sum % k);
13    if (r < 0) r += k;
14    ans += cnt[r];
15    cnt[r]++;
16 }
17 cout << ans << '\n';
```

余数题注意：如果 k 很大，不能开 `vector cnt(k)`，改用 `map` 或 `unordered_map`。

6.12 排序加扫描

6.12.1 什么题会用到

题面出现：

- 排序后相邻元素有关系。
- 需要合并区间。
- 需要按时间顺序处理事件。
- 需要按优先级选择。

6.12.2 排序后统计相同元素段

```

1 sort(a.begin(), a.end());
2 int n = (int)a.size();
3 for (int i = 0; i < n; ) {
4     int j = i;
5     while (j < n && a[j] == a[i]) {
6         j++;
7     }
8     int count = j - i;
9     cout << a[i] << ' ' << count << '\n';
10    i = j;
11 }

```

6.12.3 合并区间

输入若干区间，合并重叠部分：

```

1 vector<pair<int, int>> segs;
2 sort(segs.begin(), segs.end());
3
4 vector<pair<int, int>> merged;
5 for (auto [l, r] : segs) {
6     if (merged.empty() || l > merged.back().second) {
7         merged.push_back({l, r});
8     } else {
9         merged.back().second = max(merged.back().second, r);
10    }
11 }

```

如果题目认为相邻区间也可合并，例如 [1,2] 和 [3,4]，条件要改成 `l > merged.back().second + 1`。

6.12.4 事件排序

当题目有“按时间发生”的操作，先把事件读入结构体，再排序处理。

```

1 struct Event {
2     int time;
3     int type;
4     int id;
5 };
6
7 vector<Event> events;
8
9 sort(events.begin(), events.end(), [](const Event& a, const Event& b) {
10     if (a.time != b.time) return a.time < b.time;
11     return a.type < b.type; // tie rule depends on statement
12 });

```

```
13
14 for (const Event& e : events) {
15     // process event
16 }
```

事件排序检查：

- 同一时间多个事件的处理顺序。
- 开始事件和结束事件谁先处理。
- 是否需要稳定排序。
- 时间和答案是否要 long long。

6.12.5 坐标离散化入口

当坐标很大，但只关心出现过的坐标，可以离散化。

什么题会用到：

- 坐标可达 10^9 ，不能直接开数组。
- 点的数量只有 10^5 。
- 只需要比较大小、排序、相对位置。

模板：

```
1 vector<int> xs;
2 for (int x : original) {
3     xs.push_back(x);
4 }
5
6 sort(xs.begin(), xs.end());
7 xs.erase(unique(xs.begin(), xs.end()), xs.end());
8
9 auto getId = [&](int x) {
10     return (int)(lower_bound(xs.begin(), xs.end(), x) - xs.begin()) + 1;
11 };
12
13 int id = getId(original[0]);
```

区间离散化要小心。如果题目处理连续区间长度，不能只保存端点编号，还要考虑相邻坐标之间的真实长度。

6.13 堆和动态维护

6.13.1 priority_queue 什么题会用到

题面出现：

- 每次取当前最大或最小。
- 动态加入元素。

- 模拟优先级队列。
- 求前 k 大、前 k 小。

6.13.2 小根堆

```

1 priority_queue<int, vector<int>, greater<int>> pq;
2 pq.push(5);
3 pq.push(2);
4 cout << pq.top() << '\n'; // 2
5 pq.pop();

```

6.13.3 保留最大的 k 个数

用小根堆保存当前最大的 k 个数。要求 $1 \leq k \leq a.size()$ 。

```

1 int k;
2 cin >> k;
3 priority_queue<int, vector<int>, greater<int>> pq;
4
5 for (int x : a) {
6     pq.push(x);
7     if ((int)pq.size() > k) {
8         pq.pop();
9     }
10 }
11
12 cout << pq.top() << '\n'; // kth largest

```

6.13.4 set 动态维护

set 适合动态插入、删除、找前驱后继。

```

1 set<int> s;
2 s.insert(10);
3 s.insert(20);
4 s.insert(30);
5
6 int x = 25;
7 auto it = s.lower_bound(x); // first >= x
8 if (it != s.end()) {
9     cout << "next=" << *it << '\n';
10 }
11 if (it != s.begin()) {
12     --it;
13     cout << "prev=" << *it << '\n';
14 }

```

迭代器小心： 移动迭代器前先判断是不是 `begin()` 或 `end()`。

6.13.5 map 动态维护

map 适合动态统计并按 key 有序遍历。

```
1 map<int, int> cnt;
2 cnt[x]++;
3
4 cnt[x]--;
5 if (cnt[x] == 0) {
6     cnt.erase(x);
7 }
```

取最小 key 和最大 key:

```
1 int mn = cnt.begin()->first;
2 int mx = cnt.rbegin()->first;
```

取 begin/rbegin 前确认 map 非空。

6.14 基础贪心

6.14.1 什么题会用到

B 题中的贪心通常比较直接，常见信号：

- 尽量多选一些区间、任务、物品。
- 每次选择当前最早结束、最小代价、最大收益。
- 排序后从前往后扫描。
- 题目问最少次数覆盖、最多不冲突任务。

6.14.2 最多不重叠区间

按右端点从小到大排序，每次选最早结束的区间。

```
1 vector<pair<int, int>> segs; // l, r
2 sort(segs.begin(), segs.end(), [](auto a, auto b) {
3     if (a.second != b.second) return a.second < b.second;
4     return a.first < b.first;
5 });
6
7 int ans = 0;
8 int lastEnd = 0;
9 bool hasChosen = false;
10 for (auto [l, r] : segs) {
11     if (!hasChosen || l > lastEnd) { // >= if endpoints can touch
12         hasChosen = true;
13         ans++;
14         lastEnd = r;
15     }
```

```
16 }  
17 cout << ans << '\n';
```

6.14.3 贪心易错点

- 先确认排序关键字，是按左端点、右端点、代价还是收益。
- 区间相邻是否算冲突： $l > \text{lastEnd}$ 还是 $l \geq \text{lastEnd}$ 。
- 贪心不是看起来合理就一定对。B 题若想不出证明，至少用小样例反驳自己。

6.15 单调栈和单调队列入口

6.15.1 单调栈什么题会用到

题面出现：

- 找每个元素左边或右边第一个更大/更小的元素。
- 柱状图、温度、最近更高。
- 暴力向左/向右扫描会超时。

6.15.2 右边第一个更大元素

```
1 int n;  
2 cin >> n;  
3 vector<int> a(n);  
4 for (int i = 0; i < n; i++) cin >> a[i];  
5  
6 vector<int> ans(n, -1);  
7 stack<int> st; // indices, a[st] decreasing  
8  
9 for (int i = 0; i < n; i++) {  
10     while (!st.empty() && a[i] > a[st.top()]) {  
11         ans[st.top()] = i;  
12         st.pop();  
13     }  
14     st.push(i);  
15 }
```

6.15.3 单调队列什么题会用到

题面出现固定长度滑动窗口最大值/最小值。

```
1 int n, k;  
2 cin >> n >> k;  
3 vector<int> a(n);  
4 for (int i = 0; i < n; i++) cin >> a[i];  
5
```

```

6 deque<int> dq; // indices, values decreasing
7 for (int i = 0; i < n; i++) {
8     while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
9     while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
10    dq.push_back(i);
11
12    if (i >= k - 1) {
13        cout << a[dq.front()] << '\n'; // max of window
14    }
15 }

```

练单调栈、单调队列时，最好手动跟一遍入栈和出栈过程，尤其试一下相等元素。

6.16 双指针入门

6.16.1 什么题会用到

题面出现：

- 数组已排序，找一对数。
- 连续区间满足某个条件。
- 左右端点移动，区间和或区间长度变化。

6.16.2 排序后找两数和

```

1 sort(a.begin(), a.end());
2 int l = 0, r = n - 1;
3 bool ok = false;
4
5 while (l < r) {
6     long long sum = 1LL * a[l] + a[r];
7     if (sum == target) {
8         ok = true;
9         break;
10    } else if (sum < target) {
11        l++;
12    } else {
13        r--;
14    }
15 }
16
17 cout << (ok ? "YES" : "NO") << '\n';

```

6.16.3 正数数组的连续区间

求和不超过 $limit \geq 0$ 的最长连续子数组，且每项非负时，可以用下面的滑动窗口。

```

1 int l = 0;
2 long long sum = 0;
3 int best = 0;
4
5 for (int r = 0; r < n; r++) {
6     sum += a[r];
7     while (sum > limit && l <= r) {
8         sum -= a[l];
9         l++;
10    }
11    best = max(best, r - l + 1);
12 }

```

滑动窗口前提：通常要求元素非负或条件具有单调性。若有负数，窗口移动可能失效。

6.17 二分与二分答案

6.17.1 普通二分查找

数组必须有序。

```

1 sort(a.begin(), a.end());
2 auto it = lower_bound(a.begin(), a.end(), x);
3 if (it != a.end()) {
4     cout << *it << '\n';
5 }

```

6.17.2 二分答案识别

题面出现：

- 最大化最小值。
- 最小化最大值。
- 求最小的可行时间、容量、代价。
- 给定一个答案 mid ，容易判断是否可行。

6.17.3 二分答案模板

找最小可行值：

```

1 bool check(long long mid) {
2     // return true if mid is feasible
3     return true;
4 }
5
6 long long l = 0, r = 1000000000000000000LL;
7 while (l < r) {

```

```

8   long long mid = l + (r - l) / 2;
9   if (check(mid)) {
10      r = mid;
11   } else {
12      l = mid + 1;
13   }
14 }
15 cout << l << '\n';

```

找最大可行值:

```

1   long long l = 0, r = 1000000000000000000LL;
2   while (l < r) {
3       long long mid = l + (r - l + 1) / 2;
4       if (check(mid)) {
5           l = mid;
6       } else {
7           r = mid - 1;
8       }
9   }
10  cout << l << '\n';

```

二分答案关键：必须有单调性，并自行完成 `check`，不能保留示例里的 `return true`；。最小值搜索须保证初始右端可行，最大值搜索须保证初始左端可行；否则先判无解。按题意推导上下界。

6.18 BFS 入门

6.18.1 什么题会用到

题面出现：

- 最少步数。
- 迷宫、网格移动。
- 每次操作代价相同。
- 从起点扩展到终点。

6.18.2 网格 BFS 模板

```

1   int n, m;
2   cin >> n >> m;
3   vector<string> g(n);
4   for (int i = 0; i < n; i++) cin >> g[i];
5
6   vector<vector<int>> dist(n, vector<int>(m, -1));
7   queue<pair<int, int>> q;
8
9   int sx, sy, tx, ty;

```

```
10 cin >> sx >> sy >> tx >> ty;
11
12 dist[sx][sy] = 0;
13 q.push({sx, sy});
14
15 int dx[4] = {-1, 0, 1, 0};
16 int dy[4] = {0, 1, 0, -1};
17
18 while (!q.empty()) {
19     auto [x, y] = q.front();
20     q.pop();
21
22     for (int d = 0; d < 4; d++) {
23         int nx = x + dx[d];
24         int ny = y + dy[d];
25         if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
26         if (g[nx][ny] == '#') continue;
27         if (dist[nx][ny] != -1) continue;
28
29         dist[nx][ny] = dist[x][y] + 1;
30         q.push({nx, ny});
31     }
32 }
33
34 cout << dist[tx][ty] << '\n';
```

6.18.3 BFS 易错点

- 坐标输入是 0 下标还是 1 下标。
- 障碍字符是什么。
- 起点是否可能等于终点。
- 应在入队时标记访问，避免重复入队。
- BFS 适合每步代价相同；边权不同且非负可考虑 Dijkstra；含负权不能直接套。

6.18.4 多源 BFS

如果有多个起点，且要求每个点到最近起点的距离，把所有起点同时入队。

```
1 vector<vector<int>> dist(n, vector<int>(m, -1));
2 queue<pair<int, int>> q;
3
4 for (auto [x, y] : starts) {
5     dist[x][y] = 0;
6     q.push({x, y});
7 }
8
```

```
9 while (!q.empty()) {
10     auto [x, y] = q.front();
11     q.pop();
12     for (int d = 0; d < 4; d++) {
13         int nx = x + dx[d];
14         int ny = y + dy[d];
15         if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
16         if (g[nx][ny] == '#') continue;
17         if (dist[nx][ny] != -1) continue;
18         dist[nx][ny] = dist[x][y] + 1;
19         q.push({nx, ny});
20     }
21 }
```

什么题会用到：

- 多个火源、多个出口、多个起点。
- 求离最近特殊点的距离。
- 每一步代价相同。

6.19 DFS 和连通块

6.19.1 什么题会用到

题面出现：

- 统计连通块数量。
- 找一片区域大小。
- 网格中相邻的同类格子。
- 图中从某点能到哪些点。

6.19.2 网格 DFS

```
1 int n, m;
2 vector<string> g;
3 vector<vector<int>> vis;
4 int dx[4] = {-1, 0, 1, 0};
5 int dy[4] = {0, 1, 0, -1};
6
7 int dfs(int x, int y) {
8     vis[x][y] = 1;
9     int area = 1;
10    for (int d = 0; d < 4; d++) {
11        int nx = x + dx[d];
12        int ny = y + dy[d];
13        if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
```

```

14     if (vis[nx][ny]) continue;
15     if (g[nx][ny] == '#') continue;
16     area += dfs(nx, ny);
17 }
18 return area;
19 }

```

统计连通块:

```

1 // Read n,m and g first.
2 vis.assign(n, vector<int>(m, 0));
3 int components = 0;
4 for (int i = 0; i < n; i++) {
5     for (int j = 0; j < m; j++) {
6         if (!vis[i][j] && g[i][j] != '#') {
7             components++;
8             int area = dfs(i, j);
9         }
10    }
11 }

```

递归深度风险: 如果网格很大, 递归 DFS 可能爆栈。B 题中可改用 BFS 更稳。

6.20 Floyd 入门

6.20.1 什么题会用到

- 问任意两点之间最短路。
- 点数较小, 例如 $n \leq 300$ 或 $n \leq 500$ 。
- 图可能是稠密图。

6.20.2 Floyd 模板

```

1 const long long INF = (long long)4e18;
2
3 int n, m;
4 cin >> n >> m;
5 vector<vector<long long>> dist(n + 1, vector<long long>(n + 1, INF));
6
7 for (int i = 1; i <= n; i++) {
8     dist[i][i] = 0;
9 }
10
11 for (int i = 0; i < m; i++) {
12     int u, v;
13     long long w;
14     cin >> u >> v >> w;
15     dist[u][v] = min(dist[u][v], w);

```

```

16     dist[v][u] = min(dist[v][u], w); // remove if directed
17 }
18
19 for (int k = 1; k <= n; k++) {
20     for (int i = 1; i <= n; i++) {
21         for (int j = 1; j <= n; j++) {
22             if (dist[i][k] == INF || dist[k][j] == INF) continue;
23             dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
24         }
25     }
26 }

```

6.20.3 Floyd 易错点

- 三层循环顺序必须是 k, i, j 。
- 无向图要双向赋值，有向图不要。
- 多条边取最小边权。
- 点数大时不能用 Floyd。

6.21 Dijkstra 入门

6.21.1 什么题会用到

- 非负权图最短路。
- 边权不是全 1。
- 点和边较多，不能 Floyd。
- 单源最短路：从一个起点到其他点。

6.21.2 Dijkstra 模板

```

1 struct Edge {
2     int to;
3     long long w;
4 };
5
6 const long long INF = (long long)4e18;
7
8 int n, m, s;
9 cin >> n >> m >> s;
10 vector<vector<Edge>> g(n + 1);
11
12 for (int i = 0; i < m; i++) {
13     int u, v;
14     long long w;

```

```
15     cin >> u >> v >> w;
16     g[u].push_back({v, w});
17     g[v].push_back({u, w}); // remove if directed
18 }
19
20 vector<long long> dist(n + 1, INF);
21 priority_queue<pair<long long, int>,
22             vector<pair<long long, int>>,
23             greater<pair<long long, int>>> pq;
24
25 dist[s] = 0;
26 pq.push({0, s});
27
28 while (!pq.empty()) {
29     auto [d, u] = pq.top();
30     pq.pop();
31     if (d != dist[u]) continue;
32
33     for (auto e : g[u]) {
34         if (e.w >= INF - dist[u]) continue;
35         if (dist[e.to] > dist[u] + e.w) {
36             dist[e.to] = dist[u] + e.w;
37             pq.push({dist[e.to], e.to});
38         }
39     }
40 }
```

6.21.3 Dijkstra 易错点

- 只适用于非负边权。存在负边权时不能直接用。
- 距离用 long long。
- 无向图双向加边，有向图单向加边。
- 堆中可能有过期状态，要跳过。
- 如果所有边权都是 1，用 BFS 更简单。

6.22 并查集入门

6.22.1 什么题会用到

题面出现：

- 合并两个集合。
- 判断两个元素是否属于同一组。
- 连通块数量。

- 朋友关系、同类关系、网络连通。

6.22.2 并查集模板

```
1 struct DSU {
2     vector<int> parent, sz;
3
4     DSU(int n) {
5         parent.resize(n + 1);
6         sz.assign(n + 1, 1);
7         for (int i = 1; i <= n; i++) parent[i] = i;
8     }
9
10    int find(int x) {
11        if (parent[x] == x) return x;
12        return parent[x] = find(parent[x]);
13    }
14
15    bool unite(int a, int b) {
16        int ra = find(a);
17        int rb = find(b);
18        if (ra == rb) return false;
19        if (sz[ra] < sz[rb]) swap(ra, rb);
20        parent[rb] = ra;
21        sz[ra] += sz[rb];
22        return true;
23    }
24
25    bool same(int a, int b) {
26        return find(a) == find(b);
27    }
28 };
```

6.23 拓扑排序入口

6.23.1 什么题会用到

题面出现：

- 任务必须按先后顺序完成。
- A 必须在 B 前面。
- 课程依赖、工程依赖。
- 判断是否存在合法顺序。

6.23.2 拓扑排序模板

```
1 int n, m;
2 cin >> n >> m;
3 vector<vector<int>> g(n + 1);
4 vector<int> indeg(n + 1, 0);
5
6 for (int i = 0; i < m; i++) {
7     int u, v;
8     cin >> u >> v; // u before v
9     g[u].push_back(v);
10    indeg[v]++;
11 }
12
13 queue<int> q;
14 for (int i = 1; i <= n; i++) {
15     if (indeg[i] == 0) q.push(i);
16 }
17
18 vector<int> order;
19 while (!q.empty()) {
20     int u = q.front();
21     q.pop();
22     order.push_back(u);
23
24     for (int v : g[u]) {
25         indeg[v]--;
26         if (indeg[v] == 0) q.push(v);
27     }
28 }
29
30 if ((int)order.size() == n) {
31     cout << "YES\n";
32 } else {
33     cout << "NO\n"; // has cycle
34 }
```

6.23.3 拓扑排序易错点

- 边的方向：如果 A 在 B 前，应连 A 到 B。
- 入度为 0 的点一开始全部入队。
- 输出顺序是否要求字典序最小。若要求，队列换成小根堆。
- 若排序结果少于 n 个点，说明有环。

6.24 简单 DP 入口

6.24.1 什么题会用到

题面出现：

- 求最优值：最大、最小、方案数。
- 当前结果依赖前一个或前几个位置。
- 选择或不选择某个物品。
- 容量、价值、花费、恰好装满。

6.24.2 线性 DP 示例

例如每次爬 1 或 2 级台阶。下例要求 $0 \leq n \leq 91$ ；更大时需按题意取模或使用高精度：

```

1 int n;
2 cin >> n;
3 vector<long long> dp(n + 1, 0);
4 dp[0] = 1;
5 if (n >= 1) dp[1] = 1;
6 for (int i = 2; i <= n; i++) {
7     dp[i] = dp[i - 1] + dp[i - 2];
8 }
9 cout << dp[n] << '\n';

```

6.24.3 01 背包入口

每个物品只能选一次，求重量不超过 V 的最大价值；恰好装满见 T303¹⁹⁴。

```

1 int n, V;
2 cin >> n >> V;
3 vector<int> w(n + 1);
4 vector<long long> val(n + 1);
5 for (int i = 1; i <= n; i++) {
6     cin >> w[i] >> val[i];
7 }
8
9 vector<long long> dp(V + 1, 0);
10 for (int i = 1; i <= n; i++) {
11     for (int j = V; j >= w[i]; j--) {
12         dp[j] = max(dp[j], dp[j - w[i]] + val[i]);
13     }
14 }
15 cout << dp[V] << '\n';

```

6.24.4 完全背包入口

每个物品可以选无限次，容量正序；重量必须为正，模板求重量不超过 V 的最大价值。

```

1 int n, V;
2 cin >> n >> V;
3 vector<int> w(n + 1);
4 vector<long long> val(n + 1);
5 for (int i = 1; i <= n; i++) {
6     cin >> w[i] >> val[i];
7 }
8
9 vector<long long> dp(V + 1, 0);
10 for (int i = 1; i <= n; i++) {
11     for (int j = w[i]; j <= V; j++) {
12         dp[j] = max(dp[j], dp[j - w[i]] + val[i]);
13     }
14 }
15 cout << dp[V] << '\n';

```

6.24.5 DP 易错点

- 先定义 $dp[i]$ 表示什么。
- 写出初始状态。
- 写出转移方程。
- 01 背包容量倒序，完全背包容量正序。
- 方案数可能很大，要看是否取模。

6.25 基础数学入口

6.25.1 gcd 和 lcm

```

1 // Require a,b >= 0 and lcm(a,b) <= LLONG_MAX.
2 long long g = gcd(a, b);
3 long long l = (a == 0 || b == 0) ? 0 : a / g * b;

```

先除后乘，降低溢出风险。

6.25.2 快速幂

如果题目要求 $a^b \bmod mod$:

```

1 // Require b >= 0 and 1 <= mod <= 3037000500LL.
2 long long modPow(long long a, long long b, long long mod) {
3     long long res = 1 % mod;
4     a %= mod;
5     if (a < 0) a += mod;
6     while (b > 0) {
7         if (b & 1) res = res * a % mod;

```

```

8         a = a * a % mod;
9         b >>= 1;
10    }
11    return res;
12 }

```

6.25.3 判断素数

单个数判断:

```

1 bool isPrime(long long x) {
2     if (x < 2) return false;
3     for (long long d = 2; d <= x / d; d++) {
4         if (x % d == 0) return false;
5     }
6     return true;
7 }

```

6.25.4 枚举约数

```

1 vector<long long> divisors;
2 for (long long d = 1; d <= x / d; d++) {
3     if (x % d == 0) {
4         divisors.push_back(d);
5         if (d != x / d) divisors.push_back(x / d);
6     }
7 }
8 sort(divisors.begin(), divisors.end());

```

数学题先看范围。如果要对很多个数判断素数，可能需要筛法；如果只是少量数，试除通常够用。

6.26 并查集调用示例

先复制 [T203¹⁸⁷](#) 的 DSU 定义，再在主函数中使用：

```

1 int n, m;
2 cin >> n >> m;
3 DSU dsu(n);
4
5 while (m--) {
6     int op, a, b;
7     cin >> op >> a >> b;
8     if (op == 1) dsu.unite(a, b);
9     else cout << (dsu.same(a, b) ? "YES" : "NO") << '\n';
10 }

```

6.27 B 题提交前检查

- 是否估算过复杂度。
- 是否把暴力写到了大数据上。
- 前缀和、差分的下标是否统一。
- 求和、答案、距离是否用 `long long`。
- 排序比较器是否正确。
- `lower_bound` 前是否排序。
- BFS 是否在入队时标记。
- 并查集是否初始化所有点。
- 多组数据是否清空容器。

6.28 B 题优化速查

当你已经有一个暴力思路，但担心超时，用这张表判断下一步。

当前情况	优先尝试	原因
暴力是两层循环找一对数	排序、哈希、双指针	二元关系常能降到 $O(n \log n)$ 或 $O(n)$ 。
暴力是每次求区间和	前缀和	静态区间查询标准做法。
暴力是每次区间加	差分	最后统一查询时最简单。
暴力是每次找最大/最小	堆或 set	动态维护极值。
坐标太大但点数不多	离散化	把大坐标压缩成小编号。
规则按时间发生	事件排序	统一按时间顺序处理。
左边右边都要快速知道信息	前后缀预处理	删除一个元素或分割数组常见。
涉及整除、余数相同	余数统计	同余关系常能转成计数。
最多不冲突任务或区间	贪心排序	常见按右端点排序。
每个点找旁边第一个更大/更小	单调栈	避免每次向旁边扫。
滑动窗口最大/最小	单调队列	固定窗口极值。
网格最短步数	BFS	每步代价相同。
网格连通区域	DFS/BFS	统计区域或连通块。
图边权为 1	BFS	比 Dijkstra 简单。
图边权为正	Dijkstra	单源非负权最短路。
点数小且任意两点	Floyd	写法短，适合小图。
合并和查询同组	并查集	连通性标准做法。
依赖顺序	拓扑排序	入度为 0 的点先处理。

6.29 B 题常见边界库

题型	边界样例	检查目标
前缀和	$l = 1, r = n, l = r$	端点公式。
二维前缀和	单行、单列、整个矩阵、单个格子	四项公式。
差分	修改到 $r = n$ ，只改一个点	$r+1$ 和数组大小。
排序	全相等、逆序、相同主关键字	比较器。
哈希统计	没有重复、全部重复、key 很大	容器选择。
双指针	无解、两个数刚好在两端、有重复	指针移动。
二分答案	最小值可行、最大值不可行、边界答案	check 单调性。
BFS	起点等于终点、无路、全障碍附近	距离初始化。
DFS 连通块	全障碍、全可走、单个格子	访问标记。
并查集	自己和自己合并、重复合并	find/umite。
Dijkstra	不连通、多条边、起点到自己	INF 和取最小边。
余数统计	负数、 $k = 1$ 、所有余数相同	取模修正和计数公式。
贪心	区间端点相同、相邻是否冲突	排序关键字和边界条件。
单调栈	全递增、全递减、全相等	比较符号是 $<$ 还是 $<=$ 。

6.30 B 题本地调试流程

6.30.1 先用暴力对拍思路

优化后的程序可以用小数据和暴力结果对照。来不及写对拍程序时，也可以取 3 到 6 个元素，手算几组结果。

- n 取 3 到 6。
- 手算所有操作结果。
- 用暴力程序或草稿验证优化结果。
- 专门造边界：重复、空、单点、全相等。

6.30.2 复杂度复查

提交前问：

- 有没有隐藏的双重循环。
- map 操作是否在循环里被调用很多次，是否仍可接受。
- 排序是否放在循环内部。若是，通常危险。

- BFS/Dijkstra 是否会重复跑很多次。
- 字符串拼接是否在大循环里反复发生。

6.30.3 状态清空

多组数据时，尤其注意：

```
1 g.assign(n + 1, vector<int>());
2 dist.assign(n + 1, INF);
3 vis.assign(n + 1, 0);
4 cnt.clear();
5 while (!q.empty()) q.pop();
```

队列、优先队列没有 clear。最简单的方法是在每组数据内部重新定义它。

6.31 B 题卡住时的部分分策略

1. 如果不会优化，先写 $O(n^2)$ 暴力，确认题意。
2. 如果有多档数据，先保证小数据正确。
3. 如果是区问题，先写单次查询或无修改版本。
4. 如果是图题，先写 BFS/DFS 处理无权或小图。
5. 如果是复杂排序规则，先保证主关键字正确，再处理并列规则。

6.32 B 题模板练习清单

这些模板建议在考前单独练习到“看着能快速写对”。

模板	熟练要求	主要用途
一维前缀和	建议熟练掌握	静态区间和。
二维前缀和	照着能写	子矩阵和。
一维差分	建议熟练掌握	区间加，最后查询。
二维差分	照着能写	矩形加，最后输出。
map/unordered_map 统计	建议熟练掌握	频率统计。
排序加结构体比较器	建议熟练掌握	排名、事件、贪心。
双指针	照着能写	排序后找对、滑动窗口。
二分答案	照着能写	最大化最小值、最小化最大值。
前后缀预处理	照着能写	删除一个元素、左右信息。
余数统计	照着能写	整除、同余、区间和取模。
基础贪心	照着能写	区间选择、任务安排。
单调栈/队列	了解并练过再用	最近更大、滑动窗口极值。
网格 BFS	能对照写出	最短步数。
并查集	照着能写	连通性。

模板	熟练要求	主要用途
拓扑排序	照着能写	依赖顺序。
Dijkstra	照着能写	非负权最短路。

6.33 矩阵与稀疏数据补充入口

矩阵重塑和乘法顺序见 [T111¹⁸⁴](#)；有序稀疏向量点积见 [T112¹⁸⁴](#)；质因数分解和批量素数查询见 [T515²⁰⁴](#)。购买总额至少达到门槛时，使用 [T303¹⁹⁴](#) 的可达金额版，而不是直接复制 [T301¹⁹³](#)。

第七章 C 题决策树与大模拟方法

7.1 C 题定位

C 题的长题面常见多个对象、几种命令和一些特殊规则。读题和组织代码会占不少时间，先把这些内容分清楚，后面才容易查错。

- 建议目标时间：60–100 分钟。
- 目标：能满分则满分；若细节太多，先完成题目明确给出的子任务。
- 高频主题：大模拟、字符串解析、状态维护、网格搜索、图论基础、数据结构基础、简单 DP。
- 容易出错的地方：漏规则、状态没记全、不同命令混在同一段代码里。

可以先列一张对象和命令表，确认每种操作会改哪些字段。

7.2 C 题开题流程

1. 快速浏览题面，判断是大模拟还是算法题。
2. 圈出所有对象：人、文件、任务、节点、格子、命令、事件。
3. 圈出所有状态：位置、时间、分数、开关、权限、颜色、方向。
4. 圈出所有操作：新增、删除、查询、移动、更新、比较。
5. 圈出优先级和并列规则。
6. 设计结构体和函数。
7. 先实现最基础规则，通过样例的一部分。
8. 再逐条补特殊规则。

7.3 C 题读题三遍法

题面较长时可以分几遍读：先看要模拟什么，再记状态和命令，最后核对特殊规则。

7.3.1 第一遍：只判断题型

第一遍快速读，不抠细节，只回答：

- 是大模拟，还是算法题？
- 输入是一次性数据，还是很多命令？

- 输出是每次查询输出，还是最后统一输出？
- 数据范围是否明显要求优化？

第一遍还没理清时，可以结合样例看看一次操作前后发生了什么。

7.3.2 第二遍：圈对象、状态、操作

第二遍要动笔。把题目中的名词和动词分开。

类别	怎么找	例子
对象	被操作的名词	用户、文件、进程、任务、车辆、节点、格子。
属性	对象身上的数据	编号、名字、位置、时间、状态、得分。
操作	题目中的动词或命令	新增、删除、移动、查询、合并、修改。
规则	操作发生时的限制	如果不存在则忽略；如果冲突按优先级。
输出	什么时候打印	每次 query、最后统计、发生错误时。

7.3.3 第三遍：找坑点

第三遍只找容易错的细节：

- 相同时间、相同分数、相同优先级怎么处理。
- 编号从 0 还是 1 开始。
- 找不到对象时怎么处理。
- 重复添加时怎么处理。
- 删除后是否还能查询。
- 最后一条命令或最后一个事件是否要特殊处理。
- 输出顺序按输入顺序、编号顺序、字典序还是时间顺序。

7.4 从题面到代码的转换

7.4.1 对象变成 struct

题目中的每类对象，优先考虑一个结构体。

```
1 struct Task {
2     int id;
3     int startTime;
4     int endTime;
5     int priority;
6     int status;
```

```
7 };
```

7.4.2 编号查找用 map 或 vector

如果编号是 1 到 n :

```
1 vector<Task> tasks(n + 1);
```

如果编号很大或是字符串:

```
1 map<string, Task> tasks;  
2 unordered_map<int, Task> tasksById;
```

7.4.3 命令变成函数

每个命令单独写函数:

```
1 void addTask(int id, int priority) {  
2     // add or update  
3 }  
4  
5 void deleteTask(int id) {  
6     // delete if exists  
7 }  
8  
9 void queryTask(int id) {  
10    // print answer  
11 }
```

7.4.4 规则变成 if

题目中的条件要落实到代码里。例如，查询前先判断对象是否存在，再判断它的状态:

```
1 if (!tasks.count(id)) {  
2     cout << "Not Found\n";  
3     return;  
4 }  
5  
6 if (tasks[id].status == FINISHED) {  
7     cout << "Finished\n";  
8     return;  
9 }
```

把几条不同规则分开写，样例不对时比较容易找到是哪一步。

7.5 状态维护和容器选择

7.5.1 容器选择表

需求	推荐容器	原因
编号是 1 到 n	<code>vector<Obj></code>	直接下标访问，最快。
编号很大但唯一	<code>unordered_map<int,Obj></code>	平均快速查找。
编号是字符串	<code>map<string,Obj></code> 或 <code>unordered_map</code>	字符串到对象映射。
需要按 key 有序输出	<code>map</code>	自动按 key 排序。
需要判断是否存在	<code>set</code> / <code>unordered_set</code>	查重。
需要动态最小/最大	<code>set</code> / <code>priority_queue</code>	维护极值。
需要前驱后继	<code>set</code>	<code>lower_bound</code> 。
需要按进入顺序处理	<code>queue</code>	FIFO。
需要最近使用顺序	<code>list</code> + <code>map</code>	LRU。
需要父子层级	<code>vector<Node></code> + <code>child</code>	树结构。

7.5.2 操作复杂度估算

容器	常见复杂度	注意
<code>vector</code> 下标访问	$O(1)$	中间插入删除慢。
<code>map</code> 查找插入	$O(\log n)$	有序但常数较大。
<code>unordered_map</code> 查找插入	平均 $O(1)$	无序，最坏情况不稳定。
<code>set</code> 查找插入删除	$O(\log n)$	可找前驱后继。
<code>priority_queue</code> 取最优	$O(\log n)$ 插入删除堆顶	不能直接删除中间元素。
<code>queue</code> 进出队	$O(1)$	只能访问队首。
<code>list</code> 插入删除	已有迭代器时 $O(1)$	不能随机访问。

7.5.3 C 题容器选择错误信号

- 每次命令都全表扫描，且命令数很大：应该用 `map/set/hash`。
- 每次取最大值都排序：应该用 `set` 或堆。
- 每次删除 `vector` 中间元素：可能需要标记删除、`list` 或 `set`。
- 需要有序输出却用了 `unordered_map`：输出顺序会错。
- `priority_queue` 里元素更新了但旧值还在：需要懒删除。

7.6 C 题总决策树

题面信号	优先怀疑	首选方法
题面很长，规则很多	大模拟	结构体 + 函数拆解

题面信号	优先怀疑	首选方法
输入是一串命令	命令解析模拟	stringstream / split / 状态表
按时间发生事件	事件模拟	事件排序 / 优先队列
对象有多种状态	状态机模拟	enum/int 表示状态
文件路径、配置、表达式	字符串解析	逐字符扫描 / 栈
网格、地图、移动	网格模拟或 BFS/DFS	方向数组、vis、dist
图、连通、最短	图论	BFS / Dijkstra / 并查集
区间修改查询	数据结构	前缀和 / 差分 / 树状数组 / 线段树
求最优值且有阶段	DP	先定义状态

7.7 C 题常见题型

这张表用于快速判断 C 题该从哪个方向切入。

题目长相	本质	先翻哪里
很多命令，每条命令改变系统状态	命令模拟	命令解析、状态设计、函数化
按时间发生、开始结束、排队	事件模拟	事件排序、优先级、队列模拟
字符串里有路径、参数、配置	字符串解析	split、key=value、路径规范化
输入像 URL、路由、参数	路由匹配	URL 路由和参数解析
输入像 JSON、配置、嵌套括号	结构解析	递归解析、栈、逐字符扫描
输入像 Markdown、HTML、轻量标记	标记语言模拟	行扫描、状态切换
有用户、角色、权限、规则	规则匹配	权限和规则匹配
有表格、矩阵、坐标变换	表格模拟	矩阵操作、行列映射
有缓存、队列、窗口、淘汰	数据结构模拟	队列、set、map、懒删除
有目录、树、层级关系	树结构模拟	目录树、父子关系、DFS
有地图并带钥匙/状态	状态搜索	带状态 BFS

7.8 C 题细分决策树

7.8.1 第一问：这是大模拟还是算法题

题面特征	判断	下一步
规则很多、命令很多、对象很多	大模拟	拆对象、状态、命令
题面不长但数据范围大	算法题	回到 B/D 算法决策树
有地图和最短步数	搜索题	BFS / 状态 BFS
有依赖关系	图论题	拓扑 / 最短路 / 并查集
有复杂字符串格式	解析题	字符串解析决策树
有大量修改查询	数据结构题	前缀/差分/树状数组/线段树

7.8.2 大模拟细分树

题面信号	类型	首选结构
每行一个操作名	命令系统	handleCommand 分发
对象有生命周期	增删改查模拟	map<int,Obj> 或 vector<Obj>
对象有多个状态	状态机	状态常量 + 转移函数
按时间发生	事件模拟	Event + 排序
每次取最优对象	优先级模拟	堆 / set / 懒删除堆
行列、表格、棋盘	表格模拟	二维数组 / 行列映射
路径、目录、父子层级	树结构模拟	节点数组 + child map
规则匹配、权限判断	规则系统	Rule + match

7.8.3 字符串解析细分树

字符串形态	优先方法	小心
按空格分隔	stringstream	前面是否要 ignore
按固定字符分隔	split	连续分隔符
key=value	find + substr	值是否含空格
路径 /a/b/c	路径 split / normalize	根目录、..
括号匹配	栈	空栈和最后剩余
表达式求值	扫描 / 递归解析	优先级、括号
URL 参数	先按 ?, 再按 &	参数为空
嵌套 JSON 类	逐字符扫描 / 栈	先利用题目限制

7.8.4 图网格细分树

题面信号	类型	首选模板
最少步数、每步代价相同	无权最短路	BFS

题面信号	类型	首选模板
多个起点最近距离	多源 BFS	所有起点入队
有钥匙、方向、模式等额外状态	状态 BFS	<code>dist[x][y][state]</code>
统计区域大小	连通块	DFS / BFS
正权边最短路	正权图	Dijkstra
只问是否连通	连通性	并查集 / DFS
任务依赖顺序	DAG	拓扑排序

7.9 大模拟总方法

7.9.1 大模拟什么题会用到

- 题面长度明显长。
- 有很多“如果……否则……”规则。
- 输入包含多条命令。
- 有多个对象，每个对象有状态。
- 输出要求根据最终状态或每次查询得到。

7.9.2 大模拟代码骨架

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 struct Item {
6     int id;
7     string name;
8     int state;
9 };
10
11 int n, m;
12 vector<Item> items;
13
14 void readInput() {
15     cin >> n >> m;
16     items.resize(n);
17     for (int i = 0; i < n; i++) {
18         cin >> items[i].id >> items[i].name;
19         items[i].state = 0;
20     }
21 }
22
```

```
23 void handleCommand(const string& cmd) {
24     if (cmd == "ADD") {
25         // handle add
26     } else if (cmd == "DEL") {
27         // handle delete
28     } else if (cmd == "QUERY") {
29         // handle query
30     }
31 }
32
33 void solve() {
34     for (int i = 0; i < m; i++) {
35         string cmd;
36         cin >> cmd;
37         handleCommand(cmd);
38     }
39 }
40
41 int main() {
42     ios::sync_with_stdio(false);
43     cin.tie(nullptr);
44
45     readInput();
46     solve();
47     return 0;
48 }
```

7.9.3 为什么要拆函数

- 读入、处理、输出分开，查错更容易。
- 每个命令一个函数，避免主函数过长。
- 特殊规则可以单独补，不容易破坏已有逻辑。
- 样例错时能定位是哪条规则错。

7.10 长题面拆解表

读 C 题时，在草稿纸上画这个表：

项目	要记录什么	示例
对象	题目中被操作的东西	用户、文件、任务、格子、节点。
状态	对象身上的变量	位置、分数、权限、颜色、是否激活。
命令	输入中每种操作	add、delete、query、move。
规则	每条命令如何改变状态	如果存在则更新，否则新增。
优先级	相同条件时谁先	时间小优先、编号小优先。

项目	要记录什么	示例
边界	特殊或非法情况	找不到、重复、越界、空集合。
输出	什么时候输出什么	每次 query 输出，最后统一输出。

7.11 命令解析

7.11.1 固定格式命令

如果每条命令格式固定：

```

1 string op;
2 cin >> op;
3 if (op == "add") {
4     int id, x;
5     cin >> id >> x;
6 } else if (op == "query") {
7     int id;
8     cin >> id;
9 }
```

7.11.2 整行命令

如果命令中可能有空格，先读整行。

```

1 int q;
2 cin >> q;
3 cin.ignore(numeric_limits<streamsize>::max(), '\n');
4
5 while (q--) {
6     string line;
7     getline(cin, line);
8     stringstream ss(line);
9     string op;
10    ss >> op;
11
12    if (op == "add") {
13        string name;
14        int x;
15        ss >> name >> x;
16    }
17 }
```

7.11.3 手写 split

按指定字符分割路径、日期、配置项：

```

1 vector<string> split(const string& s, char sep) {
```

```
2   vector<string> res;
3   string cur;
4   for (char c : s) {
5       if (c == sep) {
6           res.push_back(cur);
7           cur.clear();
8       } else {
9           cur.push_back(c);
10      }
11  }
12  res.push_back(cur);
13  return res;
14 }
```

7.11.4 命令分发模板

命令较多时，可以让每种命令对应一个函数，主循环只负责读入和调用。

```
1 void handleAdd() {
2     int id, x;
3     cin >> id >> x;
4     // add logic
5 }
6
7 void handleDelete() {
8     int id;
9     cin >> id;
10    // delete logic
11 }
12
13 void handleQuery() {
14     int id;
15     cin >> id;
16     // query logic
17 }
18
19 void handleCommand() {
20     string op;
21     cin >> op;
22     if (op == "add") handleAdd();
23     else if (op == "delete") handleDelete();
24     else if (op == "query") handleQuery();
25 }
```

7.11.5 命令解析易错点

- 命令参数个数是否固定。

- 参数中是否可能包含空格。
- 命令大小写是否敏感。
- 非法命令是否会出现。
- 每条命令是否都可能输出。
- 查询不存在对象时输出什么。

7.12 状态设计

7.12.1 用结构体保存对象

```
1 struct User {  
2     int id;  
3     string name;  
4     int score;  
5     bool active;  
6 };  
7  
8 map<int, User> users;
```

7.12.2 状态编号

状态不多时可以用整数常量：

```
1 const int WAITING = 0;  
2 const int RUNNING = 1;  
3 const int FINISHED = 2;  
4 const int FAILED = 3;
```

也可以用 enum：

```
1 enum Status {  
2     WAITING,  
3     RUNNING,  
4     FINISHED,  
5     FAILED  
6 };
```

7.12.3 状态转移表

复杂状态题建议先画表：

当前状态	操作	新状态	输出/副作用
WAITING	start	RUNNING	记录开始时间
RUNNING	finish	FINISHED	计算得分

当前状态	操作	新状态	输出/副作用
RUNNING	fail	FAILED	输出错误信息

7.13 状态机模拟

7.13.1 什么题会用到

题面出现：

- 一个对象有多个状态。
- 不同状态下，同一个命令效果不同。
- 状态按规则切换。
- 非法状态转换需要忽略或报错。

7.13.2 状态机模板

```
1  const int WAITING = 0;
2  const int RUNNING = 1;
3  const int FINISHED = 2;
4
5  struct Job {
6      int id;
7      int status;
8      int score;
9  };
10
11 void start(Job& job) {
12     if (job.status != WAITING) {
13         return;
14     }
15     job.status = RUNNING;
16 }
17
18 void finish(Job& job) {
19     if (job.status != RUNNING) {
20         return;
21     }
22     job.status = FINISHED;
23     job.score += 10;
24 }
```

7.13.3 状态机检查

- 初始状态是什么。

- 每个操作允许在哪些状态下发生。
- 不允许的操作是忽略、报错，还是改变成特殊状态。
- 终止状态是否还能被修改。
- 状态改变时是否还要更新其他字段。

7.14 事件模拟

7.14.1 什么题会用到

题面出现：

- 按时间顺序发生事件。
- 事件有开始、结束、到达、离开。
- 同一时间有多个事件。
- 需要维护当前正在发生的对象集合。

7.14.2 事件结构体

```
1 struct Event {  
2     int time;  
3     int type;  
4     int id;  
5 };  
6  
7 vector<Event> events;
```

7.14.3 事件排序

```
1 sort(events.begin(), events.end(), [](const Event& a, const Event& b) {  
2     if (a.time != b.time) return a.time < b.time;  
3     return a.type < b.type; // depends on statement  
4 });
```

7.14.4 事件处理模板

```
1 for (const Event& e : events) {  
2     if (e.type == 0) {  
3         // start event  
4     } else if (e.type == 1) {  
5         // end event  
6     } else {  
7         // query event  
8     }  
9 }
```

7.14.5 同一时间事件顺序

这是事件模拟最容易错的地方。一定要从题面确定：

- 同一时间，结束事件先处理还是开始事件先处理。
- 查询是在事件前还是事件后。
- 如果没有说明，要从样例推断，但不要过度猜。
- 排序比较器中必须体现这个顺序。

7.15 优先级模拟

7.15.1 什么题会用到

题面出现：

- 每次选择优先级最高的任务。
- 优先级相同按时间、编号、字典序。
- 动态加入任务。
- 当前最大、当前最小。

7.15.2 priority_queue 模板

```
1 struct Task {
2     int priority;
3     int time;
4     int id;
5 };
6
7 struct Cmp {
8     bool operator()(const Task& a, const Task& b) const {
9         if (a.priority != b.priority) return a.priority < b.priority;
10        if (a.time != b.time) return a.time > b.time;
11        return a.id > b.id;
12    }
13 };
14
15 priority_queue<Task, vector<Task>, Cmp> pq;
```

这个比较器表示：

- 优先级大的先出。
- 优先级相同，时间小的先出。
- 时间相同，编号小的先出。

priority_queue 的比较器和 sort 相反，容易写错。不确定时，用几个任务手动推一下 `top()` 是谁。

7.16 复杂排序和排名

7.16.1 什么题会用到

题面出现：

- 多个排序关键字。
- 排名、积分榜、成绩表。
- 相同分数按编号、时间、字典序排序。
- 并列排名或去重排名。

7.16.2 复杂排序模板

```
1 struct Player {
2     int id;
3     int score;
4     int penalty;
5     string name;
6 };
7
8 sort(a.begin(), a.end(), [](const Player& x, const Player& y) {
9     if (x.score != y.score) return x.score > y.score;
10    if (x.penalty != y.penalty) return x.penalty < y.penalty;
11    if (x.name != y.name) return x.name < y.name;
12    return x.id < y.id;
13 });
```

7.16.3 并列排名

```
1 int rank = 1;
2 for (int i = 0; i < (int)a.size(); i++) {
3     if (i > 0 && a[i].score != a[i - 1].score) {
4         rank = i + 1;
5     }
6     cout << a[i].id << ' ' << rank << '\n';
7 }
```

7.16.4 复杂排序检查

- 每个关键字是升序还是降序。
- 相同关键字时是否继续比较。
- 是否要求稳定排序。
- 字符串按字典序还是输入顺序。
- 比较器相等时不能返回 `true`。

7.17 表格和矩阵模拟

7.17.1 什么题会用到

题面出现：

- 表格、座位、棋盘、网格。
- 行列操作。
- 旋转、翻转、交换行列。
- 查询某个单元格。

7.17.2 矩阵读入和输出

```
1 int n, m;
2 cin >> n >> m;
3 vector<vector<int>> a(n, vector<int>(m));
4 for (int i = 0; i < n; i++) {
5     for (int j = 0; j < m; j++) {
6         cin >> a[i][j];
7     }
8 }
```

7.17.3 交换两行

```
1 int r1, r2;
2 cin >> r1 >> r2;
3 --r1; --r2;
4 swap(a[r1], a[r2]);
```

7.17.4 交换两列

```
1 int c1, c2;
2 cin >> c1 >> c2;
3 --c1; --c2;
4 for (int i = 0; i < n; i++) {
5     swap(a[i][c1], a[i][c2]);
6 }
```

7.17.5 顺时针旋转矩阵

```
1 vector<vector<int>> rotateClockwise(const vector<vector<int>>& a) {
2     int n = (int)a.size();
3     if (n == 0) return {};
4     int m = (int)a[0].size();
5     vector<vector<int>> b(m, vector<int>(n));
```

```
6   for (int i = 0; i < n; i++) {
7       for (int j = 0; j < m; j++) {
8           b[j][n - 1 - i] = a[i][j];
9       }
10  }
11  return b;
12 }
```

7.17.6 表格模拟易错点

- 题目行列通常从 1 开始，代码常从 0 开始。
- 旋转后行数和列数会交换。
- 如果操作很多，真实移动整个矩阵可能超时，要考虑只维护行列映射。
- 输出每行空格格式。

7.18 目录树和路径模拟

7.18.1 什么题会用到

题面出现：

- 文件、目录、路径。
- /a/b/c、相对路径、绝对路径。
- 创建、删除、查询文件。
- 父目录、子目录。

7.18.2 路径规范化

下面处理绝对路径的 `.` 和 `..`；相对路径需先拼接当前目录。依赖前文 `split`：

```
1  vector<string> normalizePath(const string& path) {
2      vector<string> st;
3      vector<string> parts = split(path, '/');
4      for (string part : parts) {
5          if (part.empty() || part == ".") {
6              continue;
7          } else if (part == "..") {
8              if (!st.empty()) st.pop_back();
9          } else {
10             st.push_back(part);
11         }
12     }
13     return st;
14 }
```

7.18.3 路径转字符串

```
1 string joinPath(const vector<string>& parts) {
2     if (parts.empty()) return "/";
3     string res;
4     for (const string& p : parts) {
5         res += "/";
6         res += p;
7     }
8     return res;
9 }
```

7.18.4 目录树节点

```
1 struct Node {
2     string name;
3     map<string, int> child;
4     bool isFile = false;
5 };
6
7 vector<Node> tree;
```

新建节点：

```
1 int newNode(const string& name, bool isFile) {
2     Node node;
3     node.name = name;
4     node.isFile = isFile;
5     tree.push_back(node);
6     return (int)tree.size() - 1;
7 }
```

7.18.5 路径题易错点

- 根目录怎么表示。
- 连续斜杠是否可能出现。
- 路径末尾斜杠是否影响结果。
- 查询不存在路径时输出什么。
- 文件和目录是否允许同名。

7.19 URL 路由和参数解析

7.19.1 什么题会用到

题面出现：

- URL、路径、路由。
- /user/123/profile。
- 查询参数，例如 ?a=1&b=2。
- 通配符、变量匹配。

7.19.2 拆分 URL

```
1 struct Url {
2     string path;
3     map<string, string> query;
4 };
5
6 Url parseUrl(const string& s) {
7     Url res;
8     size_t qpos = s.find('?');
9     if (qpos == string::npos) {
10         res.path = s;
11         return res;
12     }
13
14     res.path = s.substr(0, qpos);
15     string qs = s.substr(qpos + 1);
16     vector<string> pairs = split(qs, '&');
17     for (string p : pairs) {
18         if (p.empty()) continue;
19         size_t eq = p.find('=');
20         if (eq == string::npos) {
21             res.query[p] = "";
22         } else {
23             string key = p.substr(0, eq);
24             string val = p.substr(eq + 1);
25             res.query[key] = val;
26         }
27     }
28     return res;
29 }
```

7.19.3 简单路由匹配

模式 /user/<id> 匹配 /user/123。先定义后文的 `parsePath`，或在函数前声明它。此简化版忽略连续/末尾斜杠，仅支持整段参数，不支持 URL 映射真题的类型参数和剩余路径规则：

```
1 bool matchRoute(const string& pattern, const string& path,
2                 map<string, string>& params) {
3     params.clear();
4     map<string, string> candidate;
```

```
5   vector<string> a = parsePath(pattern);
6   vector<string> b = parsePath(path);
7   if (a.size() != b.size()) return false;
8
9   for (int i = 0; i < (int)a.size(); i++) {
10      if (!a[i].empty() && a[i][0] == '<' && a[i].back() == '>') {
11         string name = a[i].substr(1, (int)a[i].size() - 2);
12         candidate[name] = b[i];
13      } else if (a[i] != b[i]) {
14         return false;
15      }
16   }
17   params = move(candidate);
18   return true;
19 }
```

7.19.4 URL 题易错点

- 有没有查询参数。
- 参数值是否可能为空。
- 路径末尾斜杠是否等价。
- 多个路由都匹配时，第一条生效还是更具体的生效。
- URL 是否需要解码，题目不要求就不要自己加。

7.20 配置和 JSON 类解析入口

7.20.1 什么题会用到

题面出现：

- 键值对。
- 花括号、方括号、冒号、逗号。
- 嵌套对象。
- 查询某个字段。

7.20.2 简单键值配置

每行形如 key=value:

```
1  int n;
2  cin >> n;
3  map<string, string> conf;
4  for (int i = 0; i < n; i++) {
5      string line;
6      cin >> line;
```

```
7   int pos = (int)line.find('=');
8   string key = line.substr(0, pos);
9   string value = line.substr(pos + 1);
10  conf[key] = value;
11 }
```

7.20.3 读取带空格的配置值

```
1 string line;
2 getline(cin, line);
3 int pos = (int)line.find('=');
4 string key = line.substr(0, pos);
5 string value = line.substr(pos + 1);
```

7.20.4 嵌套结构处理思路

JSON 类题先看题目允许哪些数据类型和格式，只实现题目实际需要的部分：

- 是否只有字符串和对象，没有数组。
- 是否保证格式合法。
- 查询是否只按点号路径。
- 是否只需要保存完整路径到值的映射。

如果只需要查询路径，可以把嵌套字段拍平成 key：

```
1 // Example flattened keys:
2 // user.name -> Alice
3 // user.age -> 18
4 map<string, string> value;
```

配置/JSON 题先利用限制。CSP 题通常不会要求你写工业级 JSON 解析器，重点是按题目给定范围实现。

7.21 字符串解析

7.21.1 什么题会用到

遇到下面这些格式时，需要先想清楚字符串怎样拆开：

- 路径，例如 /a/b/c。
- 配置项，例如 key=value。
- 表达式，例如括号、加减乘除。
- 带引号的字符串。
- 嵌套结构。
- 命令参数中可能包含空格。

7.21.2 逐字符扫描模板

```
1 string s;  
2 cin >> s;  
3  
4 for (int i = 0; i < (int)s.size(); i++) {  
5     char c = s[i];  
6     if (isdigit((unsigned char)c)) {  
7         int j = i;  
8         while (j < (int)s.size() && isdigit((unsigned char)s[j])) j++;  
9         string number = s.substr(i, j - i);  
10        i = j - 1;  
11    } else if (isalpha((unsigned char)c)) {  
12        int j = i;  
13        while (j < (int)s.size() && isalpha((unsigned char)s[j])) j++;  
14        string word = s.substr(i, j - i);  
15        i = j - 1;  
16    } else {  
17        // symbol  
18    }  
19 }
```

7.21.3 解析 key=value

```
1 string s;  
2 cin >> s;  
3 int pos = (int)s.find('=');  
4 string key = s.substr(0, pos);  
5 string value = s.substr(pos + 1);
```

7.21.4 路径解析

```
1 vector<string> parsePath(const string& path) {  
2     vector<string> parts;  
3     string cur;  
4     for (char c : path) {  
5         if (c == '/') {  
6             if (!cur.empty()) {  
7                 parts.push_back(cur);  
8                 cur.clear();  
9             }  
10        } else {  
11            cur.push_back(c);  
12        }  
13    }  
14    if (!cur.empty()) parts.push_back(cur);  
15    return parts;
```

16 }

7.21.5 括号匹配

```
1 bool checkBrackets(const string& s) {  
2     stack<char> st;  
3     for (char c : s) {  
4         if (c == '(' || c == '[' || c == '{') {  
5             st.push(c);  
6         } else if (c == ')' || c == ']' || c == '}') {  
7             if (st.empty()) return false;  
8             char t = st.top();  
9             st.pop();  
10            if (c == ')' && t != '(') return false;  
11            if (c == ']' && t != '[') return false;  
12            if (c == '}' && t != '{') return false;  
13        }  
14    }  
15    return st.empty();  
16 }
```

7.21.6 字符串解析易错点

- `cin >> s` 不能读取空格。
- `getline` 前可能需要 `cin.ignore()`。
- `find` 找不到时返回 `string::npos`。
- `substr(pos, len)` 的第二个参数是长度，不是结束位置。
- 连续分隔符和开头/结尾分隔符要单独考虑。
- 字符串数字可能超过 `long long`。

7.22 表达式解析入口

7.22.1 什么题会用到

题面出现：

- 表达式、括号、运算符。
- 加减乘除优先级。
- 逻辑表达式。
- 嵌套结构。

7.22.2 只含加减的表达式

```

1 long long evalPlusMinus(const string& s) {
2     long long ans = 0;
3     long long num = 0;
4     int sign = 1;
5
6     for (int i = 0; i <= (int)s.size(); i++) {
7         if (i < (int)s.size() && isspace((unsigned char)s[i])) continue;
8         if (i < (int)s.size() && isdigit((unsigned char)s[i])) {
9             num = num * 10 + (s[i] - '0');
10        } else {
11            ans += sign * num;
12            num = 0;
13            if (i < (int)s.size()) {
14                if (s[i] == '+') sign = 1;
15                else if (s[i] == '-') sign = -1;
16            }
17        }
18    }
19    return ans;
20 }

```

完整整数四则运算见 [T610²¹²](#)；下面仅展示读取一个数字的基础片段。

7.22.3 递归解析思想

复杂表达式通常需要递归下降。新手先掌握思想：

```

1 string s;
2 int pos = 0;
3
4 long long parseNumber() {
5     long long x = 0;
6     while (pos < (int)s.size() && isdigit((unsigned char)s[pos])) {
7         x = x * 10 + (s[pos] - '0');
8         pos++;
9     }
10    return x;
11 }

```

表达式题如果没练过，不要强行写复杂递归下降。先拿没有括号、没有优先级的部分分。

7.23 缓存和队列模拟

7.23.1 什么题会用到

题面出现：

- 缓存、页面置换、最近使用。

- 队列、排队、服务窗口。
- 先进先出、最近最少使用。
- 容量限制，满了要删除一个。

7.23.2 FIFO 缓存

```
1 int cap;
2 cin >> cap;
3 queue<int> q;
4 set<int> inCache;
5
6 for (int x : requests) {
7     if (cap <= 0) continue;
8     if (inCache.count(x)) {
9         continue;
10    }
11    if ((int)q.size() == cap) {
12        int old = q.front();
13        q.pop();
14        inCache.erase(old);
15    }
16    q.push(x);
17    inCache.insert(x);
18 }
```

7.23.3 LRU 缓存入口

LRU 可用 `list + unordered_map`: 链表记访问顺序, 映射保存每个对象的位置。

```
1 int cap;
2 list<int> order; // front is most recent
3 unordered_map<int, list<int>::iterator> pos;
4
5 void access(int x) {
6     if (cap <= 0) return;
7     if (pos.count(x)) {
8         order.erase(pos[x]);
9     } else if ((int)order.size() == cap) {
10        int old = order.back();
11        order.pop_back();
12        pos.erase(old);
13    }
14    order.push_front(x);
15    pos[x] = order.begin();
16 }
```

7.23.4 队列模拟易错点

- 队列为空时不能取 front。
- 缓存容量可能为 0，要看题目是否允许。
- 命中缓存时是否需要更新时间。
- 删除元素时，辅助集合或 map 也要同步删除。

7.24 懒删除优先队列

7.24.1 什么题会用到

题面出现：

- 需要动态删除任务。
- 还要每次取最大/最小。
- priority_queue 中间删除不方便。

7.24.2 懒删除思想

不直接从堆里删除旧元素，而是在取堆顶时检查它是否还有效。

```
1 struct Task {
2     int priority;
3     int id;
4 };
5
6 struct Cmp {
7     bool operator()(const Task& a, const Task& b) const {
8         if (a.priority != b.priority) return a.priority < b.priority;
9         return a.id > b.id;
10    }
11 };
12
13 priority_queue<Task, vector<Task>, Cmp> pq;
14 map<int, int> currentPriority;
15
16 void addOrUpdate(int id, int priority) {
17     currentPriority[id] = priority;
18     pq.push({priority, id});
19 }
20
21 void removeTask(int id) {
22     currentPriority.erase(id);
23 }
24
25 Task getTop() {
26     while (!pq.empty()) {
```

```
27     Task t = pq.top();
28     if (currentPriority.count(t.id) &&
29         currentPriority[t.id] == t.priority) {
30         return t;
31     }
32     pq.pop();
33 }
34 return {-1, -1};
35 }
```

懒删除会让堆里有废数据。 每次取 top 前必须清理过期元素。

7.25 时间和日程模拟

7.25.1 什么题会用到

题面出现：

- 时间段、会议、日程。
- 判断冲突。
- 开始时间、结束时间。
- 按时间排序处理。

7.25.2 时间转分钟

```
1 int toMinute(const string& s) {
2     int h = stoi(s.substr(0, 2));
3     int m = stoi(s.substr(3, 2));
4     return h * 60 + m;
5 }
```

7.25.3 判断区间冲突

两个半开区间 $[l1, r1)$ 和 $[l2, r2)$ 冲突：

```
1 bool overlap(int l1, int r1, int l2, int r2) {
2     return max(l1, l2) < min(r1, r2);
3 }
```

如果题目是闭区间，条件要改。

7.25.4 扫描事件统计同时进行数量

```
1 vector<pair<int, int>> events;
2 for (auto [l, r] : intervals) {
3     if (l >= r) continue; // empty half-open interval
4     events.push_back({l, 1});
```

```
5     events.push_back({r, -1});
6 }
7
8 sort(events.begin(), events.end(), [](auto a, auto b) {
9     if (a.first != b.first) return a.first < b.first;
10    return a.second < b.second; // end before start for [l,r)
11 });
12
13 int cur = 0, best = 0;
14 for (auto [time, delta] : events) {
15     cur += delta;
16     best = max(best, cur);
17 }
```

7.25.5 时间题易错点

- 时间段是闭区间还是半开区间。
- 结束时间和开始时间相同是否冲突。
- 跨天如何处理。
- 同一时间开始和结束谁先处理。

7.26 树形结构模拟

7.26.1 什么题会用到

题面出现：

- 层级、父子、目录、组织结构。
- 每个节点有父节点。
- 查询祖先、子树、路径。

7.26.2 父子关系存储

```
1 int n;
2 cin >> n;
3 vector<vector<int>> children(n + 1);
4 vector<int> parent(n + 1, 0);
5
6 for (int i = 2; i <= n; i++) {
7     int p;
8     cin >> p;
9     parent[i] = p;
10    children[p].push_back(i);
11 }
```

7.26.3 DFS 遍历树

```
1 vector<int> depth(n + 1, 0);
2
3 void dfsTree(int u) {
4     for (int v : children[u]) {
5         depth[v] = depth[u] + 1;
6         dfsTree(v);
7     }
8 }
```

7.26.4 子树大小

```
1 vector<int> subtree(n + 1, 1);
2
3 void calcSubtree(int u) {
4     subtree[u] = 1;
5     for (int v : children[u]) {
6         calcSubtree(v);
7         subtree[u] += subtree[v];
8     }
9 }
```

7.26.5 树题易错点

- 根节点是谁。
- 输入给的是父节点，还是边。
- 节点编号从 0 还是 1 开始。
- 树很深时递归可能爆栈。
- 如果需要频繁祖先查询，简单爬父亲可能超时。

7.27 权限和规则匹配

7.27.1 什么题会用到

题面出现：

- 用户、角色、权限。
- 规则匹配。
- 黑名单、白名单。
- 多条规则按顺序匹配，第一条生效。
- 默认允许或默认拒绝。

7.27.2 规则结构体

```
1 struct Rule {  
2     string user;  
3     string action;  
4     string resource;  
5     bool allow;  
6 };  
7  
8 vector<Rule> rules;
```

7.27.3 规则匹配函数

```
1 bool match(const Rule& r, const string& user,  
2           const string& action, const string& resource) {  
3     if (r.user != "*" && r.user != user) return false;  
4     if (r.action != "*" && r.action != action) return false;  
5     if (r.resource != "*" && r.resource != resource) return false;  
6     return true;  
7 }
```

7.27.4 按顺序匹配

```
1 bool checkPermission(const string& user,  
2                     const string& action,  
3                     const string& resource) {  
4     for (const Rule& r : rules) {  
5         if (match(r, user, action, resource)) {  
6             return r.allow;  
7         }  
8     }  
9     return false; // default deny  
10 }
```

7.27.5 规则题易错点

- 多条规则是第一条生效，还是最后一条生效。
- 默认值是允许还是拒绝。
- 通配符如何匹配。
- 大小写是否敏感。
- 资源路径是否需要前缀匹配。

7.28 网格和图混合题

7.28.1 什么题会用到

题面出现：

- 地图、障碍、传送门、钥匙、门。
- 不只是简单最短路，还有状态。
- 每个格子可能有属性。
- 移动过程中会改变状态。

7.28.2 普通网格 BFS

如果只有障碍和空地，用 B 题 BFS 模板即可。

7.28.3 带状态 BFS 入口

如果移动时还带着状态，例如是否拿到钥匙，可以把状态加入距离数组。

```
1 struct Node {
2     int x;
3     int y;
4     int state;
5 };
6
7 vector<vector<vector<int>>> dist(n,
8     vector<vector<int>>(m, vector<int>(S, -1)));
9
10 queue<Node> q;
11 dist[sx][sy][0] = 0;
12 q.push({sx, sy, 0});
13
14 while (!q.empty()) {
15     Node cur = q.front();
16     q.pop();
17
18     for (int d = 0; d < 4; d++) {
19         int nx = cur.x + dx[d];
20         int ny = cur.y + dy[d];
21         int ns = cur.state;
22         if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
23         if (g[nx][ny] == '#') continue;
24         // Check door permission, then update ns from g[nx][ny].
25         // Do not index the grid before the bounds check.
26         if (dist[nx][ny][ns] != -1) continue;
27         dist[nx][ny][ns] = dist[cur.x][cur.y][cur.state] + 1;
28         q.push({nx, ny, ns});
29     }
```

本段仅是状态扩展框架，必须按题意补全门禁检查、状态转移和起点状态。空间约为 $4nmS$ 字节，先算总量。

状态数不能太大。如果状态有 2^k ，先看 k 是否很小。 $k \leq 10$ 常见可做， $k = 30$ 通常不行。

7.28.4 图模拟策略

如果题目是图但规则很多：

- 先把图建对：有向/无向、权值、编号。
- 再处理规则：哪些边能走，什么时候能走。
- 如果边权全为 1，用 BFS。
- 如果边权为正，用 Dijkstra。
- 如果只是连通关系，用并查集或 DFS。

7.29 大模拟易错点

- 同一时间多个事件的顺序。
- 相同优先级的并列规则。
- 删除元素后迭代器失效。
- 查询不存在对象。
- 重复添加同一对象。
- 空集合时取最大、最小。
- 最后一轮事件是否处理。
- 题目中的编号是 0 开始还是 1 开始。
- 输出顺序是否要求按编号、时间或字典序。

7.30 C 题分阶段实现

7.30.1 不要一次写完

每写完一种命令就运行一次，检查前后状态。可以按下面的顺序逐步完成：

1. 只写读入，打印读到的数据检查。
2. 写数据结构，不写复杂规则。
3. 实现第一种命令或最基础操作。
4. 跑样例，看是否能得到部分中间结果。
5. 再补第二种命令。
6. 最后处理特殊规则、并列规则、异常情况。

7.30.2 命令题实现顺序

阶段	做什么	目标
1	读入所有命令，只识别命令名	确认输入没错。
2	实现新增/初始化类命令	建立基本状态。
3	实现查询类命令	能输出最简单答案。
4	实现修改/删除类命令	状态开始完整。
5	实现特殊规则	冲满分。

7.30.3 模拟题实现顺序

1. 先实现没有特殊情况的主流程。
2. 再实现边界检查。
3. 再实现冲突和优先级。
4. 最后实现异常输入或题目特别说明的细节。

7.31 拆分函数时的检查

代码较长时，把解析、查询和修改拆开，定位问题会方便一些。

7.31.1 建议函数类型

函数类型	示例	作用
读入函数	<code>readInput()</code>	避免读入和处理混在一起。
解析函数	<code>parsePath(s)</code>	字符串处理单独测试。
查询函数	<code>query(id)</code>	查询逻辑独立。
修改函数	<code>update(id,x)</code>	状态改变集中处理。
检查函数	<code>canMove(x,y)</code>	规则判断集中处理。
输出函数	<code>printAnswer()</code>	控制格式。

7.31.2 函数设计原则

- 一个函数只做一类事情。
- 函数名直接反映题目操作。
- 一个函数尽量只处理一类操作；几种互不相关的规则可以拆开。
- 修改状态的函数要特别小心副作用。
- 查询函数尽量不要修改状态，除非题目要求。

7.31.3 推荐代码顺序

```
1 // constants and types
2 struct ...
3
4 // global variables
5 int n, m;
6 vector<...> ...
7
8 // helper functions
9 bool inside(...);
10 vector<string> split(...);
11
12 // operation functions
13 void handleAdd(...);
14 void handleQuery(...);
15
16 // main solve
17 void solve(){ ... }
```

7.32 分层测试方法

7.32.1 第一层：解析测试

只测试输入解析，不测算法。

- 路径是否被正确分割。
- 命令参数是否读对。
- 数字字符串是否转换正确。
- 空格和换行是否处理正确。

7.32.2 第二层：单操作测试

每种操作单独测：

- 只 add 一次。
- 只 query 一次。
- add 后 query。
- delete 后 query。

7.32.3 第三层：组合测试

测试多操作交替：

- add、delete、add 同一个对象。

- 多个对象按优先级排序。
- 同一时间多个事件。
- 状态反复切换。

7.32.4 第四层：边界测试

测试异常和边界：

- 空集合查询。
- 不存在对象。
- 重复对象。
- 最大数据附近。
- 最后一条命令。

7.33 C 题常见失分原因

失分原因	表现	预防
读题漏规则	样例某一步对不上	三遍读题，列规则表。
状态没初始化	第一次查询就错	结构体构造或统一 init。
命令读错	后续全错	先调试输出 op 和参数。
比较器错	排名/优先级错	手写 3 个对象测试 top/order。
删除不同步	查到已删除对象	容器、set、map 同步维护。
最后事件漏处理	前面都对，最后错	检查循环结束后的收尾。
输出顺序错	数值对但顺序错	看题目要求按什么顺序。
复杂度过高	小样例过，大数据超时	估算每条命令复杂度。

7.34 C 题本地调试

7.34.1 调试输出建议

调试时输出状态：

```
1 void debugTask(const Task& t) {
2     cerr << "id=" << t.id
3         << " status=" << t.status
4         << " priority=" << t.priority << '\n';
5 }
```

处理命令时：

```
1 cerr << "op=" << op << '\n';
```

提交前删除所有调试输出。C 题代码长，尤其容易漏掉 cerr 或临时 cout。

7.34.2 缩小样例

如果官方样例很长，自己造小样例：

- 1 个对象，1 条命令。
- 1 个对象，2 条命令。
- 2 个对象，测试排序或优先级。
- 不存在对象的查询。
- 重复添加和删除。

7.34.3 定位 bug 的方法

1. 先确认读入正确。
2. 再确认每条命令被识别正确。
3. 再确认命令前状态正确。
4. 再确认命令后状态正确。
5. 最后确认输出格式。

7.35 C 题常见边界库

题型	边界	检查目标
命令模拟	只有 1 条命令、没有查询、连续查询	初始化和输出。
新增删除	重复新增、删除不存在、删除后查询	对象生命周期。
状态机	非法状态转换、终止状态继续操作	状态规则。
事件模拟	同一时间多个事件、开始和结束同一时刻	事件排序。
优先级	优先级相同、时间相同、编号相同附近	比较器。
字符串解析	空字段、连续分隔符、路径以分隔符结尾	split 和 substr。
网格	起点等于终点、无路、只有一个格子	BFS 初始化。
带状态 BFS	拿不到状态、重复状态、状态数最大	dist 维度。
图	自环、重边、不连通、有向/无向	建图。

7.36 C 题部分分策略

先看子任务限制。比如“没有删除命令”“没有嵌套结构”或“对象数量很少”，都可能允许更短的实现。选定一档后，把这一档会出现的规则写完整；只实现几条命令，却忽略它们会遇到的并列和边界规则，未必能得分。

7.37 按子任务逐步完成

可以按下面的顺序写，每完成一步就用小样例检查一次。

7.37.1 读人与基本操作

确认所有输入都能读完，每种命令的参数个数正确。实现初始状态和最简单的操作，检查一次新增或修改之后，查询能否得到预期结果。

7.37.2 完成选定子任务

补齐这个子任务会用到的所有命令、并列规则和异常处理。再试重复添加、删除不存在对象、空集合查询等边界。测试点有多次输出时，所有输出都要符合题意。

7.37.3 扩大适用范围

时间允许时，再加入嵌套、更复杂的规则或更快的数据结构。修改前保留已经检查过的版本，新版本通过对照后再替换。若题目采用捆绑测试，还要满足该子任务的全部要求。

7.38 来不及写完时

7.38.1 写不完大模拟

- 保留已完成命令，不要为了补复杂规则改崩基础逻辑。
- 先完成子任务所需的状态修改，再检查相应查询输出。
- 特殊规则用清晰的 `if` 补，不要重构大段代码。

7.38.2 字符串解析写不出来

- 若有不含空格、不含嵌套的子任务，可以先处理这一档。
- 先用 `split` 处理简单分隔符。
- 没有嵌套的子任务仍要正确处理其中所有 `token` 和输出。

7.38.3 状态 BFS 写不出来

- 若子任务不含特殊状态，可以先写普通 BFS。
- 如果状态只有 2 种，尝试 `dist[x][y][2]`。
- 如果有不含特殊状态的子任务，可以先完成这一档。

7.38.4 复杂度可能超时

- 先估每条命令复杂度。
- 能用 map 查找就不要全表扫描。
- 能预处理就不要每次重复计算。
- 时间不够时，保留暴力拿小数据分。

7.39 不同题型的实现顺序

7.39.1 如果是大模拟

1. 先读懂输入输出。
2. 建结构体保存对象。
3. 实现最简单命令。
4. 实现查询输出。
5. 再补删除、修改、异常情况。
6. 最后补优先级、并列规则。

7.39.2 如果是字符串解析

1. 先确认用 `cin` 还是 `getline`。
2. 先把字符串拆成 `token`。
3. 能用 `split` 就不要写复杂解析。
4. 如果有括号，先写括号匹配。
5. 如果有嵌套，先拿无嵌套部分分。

7.39.3 如果是图或网格

1. 先把图或网格读入正确。
2. 判断是最短路、连通块还是模拟移动。
3. 无权最短路用 BFS。
4. 带状态时先估算状态数量。
5. 只有子任务确实不含特殊状态时，才能用普通 BFS 简化。

7.39.4 如果是规则匹配

1. 把每条规则保存为结构体。
2. 单独写 `match` 函数。
3. 确认第一条生效还是最后一条生效。

4. 确认默认值。
5. 用 2 到 3 条规则手测。

7.40 C 题模板索引

题面关键词	使用模板	本章位置
多命令	命令分发	命令解析
对象状态	struct + 状态机	状态设计、状态机模拟
按时间发生	Event + sort	事件模拟
优先级	priority_queue	优先级模拟
排名	多关键字 sort	复杂排序和排名
表格、矩阵	vector<vector<int>>	表格和矩阵模拟
路径、目录	split + normalizePath	目录树和路径模拟
URL、参数	parseUrl	URL 路由和参数解析
配置、键值	key=value	配置和 JSON 类解析
括号、表达式	stack / 递归解析	字符串解析、表达式解析
缓存	queue / list + map	缓存和队列模拟
动态删除最大值	懒删除堆	懒删除优先队列
时间段冲突	overlap / events	时间和日程模拟
父子层级	children + DFS	树形结构模拟
地图状态	state BFS	网格和图混合题

7.41 C 题考场速查页

7.41.1 读题 5 分钟内要确定

- 是大模拟、字符串解析、图网格、数据结构，还是 DP。
- 有哪些对象。
- 每个对象有哪些状态。
- 有哪些命令或事件。
- 输出发生在每次查询还是最后。
- 数据范围是否允许全表扫描。

7.41.2 写代码前先想清楚

- struct 里有哪些字段。
- 用 vector、map、set 还是堆。
- 每个命令对应哪个函数。
- 比较器的排序规则。

- 不存在、重复、越界怎么处理。

7.41.3 最常用代码形状

```
1 struct Obj {
2     int id;
3     int status;
4 };
5
6 map<int, Obj> obj;
7
8 void handleAdd() {}
9 void handleDelete() {}
10 void handleQuery() {}
11
12 void solve() {
13     int q;
14     cin >> q;
15     while (q--> {
16         string op;
17         cin >> op;
18         if (op == "add") handleAdd();
19         else if (op == "delete") handleDelete();
20         else if (op == "query") handleQuery();
21     }
22 }
```

7.41.4 最后 10 分钟检查

- 删除调试输出。
- 检查所有命令是否都处理。
- 检查输出大小写、空格、换行。
- 检查比较器并列规则。
- 检查空集合、找不到对象。
- 检查最后一个事件。
- 提交一个当前最稳版本。

7.42 C 题提交前检查

- 是否删除所有调试输出。
- 每种命令是否都处理了。
- 每种状态是否都有初始值。

- 不存在对象时是否按题意处理。
- 重复对象是否按题意处理。
- 排序和优先级比较器是否正确。
- 字符串读取是否会漏掉空格。
- 多组数据是否清空所有容器。
- 最后一个事件或最后一段数据是否处理。
- 输出顺序是否符合题目。

7.43 C 题模板练习清单

模板	熟练要求	用途
结构体 + map 保存对象	建议掌握	对象状态模拟。
命令分发	建议掌握	多命令题。
getline + stringstream	建议掌握	整行命令。
split	照着能写	路径、配置解析。
状态机	照着能写	多状态对象。
事件排序	照着能写	时间顺序模拟。
priority_queue 自定义比较	练过再用	优先级模拟。
括号匹配	照着能写	表达式/嵌套结构。
带状态 BFS	理解后再用	网格状态题。

第八章 D/E 题部分分策略与高级算法入口

8.1 D/E 题定位

D/E 题可以先找小数据、特殊结构，或自己练过的做法。能独立完成一个子任务，就先把它写好；暂时不会的优化留到后面。

- D 题先看子任务，按会写的做法估算能覆盖哪些数据。
- E 题也可以找小数据或特殊性质，但要给前面会做的题留够时间。
- 先完成选定子任务，再考虑扩大适用范围。
- 思考很久仍没进展时，可以先保存思路，转做其他题。

如果已经花了十几二十分钟仍没有完整思路，可以重新看子任务表，或者暂时换题。

8.2 D/E 题读题流程

1. 先看有没有子任务或部分分说明。
2. 抄下所有数据范围，尤其是小数据档。
3. 找特殊性质：链、树、DAG、无修改、无查询、边权相同、参数很小。
4. 写出最暴力做法。
5. 估算暴力能拿哪些数据点。
6. 如果会优化，再逐步优化。

8.3 部分分决策树

题面或数据特征	可写做法	目标
$n \leq 20$	DFS、枚举子集、状态压缩	小数据分
$n \leq 100$	$O(n^3)$ 、Floyd、区间 DP	小中数据
$n \leq 1000$	$O(n^2)$	中小数据
图是树	DFS、树形 DP、LCA 入门	特殊性质分
图是链	转成数组问题	特殊性质分
边权全为 1	BFS	替代 Dijkstra
没有修改	前缀和、静态预处理	静态数据分
查询很少	每次暴力	小 q 分

题面或数据特征	可写做法	目标
修改很少	修改后重算	小修改分
参数 k 很小	状压、枚举 k	参数小分
求阈值且可行性单调	二分答案 (check 须正确)	进一步优化

8.4 暴力模板

8.4.1 枚举所有点对

```

1 long long ans = 0;
2 for (int i = 0; i < n; i++) {
3     for (int j = i + 1; j < n; j++) {
4         // check pair (i, j)
5     }
6 }

```

适用： $n \leq 5000$ 需谨慎， $n \leq 1000$ 通常可试。

8.4.2 枚举所有区间

```

1 for (int l = 0; l < n; l++) {
2     long long sum = 0;
3     for (int r = l; r < n; r++) {
4         sum += a[r];
5         // process interval [l, r]
6     }
7 }

```

适用： $n \leq 5000$ 以内看时限；如果 $n = 10^5$ ，只能拿小数据分。

8.4.3 DFS 枚举选择

```

1 int n;
2 vector<int> choose;
3
4 void dfs(int idx) {
5     if (idx == n) {
6         // evaluate choose
7         return;
8     }
9
10    // not choose idx
11    dfs(idx + 1);
12
13    // choose idx
14    choose.push_back(idx);

```

```

15     dfs(idx + 1);
16     choose.pop_back();
17 }

```

适用： $n \leq 20$ 左右。

8.5 小数据常用算法

8.5.1 Floyd

点数小且问任意两点最短路：

```

1 for (int k = 1; k <= n; k++) {
2     for (int i = 1; i <= n; i++) {
3         for (int j = 1; j <= n; j++) {
4             if (dist[i][k] == INF || dist[k][j] == INF) continue;
5             dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
6         }
7     }
8 }

```

8.5.2 全排列

```

1 vector<int> p(n);
2 iota(p.begin(), p.end(), 0);
3 do {
4     // evaluate permutation
5 } while (next_permutation(p.begin(), p.end()));

```

适用： $n \leq 9$ 或 10 左右。

8.5.3 状压枚举子集

```

1 for (int mask = 0; mask < (1 << n); mask++) {
2     for (int i = 0; i < n; i++) {
3         if (mask & (1 << i)) {
4             // i is selected
5         }
6     }
7 }

```

适用： $n \leq 20$ 左右。若 $n \geq 31$ ，注意 $1LL \ll n$ 和复杂度。

8.6 特殊性质策略

8.6.1 图是链

如果图是一条链，很多图问题可以转成数组问题：

- 距离变成下标差或前缀和。
- 子树变成连续区间。
- 路径查询变成区间查询。

8.6.2 图是树

树比一般图简单：

- 没有环。
- 两点之间路径唯一。
- 可用 DFS 统计子树。
- 可用树形 DP。

8.6.3 边权相同

如果所有边权都是 1：

- 最短路用 BFS。
- 不需要 Dijkstra。

8.6.4 无修改

如果没有修改，只有查询：

- 优先预处理。
- 区间问题考虑前缀和。
- 图问题考虑一次 BFS/Dijkstra 后回答多次查询。

8.7 按题型拿部分分

8.7.1 图论题

不会满分时	可写做法	可能覆盖
不会复杂最短路	小图 Floyd	n 小的数据
不会带权最短路	无权 BFS	边权全 1 的数据
不会动态图	每次修改后重跑 BFS/DFS	操作数小的数据
不会强连通	小图逐点搜索，判断双向可达	$O(n(n+m))$ ，单次 DFS 不能代替强连通
不会树上高级算法	从每个查询端点暴力爬/DFS	n, q 小的数据

8.7.2 区间数据结构题

不会满分时	可写做法	可能覆盖
不会线段树	每次暴力扫描区间	小 nq 数据
只有查询无修改	前缀和	静态数据
只有区间修改最后输出	差分	离线数据
单点修改区间查询	树状数组入口	中等数据
区间修改单点查询	差分思想 / 树状数组	中等数据

8.7.3 DP 题

不会满分时	可写做法	可能覆盖
状态想不清	DFS 暴力枚举	n 小的数据
背包优化不会	二维 DP	小容量数据
状态压缩不会	回溯搜索	n 小的数据
转移太复杂	只处理部分状态或特殊参数	特殊性质数据

8.7.4 字符串题

不会满分时	可写做法	可能覆盖
不会 KMP	find 或暴力匹配	短字符串数据
不会 Trie	map 存完整字符串	字符串数量较少
不会复杂解析	split 简单格式	无嵌套数据
不会哈希	直接比较字符串	小数据

8.8 树状数组入口

8.8.1 什么题会用到

- 单点修改，前缀和或区间和查询。
- 需要动态维护数组和。
- 数据范围 $n, q \leq 10^5$ 。

8.8.2 树状数组模板

```

1 struct BIT {
2     int n;
3     vector<long long> tree;
4
5     BIT(int n) : n(n), tree(n + 1, 0) {}
6
7     void add(int idx, long long val) {
8         assert(1 <= idx && idx <= n); // idx=0 would never advance

```

```

9      for (; idx <= n; idx += idx & -idx) {
10          tree[idx] += val;
11      }
12  }
13
14  long long sumPrefix(int idx) {
15      assert(0 <= idx && idx <= n);
16      long long res = 0;
17      for (; idx > 0; idx -= idx & -idx) {
18          res += tree[idx];
19      }
20      return res;
21  }
22
23  long long rangeSum(int l, int r) {
24      return sumPrefix(r) - sumPrefix(l - 1);
25  }
26  };

```

树状数组一般用 1 下标。

8.9 线段树入口

8.9.1 什么题会用到

- 区间查询和区间修改交替出现。
- 需要维护区间和、区间最大值、区间最小值。
- $n, q \leq 10^5$ ，暴力扫描会超时。

8.9.2 区间和线段树框架

```

1  struct SegTree {
2      int n;
3      vector<long long> tree, lazy;
4
5      SegTree(int n) : n(n), tree(4 * n + 4), lazy(4 * n + 4) {}
6
7      void apply(int node, int l, int r, long long val) {
8          tree[node] += val * (r - l + 1);
9          lazy[node] += val;
10     }
11
12     void push(int node, int l, int r) {
13         if (lazy[node] == 0 || l == r) return;
14         int mid = (l + r) / 2;
15         apply(node * 2, l, mid, lazy[node]);
16         apply(node * 2 + 1, mid + 1, r, lazy[node]);

```

```

17     lazy[node] = 0;
18 }
19
20 void rangeAdd(int node, int l, int r, int ql, int qr, long long val) {
21     if (ql <= l && r <= qr) {
22         apply(node, l, r, val);
23         return;
24     }
25     push(node, l, r);
26     int mid = (l + r) / 2;
27     if (ql <= mid) rangeAdd(node * 2, l, mid, ql, qr, val);
28     if (qr > mid) rangeAdd(node * 2 + 1, mid + 1, r, ql, qr, val);
29     tree[node] = tree[node * 2] + tree[node * 2 + 1];
30 }
31
32 long long query(int node, int l, int r, int ql, int qr) {
33     if (ql <= l && r <= qr) return tree[node];
34     push(node, l, r);
35     int mid = (l + r) / 2;
36     long long res = 0;
37     if (ql <= mid) res += query(node * 2, l, mid, ql, qr);
38     if (qr > mid) res += query(node * 2 + 1, mid + 1, r, ql, qr);
39     return res;
40 }
41 };

```

调用：

```

1 // Require n >= 1; initial array is all zero.
2 SegTree seg(n);
3 // For each initial a[i], call rangeAdd(1,1,n,i,i,a[i]).
4 seg.rangeAdd(1, 1, n, l, r, v);
5 long long ans = seg.query(1, 1, n, l, r);

```

8.9.3 不会线段树时怎么办

- 如果没有修改，用前缀和。
- 如果只有区间修改、最后查询，用差分。
- 如果是单点修改、区间查询，用树状数组。
- 如果数据小，直接暴力扫描。

8.10 Kruskal 最小生成树入口

8.10.1 什么题会用到

- 连接所有点。

- 总代价最小。
- 无向图。
- 选择若干边让图连通。

8.10.2 Kruskal 模板

```

1 struct Edge {
2     int u, v;
3     long long w;
4 };
5
6 sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
7     return a.w < b.w;
8 });
9
10 DSU dsu(n);
11 long long ans = 0;
12 int cnt = 0;
13 for (auto e : edges) {
14     if (dsu.unite(e.u, e.v)) {
15         ans += e.w;
16         cnt++;
17     }
18 }
19
20 if (cnt == n - 1) cout << ans << '\n';
21 else cout << "Impossible\n";

```

8.11 树上问题入口

8.11.1 什么题会用到

- 给定一棵树。
- 查询两点路径。
- 查询祖先、最近公共祖先。
- 对树上路径加值或统计经过次数。

8.11.2 树上暴力拿分

如果不会 LCA，且 n, q 小，可以每次 DFS 找路径。

```

1 bool dfsPath(int u, int parent, int target, vector<int>& path) {
2     if (u == target) {
3         path.push_back(u);
4         return true;

```

```

5     }
6     for (int v : g[u]) {
7         if (v == parent) continue;
8         if (dfsPath(v, u, target, path)) {
9             path.push_back(u);
10            return true;
11        }
12    }
13    return false;
14 }

```

8.11.3 LCA 倍增入口

```

1  int LOG; // compute from n before allocating up
2  vector<vector<int>> up;
3  vector<int> depth;
4  vector<vector<int>> g;
5
6  void dfsLca(int u, int p) {
7      up[0][u] = p;
8      for (int k = 1; k < LOG; k++) {
9          up[k][u] = up[k - 1][up[k - 1][u]];
10     }
11     for (int v : g[u]) {
12         if (v == p) continue;
13         depth[v] = depth[u] + 1;
14         dfsLca(v, u);
15     }
16 }
17
18 int lca(int a, int b) {
19     if (depth[a] < depth[b]) swap(a, b);
20     int diff = depth[a] - depth[b];
21     for (int k = 0; k < LOG; k++) {
22         if (diff & (1 << k)) a = up[k][a];
23     }
24     if (a == b) return a;
25     for (int k = LOG - 1; k >= 0; k--) {
26         if (up[k][a] != up[k][b]) {
27             a = up[k][a];
28             b = up[k][b];
29         }
30     }
31     return up[0][a];
32 }

```

先读入 n 并将无向树边存入 g 。下面根为 1，根父亲为自身；深链应改用 [T209¹⁹²](#) 的无递归遍历。初

始化:

```
1 LOG = 1;
2 while ((1LL << LOG) <= n) LOG++;
3 up.assign(LOG, vector<int>(n + 1));
4 depth.assign(n + 1, 0);
5 dfsLca(1, 1);
```

8.11.4 树上差分入口

多次给路径 (u, v) 加 1, 最后统计每个点或边被经过次数。

点差分采用根的父亲为自身的约定, 经过根的路径不能再减一次根; 以下函数与 LCA 共用 `g`、`up`。处理全部路径后调用 `collect(1,1)`。

```
1 vector<long long> diff(n + 1, 0);
2
3 void addPath(int u, int v) {
4     int w = lca(u, v);
5     diff[u]++;
6     diff[v]++;
7     diff[w]--;
8     if (up[0][w] != w) diff[up[0][w]]--; // root is its own parent
9 }
10
11 void collect(int u, int p) {
12     for (int v : g[u]) {
13         if (v == p) continue;
14         collect(v, u);
15         diff[u] += diff[v];
16     }
17 }
```

点差分和边差分的公式不同。拿不准时, 可先逐条路径更新, 用小数据验证; 是否能得部分分要看子任务范围。

8.12 状态压缩 DP 入口

8.12.1 什么题会用到

- n 很小, 例如 $n \leq 20$ 。
- 每个元素只有选或不选。
- 状态可以用二进制集合表示。
- 旅行、分组、覆盖、集合最优值。

8.12.2 集合 DP 基本形状

```
1 int N = 1 << n;
2 vector<int> dp(N, INF);
3 dp[0] = 0;
4
5 for (int mask = 0; mask < N; mask++) {
6     for (int i = 0; i < n; i++) {
7         if ((mask & (1 << i)) == 0) {
8             int nmask = mask | (1 << i);
9             dp[nmask] = min(dp[nmask], dp[mask] + cost[i]);
10        }
11    }
12 }
```

8.12.3 状压易错点

- $1 \ll n$ 当 $n \geq 31$ 会溢出，要用 $1LL \ll n$ 。
- 状态数是 2^n ， $n = 25$ 已经很大。
- 数组大小和内存要提前估算。

8.13 KMP 字符串匹配入口

8.13.1 什么题会用到

- 长文本中多次查找模式串。
- 字符串长度很大，暴力匹配会超时。
- 需要统计模式串出现次数。

8.13.2 KMP 模板

```
1 vector<int> prefixFunction(const string& p) {
2     int n = (int)p.size();
3     vector<int> pi(n, 0);
4     for (int i = 1; i < n; i++) {
5         int j = pi[i - 1];
6         while (j > 0 && p[i] != p[j]) j = pi[j - 1];
7         if (p[i] == p[j]) j++;
8         pi[i] = j;
9     }
10    return pi;
11 }
12
13 int countMatch(const string& text, const string& p) {
14     if (p.empty()) return (int)text.size() + 1; // empty pattern: all gaps
15     vector<int> pi = prefixFunction(p);
```

```
16     int j = 0;
17     int cnt = 0;
18     for (char c : text) {
19         while (j > 0 && c != p[j]) j = pi[j - 1];
20         if (c == p[j]) j++;
21         if (j == (int)p.size()) {
22             cnt++;
23             j = pi[j - 1];
24         }
25     }
26     return cnt;
27 }
```

8.13.3 不会 KMP 时

- 字符串短时用暴力匹配。
- C++ 可用 `s.find(t)` 拿部分数据。
- 多模式串匹配可能要 Trie 或 AC 自动机，但新手优先拿小数据。

8.14 Trie 字典树入口

8.14.1 什么题会用到

- 很多字符串插入和查询。
- 前缀查询。
- 字典、单词集合。

8.14.2 小写字母 Trie

```
1 struct Trie {
2     struct Node {
3         int child[26];
4         bool end = false;
5         Node() {
6             memset(child, -1, sizeof child);
7         }
8     };
9
10    vector<Node> tr;
11
12    Trie() {
13        tr.push_back(Node());
14    }
15
16    void insert(const string& s) {
```

```
17     int u = 0;
18     for (char c : s) {
19         int x = c - 'a';
20         if (tr[u].child[x] == -1) {
21             tr[u].child[x] = (int)tr.size();
22             tr.push_back(Node());
23         }
24         u = tr[u].child[x];
25     }
26     tr[u].end = true;
27 }
28
29 bool find(const string& s) {
30     int u = 0;
31     for (char c : s) {
32         int x = c - 'a';
33         if (tr[u].child[x] == -1) return false;
34         u = tr[u].child[x];
35     }
36     return tr[u].end;
37 }
38 };
```

8.14.3 不会 Trie 时

- 只查完整字符串，用 `set<string>` 或 `unordered_set<string>`。
- 数据小，用逐个字符串比较。
- 前缀查询可以用排序后 `lower_bound` 尝试部分分。

8.15 D/E 考场时间策略

剩余时间	建议动作	目标
60 分钟以上	先思考满分算法，再写部分分保底	进一步优化
30–60 分钟	先写暴力或特殊性质，再看能否优化	稳定拿分
15–30 分钟	只写最确定的小数据做法	不空题
15 分钟以内	完成一个明确子任务的正确实现，清理输出	完成可做的部分

8.16 常见高级算法入口

题面信号	可能算法	先翻哪里
最大化最小值、最小化最大值	二分答案	B 题二分答案
区间修改查询交替	线段树 / 树状数组	T401¹⁹⁵ / T402¹⁹⁵
树上路径、祖先	LCA / 树上差分	T208¹⁹¹ 、本章树上差分
强连通、缩点	Tarjan	本书未收录 Tarjan; 小图可枚举可达性
最小连接代价	最小生成树	Kruskal / Prim
网络最短距离	Dijkstra / Floyd	B 题图论入口
容量和价值	背包 DP	B 题 DP 入口
集合状态很小	状态压缩 DP	状压入口
字符串匹配很多次	KMP / Trie	T502¹⁹⁹ / T503²⁰⁰

8.17 高级算法识别速查

关键词	优先想到	不会时的替代
区间加、区间查询	线段树	差分、树状数组、暴力
单点修改、区间和	树状数组	前缀和静态版、暴力
树上路径查询	LCA	每次 DFS 找路径
多次树上路径加	树上差分	暴力路径加
连接所有点最小代价	Kruskal	枚举小图、Prim 小数据
集合状态很小	状压 DP	DFS 枚举
长串匹配短串	KMP	<code>find</code> / 暴力
前缀查询很多	Trie	set / 排序后二分
最大化最小值	二分答案	枚举答案小范围
任务依赖	拓扑排序	小图 DFS 检查

8.18 D/E 模板练习建议

模板	优先级	原因
二分答案	高	B/D 都常见，代码短，提分明显。
树状数组	高	区间问题常见，比线段树简单。
Dijkstra	高	图论常见，B/D 都可能用。
Kruskal	中	最小生成树题专用，模板短。
线段树	中高	强但代码长，建议练熟基础版。
LCA	中	树上题常见，但新手可先会暴力。
状态压缩 DP	中	看到 $n \leq 20$ 很有用。

模板	优先级	原因
KMP	中低	字符串题有用，但 CSP 中不一定高频。
Trie	中低	前缀题有用，完整字符串可用 set 替代。
树上差分	低到中	需要 LCA 基础，适合进一步优化。

8.19 完整做法暂时不会时

出现以下情况，应转为部分分：

- 已经想了较久，仍没有可实现的做法。
- 会算法名字，但模板完全不会写。
- 写到一半发现状态定义不清。
- 样例都无法解释。
- A/B/C 还没稳。

8.20 D/E 题提交前检查

- 是否至少能过样例。
- 是否能过自己造的小数据。
- 是否处理了最小输入。
- 如果是暴力，是否明确知道能拿哪些数据。
- 是否删除调试输出。
- 是否没有因为追求优化改坏暴力基础逻辑。

第三部分

模板与检查

第九章 高频算法模板库

9.1 模板库使用方法

下面按编号收录常用模板，编号右上角是正文页码。复制前先看下标、初始值和适用条件，再按题目的输入输出修改。有些写法在前面的题型章也出现过，这里放在一起方便查找。

所有求和、乘法和距离仍须验证中间值能放进 `long long`。例如快速幂的普通乘法版要求模数不超过 3037000500；更大的模数需另用宽整数或安全模乘。

- 基础实现和 STL 见第 4 章（第 33 页），不另编 T001 类编号。
- T100--T199：前缀和、差分、排序、哈希、二分。
- T200--T299：搜索和图论。
- T300--T399：动态规划。
- T400--T499：数据结构。
- T500--T599：字符串、数学、日期。
- T600--T699：大模拟与解析。

9.2 模板速查表

编号	名称	什么题会用到	复杂度
T101 ¹⁷⁷	一维前缀和	多次静态区间和查询	预处理 $O(n)$ ，查询 $O(1)$
T102 ¹⁷⁷	二维前缀和	多次子矩阵和查询	预处理 $O(nm)$ ，查询 $O(1)$
T103 ¹⁷⁸	一维差分	多次区间加，最后查询	$O(n + q)$
T104 ¹⁷⁹	二维差分	多次矩形加，最后输出	$O(nm + q)$
T105 ¹⁸⁰	排序自定义比较器	多关键字排序、排名	$O(n \log n)$
T106 ¹⁸⁰	哈希统计	出现次数、查重、频率	平均 $O(n)$
T107 ¹⁸¹	二分答案	最大化最小值、最小化最大值	$O(\log V \cdot \text{check})$
T108 ¹⁸¹	双指针	排序后找对、滑动窗口	$O(n)$ 或 $O(n \log n)$
T109 ¹⁸²	坐标离散化	坐标大但点数少	$O(n \log n)$
T110 ¹⁸³	事件排序	按时间发生、扫描线	$O(n \log n)$
T111 ¹⁸⁴	矩阵重塑/乘法	下标映射、矩阵乘法换序	$O(nm)$ / 按维度

编号	名称	什么题会用到	复杂度
T112 ¹⁸⁴	稀疏向量点积	有序非零项匹配	$O(a + b)$
T113 ¹⁸⁵	有限状态倍增	多次重复映射	查询 $O(\log K)$
T201 ¹⁸⁵	网格 BFS	最短步数、迷宫	$O(nm)$
T202 ¹⁸⁶	DFS 连通块	区域大小、连通块数量	$O(n + m)$ 或 $O(nm)$
T203 ¹⁸⁷	并查集	合并集合、判断同组	均摊 $O(\alpha(n))$
T204 ¹⁸⁸	Dijkstra	非负权最短路	$O((n + m) \log n)$
T205 ¹⁸⁹	拓扑排序	依赖关系、先后顺序	$O(n + m)$
T206 ¹⁹⁰	Floyd	小图任意两点最短路	$O(n^3)$
T207 ¹⁹⁰	Kruskal	最小生成树	$O(m \log m)$
T208 ¹⁹¹	LCA 倍增	树上最近公共祖先	$O(n \log n)$ 预处理, $O(\log n)$ 查询
T209 ¹⁹²	无递归 DFS 序	深树、子树变区间	$O(n)$
T301 ¹⁹³	01 背包	每个物品选或不选	$O(nV)$
T302 ¹⁹³	完全背包	每个物品可选多次	$O(nV)$
T303 ¹⁹⁴	恰好/至少背包	恰好装满、金额门槛	$O(nV) / O(nS)$
T401 ¹⁹⁵	树状数组	单点修改、区间和	$O(\log n)$
T402 ¹⁹⁵	线段树	区间修改、区间查询	$O(\log n)$
T403 ¹⁹⁷	单调栈	最近更大/更小元素	$O(n)$
T404 ¹⁹⁷	单调队列	滑动窗口最大/最小	$O(n)$
T405 ¹⁹⁸	静态区间最大值	任意区间极值	查询 $O(\log n)$
T501 ¹⁹⁸	split 字符串分割	路径、配置、命令解析	$O(s)$
T502 ¹⁹⁹	KMP	长文本匹配模式串	$O(n + m)$
T503 ²⁰⁰	Trie	字符串集合、前缀查询	$O(\sum s)$
T510 ²⁰¹	整除取整和标准取模	分组数、负数取模、数学 floor/ceil	$O(1)$
T511 ²⁰²	快速幂	$a^b \bmod m$	$O(\log b)$
T512 ²⁰²	素数和约数	判断素数、枚举约数	$O(\sqrt{n})$
T513 ²⁰³	进制转换	任意进制与十进制	$O(\text{位数})$
T514 ²⁰⁴	日期处理	闰年、每月天数、日期推进	按实现而定
T515 ²⁰⁴	质因数与素数筛	指数分解、批量素数	见模板说明
T601 ²⁰⁵	大模拟骨架	多对象、多命令、长题面	按题目而定
T602 ²⁰⁶	命令分发	每行一个操作名	按命令复杂度
T603 ²⁰⁷	状态机	对象多状态切换	按操作复杂度
T604 ²⁰⁷	懒删除优先队列	动态删除/更新后仍取最优	$O(\log n)$ 均摊
T605 ²⁰⁹	LRU 缓存	最近最少使用淘汰	$O(1)$ 均摊
T606 ²⁰⁹	路径规范化	/a/.../b、目录路径	$O(s)$
T607 ²¹⁰	URL 参数解析	path?a=1&b=2	$O(s)$
T608 ²¹¹	括号匹配	括号合法性、嵌套结构	$O(n)$
T609 ²¹¹	简单表达式求值	只含加减的表达式	$O(n)$

编号	名称	什么题会用到	复杂度
T610 ²¹²	四则表达式	优先级、括号、一元符号	$O(s)$

9.3 T101 一维前缀和

9.3.1 适用场景

- 数组不修改。
- 多次查询 $[l, r]$ 的和。
- 需要快速得到一段累计值。

9.3.2 模板

```

1 int n, q;
2 cin >> n >> q;
3 vector<long long> pre(n + 1, 0);
4
5 for (int i = 1; i <= n; i++) {
6     long long x;
7     cin >> x;
8     pre[i] = pre[i - 1] + x;
9 }
10
11 while (q--) {
12     int l, r;
13     cin >> l >> r;
14     cout << pre[r] - pre[l - 1] << '\n';
15 }
```

9.3.3 易错点

- 模板默认下标从 1 开始。
- `pre[0]` 是 0。
- 求和用 `long long`。
- 如果有修改，普通前缀和不适用。

9.4 T102 二维前缀和

9.4.1 适用场景

多次查询子矩阵和。

```

1  int n, m, q;
2  cin >> n >> m >> q;
3  vector<vector<long long>> pre(n + 1, vector<long long>(m + 1, 0));
4
5  for (int i = 1; i <= n; i++) {
6      for (int j = 1; j <= m; j++) {
7          long long x;
8          cin >> x;
9          pre[i][j] = pre[i - 1][j] + pre[i][j - 1]
10             - pre[i - 1][j - 1] + x;
11      }
12  }
13
14  while (q--) {
15      int x1, y1, x2, y2;
16      cin >> x1 >> y1 >> x2 >> y2;
17      long long ans = pre[x2][y2] - pre[x1 - 1][y2]
18          - pre[x2][y1 - 1] + pre[x1 - 1][y1 - 1];
19      cout << ans << '\n';
20  }

```

9.4.2 易错点

- 四项公式不要写反。
- 默认坐标从 1 开始。
- 子矩阵端点是否保证 $x1 \leq x2, y1 \leq y2$ 。

9.5 T103 一维差分

本例初始数组全为 0。若有初值 $a[1..n]$ ，先令 $\text{diff}[i]=a[i]-a[i-1]$ ($a[0]=0$)，再处理修改。

9.5.1 适用场景

多次区间加，最后统一输出每个位置的值。

```

1  int n, q;
2  cin >> n >> q;
3  vector<long long> diff(n + 2, 0);
4
5  while (q--) {
6      int l, r;
7      long long v;
8      cin >> l >> r >> v;
9      diff[l] += v;
10     diff[r + 1] -= v;
11 }

```

```
12
13 long long cur = 0;
14 for (int i = 1; i <= n; i++) {
15     cur += diff[i];
16     cout << cur << '\n';
17 }
```

9.5.2 易错点

- 数组开到 $n+2$ 。
- 如果修改后立刻查询区间和，普通差分不够。
- 修改值用 long long。

9.6 T104 二维差分

本例初始矩阵全为 0。若有初值，重建后的增量应加回原矩阵；最终输出 $\text{diff}[i][j]$ 或原值加增量。

9.6.1 适用场景

多次对子矩形加值，最后输出整个矩阵。

```
1 int n, m, q;
2 cin >> n >> m >> q;
3 vector<vector<long long>> diff(n + 2, vector<long long>(m + 2, 0));
4
5 while (q--) {
6     int x1, y1, x2, y2;
7     long long v;
8     cin >> x1 >> y1 >> x2 >> y2 >> v;
9     diff[x1][y1] += v;
10    diff[x2 + 1][y1] -= v;
11    diff[x1][y2 + 1] -= v;
12    diff[x2 + 1][y2 + 1] += v;
13 }
14
15 for (int i = 1; i <= n; i++) {
16     for (int j = 1; j <= m; j++) {
17         diff[i][j] += diff[i - 1][j] + diff[i][j - 1]
18             - diff[i - 1][j - 1];
19     }
20 }
```

9.7 T105 排序自定义比较器

9.7.1 适用场景

多关键字排序、排名、优先级排序。

```
1 struct Node {  
2     int id;  
3     int score;  
4     int time;  
5 };  
6  
7 sort(a.begin(), a.end(), [](const Node& x, const Node& y) {  
8     if (x.score != y.score) return x.score > y.score;  
9     if (x.time != y.time) return x.time < y.time;  
10    return x.id < y.id;  
11 });
```

9.7.2 易错点

- 相等时不能返回 true。
- 每个关键字升序还是降序要写清楚。
- 需要保持原顺序时用 `stable_sort`。

9.8 T106 哈希统计

计数数组版要求每个值在 0 到 100000；负数须偏移或改用 `map`。字符串哈希还要计入总字符长度。

9.8.1 值域小

```
1 vector<int> cnt(100001, 0);  
2 for (int x : a) {  
3     cnt[x]++;  
4 }
```

9.8.2 值域大或字符串

```
1 unordered_map<string, int> cnt;  
2 for (string s : words) {  
3     cnt[s]++;  
4 }
```

9.8.3 易错点

- 需要有序输出时用 `map`。

- key 很大不能开计数数组。
- `cnt[x]` 会自动创建 key。

9.9 T107 二分答案

9.9.1 适用场景

- 求最小可行值。
- 求最大可行值。
- 给定答案 *mid* 能判断是否可行。
- 可行性具有单调性。

9.9.2 最小可行值

```
1 bool check(long long mid) {  
2     // return true if mid is feasible  
3 }  
4  
5 long long l = 0, r = 1000000000000000000LL;  
6 while (l < r) {  
7     long long mid = l + (r - l) / 2;  
8     if (check(mid)) r = mid;  
9     else l = mid + 1;  
10 }  
11 cout << l << '\n';
```

9.9.3 最大可行值

```
1 long long l = 0, r = 1000000000000000000LL;  
2 while (l < r) {  
3     long long mid = l + (r - l + 1) / 2;  
4     if (check(mid)) l = mid;  
5     else r = mid - 1;  
6 }  
7 cout << l << '\n';
```

二分答案必须有单调性。最小可行值模板要求初始右端点可行，最大可行值模板要求初始左端点可行；若不能保证，应先判无解。范围按题目推导，不能把 10^{18} 当成通用答案上界。`check` 必须自行实现；上下界差和中点运算也不能溢出。

9.10 T108 双指针

窗口版求非负数组中“和不超过 *limit*”的最长连续段，要求 *limit* 不小于 0；两数和排序后原下标会丢失。

9.10.1 排序后找两数和

```
1 sort(a.begin(), a.end());
2 int l = 0, r = n - 1;
3 bool ok = false;
4
5 while (l < r) {
6     long long sum = 1LL * a[l] + a[r];
7     if (sum == target) {
8         ok = true;
9         break;
10    } else if (sum < target) {
11        l++;
12    } else {
13        r--;
14    }
15 }
```

9.10.2 非负数组滑动窗口

```
1 int l = 0;
2 long long sum = 0;
3 int best = 0;
4
5 for (int r = 0; r < n; r++) {
6     sum += a[r];
7     while (sum > limit && l <= r) {
8         sum -= a[l];
9         l++;
10    }
11    best = max(best, r - l + 1);
12 }
```

滑动窗口通常要求元素非负或条件具有单调性。

9.11 T109 坐标离散化

9.11.1 适用场景

坐标值很大，但出现的坐标数量不多。

```
1 vector<int> xs;
2 for (int x : original) {
3     xs.push_back(x);
4 }
5
6 sort(xs.begin(), xs.end());
7 xs.erase(unique(xs.begin(), xs.end()), xs.end());
```

```

8
9 auto getId = [&](int x) {
10     return (int)(lower_bound(xs.begin(), xs.end(), x) - xs.begin()) + 1;
11 };

```

如果题目涉及区间长度，不能只看离散编号，还要保留真实坐标差。

9.12 T110 事件排序

9.12.1 适用场景

按时间顺序处理事件，或扫描区间开始/结束。

```

1 struct Event {
2     int time;
3     int type;
4     int id;
5 };
6
7 sort(events.begin(), events.end(), [](const Event& a, const Event& b) {
8     if (a.time != b.time) return a.time < b.time;
9     return a.type < b.type;
10 });
11
12 for (const Event& e : events) {
13     if (e.type == 0) {
14         // start
15     } else {
16         // end
17     }
18 }

```

9.12.2 区间同时进行数量

```

1 vector<pair<int, int>> events;
2 for (auto [l, r] : intervals) {
3     if (l >= r) continue; // empty half-open interval
4     events.push_back({l, 1});
5     events.push_back({r, -1});
6 }
7
8 sort(events.begin(), events.end(), [](auto a, auto b) {
9     if (a.first != b.first) return a.first < b.first;
10    return a.second < b.second; // end before start for [l,r)
11 });
12
13 int cur = 0, best = 0;
14 for (auto [t, delta] : events) {

```

```

15     cur += delta;
16     best = max(best, cur);
17 }

```

同一时间开始和结束谁先处理，要按题意决定。

9.13 T111 矩阵重塑与乘法顺序

重塑：按行展开，旧位置 (i, j) 的线性编号 $k = i \cdot m + j$ ，重塑为 $r \times c$ 后位于 $(k/c, k \bmod c)$ 。要求 $nm = rc$ ，下标从 0 开始。

```

1 vector<vector<long long>> b(r, vector<long long>(c));
2 for (int i = 0; i < n; i++)
3     for (int j = 0; j < m; j++) {
4         long long k = 1LL * i * m + j;
5         b[k / c][k % c] = a[i][j];
6     }

```

乘法： A 为 $n \times d$ ， B 为 $d \times m$ ， $C_{ij} = \sum_k A_{ik} B_{kj}$ ，复杂度 $O(ndm)$ 。

```

1 vector<vector<long long>> C(n, vector<long long>(m, 0));
2 for (int i = 0; i < n; i++)
3     for (int k = 0; k < d; k++)
4         for (int j = 0; j < m; j++)
5             C[i][j] += A[i][k] * B[k][j];

```

若 Q, K, V 都是 $n \times d$ 且 $d \ll n$ ，计算 $(QK^T)V$ 可改成 $Q(K^TV)$ ，由 $O(n^2d)$ 降至 $O(nd^2)$ ，避免构造 $n \times n$ 中间矩阵。只能在数学运算允许结合时换序；非线性函数、截断取整不能任意移走。浮点换序可能改变舍入误差。逐行权重乘法必须按原公式位置处理。

9.14 T112 稀疏向量点积

两个向量仅存非零项，按下标严格升序且各自下标不重复。相同下标的值相乘后相加，复杂度 $O(a + b)$ 。

```

1 long long sparseDot(const vector<pair<int, long long>>& a,
2                     const vector<pair<int, long long>>& b) {
3     size_t i = 0, j = 0;
4     long long ans = 0;
5     while (i < a.size() && j < b.size()) {
6         if (a[i].first < b[j].first) i++;
7         else if (a[i].first > b[j].first) j++;
8         else {
9             ans += a[i].second * b[j].second;
10            i++; j++;
11        }
12    }
13    return ans;
14 }

```

若输入无序先排序；若同一下标可重复先合并。乘积与总和均须在 long long 范围内。

9.15 T113 有限状态映射与倍增

若每个状态有唯一后继 $\text{next}[v]$ ，重复 k 次不必逐步模拟。令 $\text{up}[b][v]$ 表示执行 2^b 次后的状态，由两个 2^{b-1} 次复合得到。适合字符替换、指针跳转；要求所有后继在 0 到 $V-1$ ，未指定的映射按题意初始化。

```

1 struct JumpTable {
2     vector<vector<int>> up;
3     explicit JumpTable(const vector<int>& next) {
4         int V = (int)next.size();
5         up.assign(64, vector<int>(V));
6         up[0] = next;
7         for (int b = 1; b < 64; b++)
8             for (int v = 0; v < V; v++)
9                 up[b][v] = up[b-1][up[b-1][v]];
10    }
11    int jump(int v, unsigned long long k) const {
12        for (int b = 0; b < 64; b++)
13            if ((k >> b) & 1ULL) v = up[b][v];
14        return v;
15    }
16 };

```

预处理时间、空间 $O(V \log K)$ ，单次跳转 $O(\log K)$ ，此处为无符号 64 位 k 留足 64 层。若映射在有限集合上是单射，则每个连通分量是环，也可按各自环长取模；不能随意用固定数 26 或 63 代替环长。每个查询应从题目指定的初态出发。

值域很小且求逆映射时，还可枚举全部输入 x ，算出 $y=F(x)$ ，记录 $\text{inverse}[y]=x$ ；仅在 F 确实单射时才有唯一逆。

9.16 T201 网格 BFS

默认输入合法的 0 下标坐标，起点终点均可通行，井号为障碍；不可达输出 -1。若题目允许阻塞起点，应先判定。

9.16.1 适用场景

迷宫、最短步数、每一步代价相同。

```

1 int n, m;
2 cin >> n >> m;
3 vector<string> g(n);
4 for (int i = 0; i < n; i++) cin >> g[i];
5
6 int sx, sy, tx, ty;
7 cin >> sx >> sy >> tx >> ty;
8

```

```
9 vector<vector<int>> dist(n, vector<int>(m, -1));
10 queue<pair<int, int>> q;
11
12 dist[sx][sy] = 0;
13 q.push({sx, sy});
14
15 int dx[4] = {-1, 0, 1, 0};
16 int dy[4] = {0, 1, 0, -1};
17
18 while (!q.empty()) {
19     auto [x, y] = q.front();
20     q.pop();
21
22     for (int d = 0; d < 4; d++) {
23         int nx = x + dx[d];
24         int ny = y + dy[d];
25         if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
26         if (g[nx][ny] == '#') continue;
27         if (dist[nx][ny] != -1) continue;
28         dist[nx][ny] = dist[x][y] + 1;
29         q.push({nx, ny});
30     }
31 }
32
33 cout << dist[tx][ty] << '\n';
```

9.16.2 易错点

- 坐标是 0 下标还是 1 下标。
- 入队时标记访问。
- 边权不全相等时不要用普通 BFS。

9.17 T202 DFS 连通块

先读入 n 、 m 、 g ，并执行 `vis.assign(n, vector<int>(m, 0))`；只从未访问且可通行的格子调用。

9.17.1 网格 DFS

```
1 int n, m;
2 vector<string> g;
3 vector<vector<int>> vis;
4 int dx[4] = {-1, 0, 1, 0};
5 int dy[4] = {0, 1, 0, -1};
6
7 int dfs(int x, int y) {
8     vis[x][y] = 1;
```

```
9   int area = 1;
10  for (int d = 0; d < 4; d++) {
11      int nx = x + dx[d];
12      int ny = y + dy[d];
13      if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
14      if (vis[nx][ny]) continue;
15      if (g[nx][ny] == '#') continue;
16      area += dfs(nx, ny);
17  }
18  return area;
19 }
```

9.17.2 易错点

- 大网格递归可能爆栈，可改 BFS。
- 障碍字符要按题目改。

9.18 T203 并查集

```
1  struct DSU {
2      vector<int> parent, sz;
3
4      DSU(int n) {
5          parent.resize(n + 1);
6          sz.assign(n + 1, 1);
7          for (int i = 1; i <= n; i++) parent[i] = i;
8      }
9
10     int find(int x) {
11         if (parent[x] == x) return x;
12         return parent[x] = find(parent[x]);
13     }
14
15     bool unite(int a, int b) {
16         int ra = find(a);
17         int rb = find(b);
18         if (ra == rb) return false;
19         if (sz[ra] < sz[rb]) swap(ra, rb);
20         parent[rb] = ra;
21         sz[ra] += sz[rb];
22         return true;
23     }
24
25     bool same(int a, int b) {
26         return find(a) == find(b);
27     }
28 }
```

```
28 };
```

9.18.1 适用场景

合并集合、判断是否同组、连通块。

9.19 T204 Dijkstra

INF 必须大于所有有效最短路；本例仅处理非负权，无法到达的点 dist 仍为 INF，输出时按题意转换。

```
1 struct Edge {
2     int to;
3     long long w;
4 };
5
6 const long long INF = (long long)4e18;
7
8 int n, m, s;
9 cin >> n >> m >> s;
10 vector<vector<Edge>> g(n + 1);
11
12 for (int i = 0; i < m; i++) {
13     int u, v;
14     long long w;
15     cin >> u >> v >> w;
16     g[u].push_back({v, w});
17     g[v].push_back({u, w}); // remove if directed
18 }
19
20 vector<long long> dist(n + 1, INF);
21 priority_queue<pair<long long, int>,
22               vector<pair<long long, int>>,
23               greater<pair<long long, int>>> pq;
24
25 dist[s] = 0;
26 pq.push({0, s});
27
28 while (!pq.empty()) {
29     auto [d, u] = pq.top();
30     pq.pop();
31     if (d != dist[u]) continue;
32
33     for (auto e : g[u]) {
34         if (e.w >= INF - dist[u]) continue;
35         if (dist[e.to] > dist[u] + e.w) {
36             dist[e.to] = dist[u] + e.w;
```

```

37         pq.push({dist[e.to], e.to});
38     }
39 }
40 }

```

9.19.1 易错点

- 不能处理负边权。
- 无向图双向加边，有向图单向加边。
- 距离用 long long。

9.20 T205 拓扑排序

9.20.1 适用场景

任务依赖、课程先修、判断有向图是否有环。

```

1  int n, m;
2  cin >> n >> m;
3  vector<vector<int>> g(n + 1);
4  vector<int> indeg(n + 1, 0);
5
6  for (int i = 0; i < m; i++) {
7      int u, v;
8      cin >> u >> v; // u before v
9      g[u].push_back(v);
10     indeg[v]++;
11 }
12
13 queue<int> q;
14 for (int i = 1; i <= n; i++) {
15     if (indeg[i] == 0) q.push(i);
16 }
17
18 vector<int> order;
19 while (!q.empty()) {
20     int u = q.front();
21     q.pop();
22     order.push_back(u);
23     for (int v : g[u]) {
24         indeg[v]--;
25         if (indeg[v] == 0) q.push(v);
26     }
27 }
28
29 if ((int)order.size() == n) {
30     // has topological order

```

```

31 } else {
32     // has cycle
33 }

```

9.21 T206 Floyd

要求不存在负环，且所有有限距离及相加的中间值都在 long long 范围内。INF 必须大于有效距离。无向负权边会形成负环。

9.21.1 适用场景

点数小，任意两点最短路。

```

1  const long long INF = (long long)4e18;
2
3  int n, m;
4  cin >> n >> m;
5  vector<vector<long long>> dist(n + 1, vector<long long>(n + 1, INF));
6
7  for (int i = 1; i <= n; i++) dist[i][i] = 0;
8
9  for (int i = 0; i < m; i++) {
10     int u, v;
11     long long w;
12     cin >> u >> v >> w;
13     dist[u][v] = min(dist[u][v], w);
14     dist[v][u] = min(dist[v][u], w); // remove if directed
15 }
16
17 for (int k = 1; k <= n; k++) {
18     for (int i = 1; i <= n; i++) {
19         for (int j = 1; j <= n; j++) {
20             if (dist[i][k] == INF || dist[k][j] == INF) continue;
21             dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
22         }
23     }
24 }

```

三层循环顺序必须是 k, i, j 。

9.22 T207 Kruskal

先复制 T203¹⁸⁷ 的 DSU 定义，并读入 n, m 。断开时的 Impossible 仅为示例输出，需按题目替换。

9.22.1 适用场景

无向图，连接所有点的最小总代价。

```

1 struct Edge {
2     int u, v;
3     long long w;
4 };
5
6 vector<Edge> edges;
7 for (int i = 0; i < m; i++) {
8     int u, v;
9     long long w;
10    cin >> u >> v >> w;
11    edges.push_back({u, v, w});
12 }
13 sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
14     return a.w < b.w;
15 });
16
17 DSU dsu(n);
18 long long ans = 0;
19 int cnt = 0;
20
21 for (auto e : edges) {
22     if (dsu.unite(e.u, e.v)) {
23         ans += e.w;
24         cnt++;
25     }
26 }
27
28 if (cnt == n - 1) cout << ans << '\n';
29 else cout << "Impossible\n";

```

9.23 T208 LCA 倍增

先建好 n 个点的无向树，根为 1，根的父亲为自身。深链可能导致递归爆栈；需要无递归遍历时见 T209¹⁹²。

9.23.1 适用场景

树上最近公共祖先、树上路径查询。

```

1 int LOG; // compute from n before allocating up
2 vector<vector<int>>> up;
3 vector<int> depth;
4 vector<vector<int>>> g;
5
6 void dfsLca(int u, int p) {
7     up[0][u] = p;
8     for (int k = 1; k < LOG; k++) {

```

```

9      up[k][u] = up[k - 1][up[k - 1][u]];
10  }
11  for (int v : g[u]) {
12      if (v == p) continue;
13      depth[v] = depth[u] + 1;
14      dfsLca(v, u);
15  }
16 }
17
18 int lca(int a, int b) {
19     if (depth[a] < depth[b]) swap(a, b);
20     int diff = depth[a] - depth[b];
21     for (int k = 0; k < LOG; k++) {
22         if (diff & (1 << k)) a = up[k][a];
23     }
24     if (a == b) return a;
25     for (int k = LOG - 1; k >= 0; k--) {
26         if (up[k][a] != up[k][b]) {
27             a = up[k][a];
28             b = up[k][b];
29         }
30     }
31     return up[0][a];
32 }

```

初始化:

```

1 LOG = 1;
2 while ((1LL << LOG) <= n) LOG++;
3 up.assign(LOG, vector<int>(n + 1));
4 depth.assign(n + 1, 0);
5 dfsLca(1, 1);

```

9.24 T209 无递归树遍历与子树区间

对无向树生成 DFS 先序、父亲、深度和子树大小，时间与空间 $O(n)$ 。根的父亲为自身，可避开深链递归爆栈。要求 $n \geq 1$ 、输入连通且无环。

```

1 vector<int> parent(n + 1, 0), depth(n + 1, 0);
2 vector<int> order, tin(n + 1), subtree(n + 1, 1);
3 stack<int> st;
4 parent[1] = 1;
5 st.push(1);
6 while (!st.empty()) {
7     int u = st.top(); st.pop();
8     tin[u] = (int)order.size() + 1;
9     order.push_back(u);
10    for (int v : g[u]) {

```

```

11     if (v == parent[u]) continue;
12     parent[v] = u;
13     depth[v] = depth[u] + 1;
14     st.push(v);
15 }
16 }
17 for (int i = n - 1; i > 0; i--) {
18     int u = order[i];
19     subtree[parent[u]] += subtree[u];
20 }
21 // subtree(u) is [tin[u], tin[u] + subtree[u] - 1].

```

用此先序填 T208¹⁹¹ 的倍增表即可替代 dfsLca；父亲总是先于孩子。子树求和可按 tin 重排权值后接 T101¹⁷⁷ 或 T401¹⁹⁵。树上点差分也可按 order 的逆序向父亲累加。

9.25 T301 01 背包

此版求总重量不超过 V 的最大价值，可一件不选。要求重量非负；价值和能放入 long long。恰好装满或达到金额门槛请看 T303¹⁹⁴。

```

1  int n, V;
2  cin >> n >> V;
3  vector<int> w(n + 1);
4  vector<long long> val(n + 1);
5  for (int i = 1; i <= n; i++) {
6      cin >> w[i] >> val[i];
7  }
8
9  vector<long long> dp(V + 1, 0);
10 for (int i = 1; i <= n; i++) {
11     for (int j = V; j >= w[i]; j--) {
12         dp[j] = max(dp[j], dp[j - w[i]] + val[i]);
13     }
14 }
15 cout << dp[V] << '\n';

```

01 背包容量倒序。

9.26 T302 完全背包

此版求不超过 V 的最大价值，要求每件重量严格大于 0。零重量正价值可无限选时没有有限最优值。

```

1  int n, V;
2  cin >> n >> V;
3  vector<int> w(n + 1);
4  vector<long long> val(n + 1);
5  for (int i = 1; i <= n; i++) {
6      cin >> w[i] >> val[i];

```

```

7 }
8
9 vector<long long> dp(V + 1, 0);
10 for (int i = 1; i <= n; i++) {
11     for (int j = w[i]; j <= V; j++) {
12         dp[j] = max(dp[j], dp[j - w[i]] + val[i]);
13     }
14 }
15 cout << dp[V] << '\n';

```

完全背包容量正序。

9.27 T303 恰好装满与至少达到

恰好装满求最大价值：除 $dp[0]=0$ 外，其余状态必须设为不可达。直接使用全 0 的 T301¹⁹³ 会把未装满状态混入答案。

```

1 const long long NEG = LLONG_MIN;
2 vector<long long> dp(V + 1, NEG);
3 dp[0] = 0;
4 for (int i = 0; i < n; i++) {
5     for (int j = V; j >= w[i]; j--) {
6         if (dp[j - w[i]] != NEG)
7             dp[j] = max(dp[j], dp[j - w[i]] + val[i]);
8     }
9 }
10 if (dp[V] == NEG) { /* no solution: output as required */ }
11 else cout << dp[V] << '\n';

```

要求重量非负，价值和不超过且不等于哨兵。完全背包且重量正数时改为容量正序。

至少达到金额 x ，求最小花费（何以包邮型）：价格为非负整数，每件至多一次，价格总和 S 可开数组。用可达性 DP 枚举金额，从 x 往上找第一个可达值，复杂度 $O(nS)$ ，空间 $O(S)$ 。

```

1 int minAtLeast(const vector<int>& price, int x) {
2     int S = accumulate(price.begin(), price.end(), 0);
3     if (x > S) return -1;
4     vector<bool> can(S + 1, false);
5     can[0] = true;
6     for (int v : price)
7         for (int j = S; j >= v; j--)
8             if (can[j - v]) can[j] = true;
9     for (int j = x; j <= S; j++)
10         if (can[j]) return j;
11     return -1;
12 }

```

要求 x 非负、总和可放 int 且内存时间允许；若总和很大而 n 很小，改枚举子集或折半搜索。只把 T301¹⁹³ 的容量设成 x ，求出的是不超过门槛的最大值，方向相反。

9.28 T401 树状数组

初始全 0，下标从 1 开始。先对每个初值调用 `add(i,a[i])`；赋值 `a[i]=v` 要加 `v-a[i]` 并更新 `a[i]`，不是直接 `add(i,v)`。

```

1 struct BIT {
2     int n;
3     vector<long long> tree;
4
5     BIT(int n) : n(n), tree(n + 1, 0) {}
6
7     void add(int idx, long long val) {
8         assert(1 <= idx && idx <= n); // idx=0 would never advance
9         for (; idx <= n; idx += idx & -idx) {
10             tree[idx] += val;
11         }
12     }
13
14     long long sumPrefix(int idx) {
15         assert(0 <= idx && idx <= n);
16         long long res = 0;
17         for (; idx > 0; idx -= idx & -idx) {
18             res += tree[idx];
19         }
20         return res;
21     }
22
23     long long rangeSum(int l, int r) {
24         return sumPrefix(r) - sumPrefix(l - 1);
25     }
26 };

```

9.28.1 适用场景

单点修改、区间和查询。

9.29 T402 线段树

9.29.1 适用场景

用于区间加和区间和查询。代码较长，练习时先跟一遍标记下传和合并过程。

```

1 struct SegTree {
2     int n;
3     vector<long long> tree, lazy;
4
5     SegTree(int n) : n(n), tree(4 * n + 4), lazy(4 * n + 4) {}
6
7     void apply(int node, int l, int r, long long val) {

```

```

8     tree[node] += val * (r - l + 1);
9     lazy[node] += val;
10 }
11
12 void push(int node, int l, int r) {
13     if (lazy[node] == 0 || l == r) return;
14     int mid = (l + r) / 2;
15     apply(node * 2, l, mid, lazy[node]);
16     apply(node * 2 + 1, mid + 1, r, lazy[node]);
17     lazy[node] = 0;
18 }
19
20 void rangeAdd(int node, int l, int r, int ql, int qr, long long val) {
21     if (ql <= l && r <= qr) {
22         apply(node, l, r, val);
23         return;
24     }
25     push(node, l, r);
26     int mid = (l + r) / 2;
27     if (ql <= mid) rangeAdd(node * 2, l, mid, ql, qr, val);
28     if (qr > mid) rangeAdd(node * 2 + 1, mid + 1, r, ql, qr, val);
29     tree[node] = tree[node * 2] + tree[node * 2 + 1];
30 }
31
32 long long query(int node, int l, int r, int ql, int qr) {
33     if (ql <= l && r <= qr) return tree[node];
34     push(node, l, r);
35     int mid = (l + r) / 2;
36     long long res = 0;
37     if (ql <= mid) res += query(node * 2, l, mid, ql, qr);
38     if (qr > mid) res += query(node * 2 + 1, mid + 1, r, ql, qr);
39     return res;
40 }
41 };

```

调用：

```

1 // Require n >= 1; initial array is all zero.
2 SegTree seg(n);
3 // Nonzero a[1..n]: call rangeAdd(1,1,n,i,i,a[i]) for each i.
4 seg.rangeAdd(1, 1, n, l, r, v);
5 long long ans = seg.query(1, 1, n, l, r);

```

9.29.2 易错点

- 默认区间是 1 到 n 。
- 懒标记下传不要漏。

- 求和用 long long。

9.30 T403 单调栈

9.30.1 适用场景

找每个元素右边第一个更大元素、左边第一个更小元素等。

```
1 vector<int> ans(n, -1);
2 stack<int> st; // indices
3
4 for (int i = 0; i < n; i++) {
5     while (!st.empty() && a[i] > a[st.top()]) {
6         ans[st.top()] = i;
7         st.pop();
8     }
9     st.push(i);
10 }
```

9.30.2 易错点

- 比较符号决定严格更大还是大于等于。
- 栈里通常存下标，不是值。
- 全相等数据要测试。

9.31 T404 单调队列

要求 $1 \leq k \leq n$ ；窗口为空或超过数组长度时，输出规则由题意决定。

9.31.1 适用场景

固定长度滑动窗口最大值或最小值。

```
1 deque<int> dq; // indices, values decreasing
2 for (int i = 0; i < n; i++) {
3     while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
4     while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
5     dq.push_back(i);
6
7     if (i >= k - 1) {
8         cout << a[dq.front()] << '\n';
9     }
10 }
```

9.31.2 易错点

- 先删除过期下标。
- 最大值队列保持递减，最小值队列保持递增。
- 输出窗口从 $i \geq k - 1$ 开始。

9.32 T405 静态区间最大值

任意区间最大值不能由两个前缀最大值相减。此处用迭代线段树，构建 $O(n)$ ，查询 $O(\log n)$ ，空间 $O(n)$ ；下标从 0 开始，查询为闭区间，要求数组非空。

```

1 struct RangeMax {
2     int n;
3     vector<long long> t;
4     RangeMax(const vector<long long>& a) : n((int)a.size()), t(2*n) {
5         assert(n > 0);
6         copy(a.begin(), a.end(), t.begin() + n);
7         for (int i = n - 1; i > 0; i--)
8             t[i] = max(t[i*2], t[i*2+1]);
9     }
10    long long query(int l, int r) const {
11        assert(0 <= l && l <= r && r < n);
12        long long ans = LLONG_MIN;
13        for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
14            if (l & 1) ans = max(ans, t[l++]);
15            if (r & 1) ans = max(ans, t[--r]);
16        }
17        return ans;
18    }
19 };

```

`RangeMax rm(a); rm.query(l,r);`。固定长度窗口用 [T404¹⁹⁷](#) 更简单；最近更大元素用 [T403¹⁹⁷](#)，与区间最值是不同问题。

9.33 T501 split 字符串分割

9.33.1 按指定字符分割

```

1 vector<string> split(const string& s, char sep) {
2     vector<string> res;
3     string cur;
4     for (char c : s) {
5         if (c == sep) {
6             res.push_back(cur);
7             cur.clear();
8         } else {
9             cur.push_back(c);

```

```

10     }
11 }
12 res.push_back(cur);
13 return res;
14 }

```

9.33.2 按空格分割

```

1 stringstream ss(line);
2 string word;
3 while (ss >> word) {
4     // use word
5 }

```

9.34 T502 KMP

统计允许重叠的匹配；空模式串在此约定匹配 `text.size()+1` 个间隙。题目只允许非空模式时不影响正常结果。

```

1 vector<int> prefixFunction(const string& p) {
2     int n = (int)p.size();
3     vector<int> pi(n, 0);
4     for (int i = 1; i < n; i++) {
5         int j = pi[i - 1];
6         while (j > 0 && p[i] != p[j]) j = pi[j - 1];
7         if (p[i] == p[j]) j++;
8         pi[i] = j;
9     }
10    return pi;
11 }
12
13 int countMatch(const string& text, const string& p) {
14     if (p.empty()) return (int)text.size() + 1; // empty pattern: all gaps
15     vector<int> pi = prefixFunction(p);
16     int j = 0, cnt = 0;
17     for (char c : text) {
18         while (j > 0 && c != p[j]) j = pi[j - 1];
19         if (c == p[j]) j++;
20         if (j == (int)p.size()) {
21             cnt++;
22             j = pi[j - 1];
23         }
24     }
25     return cnt;
26 }

```

9.35 T503 Trie

find 查询完整单词。前缀查询需沿 child 走完后返回 true，不能要求末节点 end=true；本版不支持重复次数统计。

```
1 struct Trie {
2     struct Node {
3         int child[26];
4         bool end = false;
5         Node() {
6             memset(child, -1, sizeof child);
7         }
8     };
9
10    vector<Node> tr;
11
12    Trie() {
13        tr.push_back(Node());
14    }
15
16    void insert(const string& s) {
17        int u = 0;
18        for (char c : s) {
19            int x = c - 'a';
20            if (tr[u].child[x] == -1) {
21                tr[u].child[x] = (int)tr.size();
22                tr.push_back(Node());
23            }
24            u = tr[u].child[x];
25        }
26        tr[u].end = true;
27    }
28
29    bool find(const string& s) {
30        int u = 0;
31        for (char c : s) {
32            int x = c - 'a';
33            if (tr[u].child[x] == -1) return false;
34            u = tr[u].child[x];
35        }
36        return tr[u].end;
37    }
38};
```

这个 Trie 模板只适合小写英文字母。

9.36 T510 整除取整和标准取模

9.36.1 适用场景

- 分组、分页、分块，要求正整数向上取整。
- 坐标、时间差、偏移量可能为负，要求数学意义的向上或向下取整。
- 同余、周期、哈希下标中需要把负数余数转成 0 到 $m - 1$ 。

9.36.2 正整数向上取整

若 $a \geq 0, b > 0$:

```
1 long long ceil_div_positive(long long a, long long b) {  
2     return a / b + (a % b != 0);  
3 }
```

示例:

```
1 long long pages = ceil_div_positive(n, pageSize);  
2 long long blocks = ceil_div_positive(n, blockSize);
```

9.36.3 有符号通用版本

```
1 // Require b != 0; exclude LLONG_MIN / -1.  
2 long long floor_div(long long a, long long b) {  
3     long long q = a / b;  
4     long long r = a % b;  
5     if (r != 0 && ((r > 0) != (b > 0))) q--;  
6     return q;  
7 }  
8  
9 long long ceil_div(long long a, long long b) {  
10    long long q = a / b;  
11    long long r = a % b;  
12    if (r != 0 && ((r > 0) == (b > 0))) q++;  
13    return q;  
14 }
```

9.36.4 标准非负余数

```
1 long long norm_mod(long long x, long long m) {  
2     long long r = x % m;  
3     if (r < 0) r += m;  
4     return r;  
5 }
```

9.36.5 易错点

- $a/b+(a\%b!=0)$ 在此只用于 $a \geq 0, b > 0$, 不是通用向上取整。
- C++ 整数除法向 0 截断, $-3 / 2$ 是 -1 。
- C++ 负数取模可能得到负数, $-1 \% 5$ 是 -1 。
- 除数 b 或模数 m 不能为 0。
- 若 $a + b - 1$ 可能溢出, 需要改写或先确认范围。

9.37 T511 快速幂

9.37.1 适用场景

计算 $a^b \bmod mod$ 。

```

1 // Require b >= 0 and 1 <= mod <= 3037000500LL.
2 long long modPow(long long a, long long b, long long mod) {
3     long long res = 1 % mod;
4     a %= mod;
5     if (a < 0) a += mod;
6     while (b > 0) {
7         if (b & 1) res = res * a % mod;
8         a = a * a % mod;
9         b >>= 1;
10    }
11    return res;
12 }
```

9.38 T512 素数和约数

9.38.1 gcd 和 lcm

```

1 // Require a,b >= 0 and lcm(a,b) <= LLONG_MAX.
2 long long g = gcd(a, b);
3 long long l = (a == 0 || b == 0) ? 0 : a / g * b;
```

9.38.2 判断素数

```

1 bool isPrime(long long x) {
2     if (x < 2) return false;
3     for (long long d = 2; d <= x / d; d++) {
4         if (x % d == 0) return false;
5     }
6     return true;
7 }
```

9.38.3 枚举约数

```
1 vector<long long> divisors;
2 for (long long d = 1; d <= x / d; d++) {
3     if (x % d == 0) {
4         divisors.push_back(d);
5         if (d != x / d) divisors.push_back(x / d);
6     }
7 }
8 sort(divisors.begin(), divisors.end());
```

9.39 T513 进制转换

9.39.1 任意进制转十进制

```
1 // Valid nonempty digits; 2 <= base <= 16; result <= LLONG_MAX.
2 long long toDecimal(const string& s, int base) {
3     long long ans = 0;
4     for (char c : s) {
5         int v;
6         if ('0' <= c && c <= '9') v = c - '0';
7         else if ('A' <= c && c <= 'F') v = c - 'A' + 10;
8         else v = c - 'a' + 10;
9         ans = ans * base + v;
10    }
11    return ans;
12 }
```

9.39.2 十进制转任意进制

```
1 // Require x >= 0 and 2 <= base <= 16.
2 string fromDecimal(long long x, int base) {
3     if (x == 0) return "0";
4     string digits = "0123456789ABCDEF";
5     string res;
6     while (x > 0) {
7         res.push_back(digits[x % base]);
8         x /= base;
9     }
10    reverse(res.begin(), res.end());
11    return res;
12 }
```

数字很长时不能转成 long long，要用字符串模拟。

9.40 T514 日期处理

9.40.1 闰年

```
1 bool leap(int y) {  
2     return (y % 400 == 0) || (y % 4 == 0 && y % 100 != 0);  
3 }
```

9.40.2 每月天数

```
1 int mdays(int y, int m) {  
2     int d[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};  
3     if (m == 2 && leap(y)) return 29;  
4     return d[m];  
5 }
```

9.40.3 下一天

```
1 void nextDay(int& y, int& m, int& d) {  
2     d++;  
3     if (d > mdays(y, m)) {  
4         d = 1;  
5         m++;  
6         if (m > 12) {  
7             m = 1;  
8             y++;  
9         }  
10    }  
11 }
```

9.40.4 时间转秒

```
1 int toSeconds(int h, int m, int s) {  
2     return h * 3600 + m * 60 + s;  
3 }
```

9.41 T515 质因数分解与筛素数

质因数分解：要求 $x \geq 1$ ；记录每个素因子的指数，复杂度最坏 $O(\sqrt{x})$ 。

```
1 vector<pair<long long,int>> factorize(long long x) {  
2     vector<pair<long long,int>> f;  
3     for (long long p = 2; p <= x / p; p++) {  
4         if (x % p != 0) continue;  
5         int cnt = 0;  
6         do { x /= p; cnt++; } while (x % p == 0);
```

```

7         f.push_back({p, cnt});
8     }
9     if (x > 1) f.push_back({x, 1});
10    return f;
11 }

```

例如 $72 = 2^3 \cdot 3^2$ 。因子化简类题应按题意决定保留整个 p^e ，还是只保留指数中的某部分，不能仅保留素因子 p 。x=1 时列表为空。

筛出 0 到 N 的素数：适合大量小值域查询。时间 $O(N \log \log N)$ ，空间 $O(N)$ 。

```

1 vector<char> sieve(int N) {
2     vector<char> prime(N + 1, true);
3     prime[0] = false;
4     if (N >= 1) prime[1] = false;
5     for (int p = 2; p <= N / p; p++) {
6         if (!prime[p]) continue;
7         for (long long j = 1LL*p*p; j <= N; j += p)
8             prime[j] = false;
9     }
10    return prime;
11 }

```

要求 N 非负且数组可开；大素数接近 10^{18} 时试除会极慢，整数类型能存下这个数，也不代表试除次数能在时限内跑完。

9.42 T601 大模拟骨架

9.42.1 适用场景

C 题长题面、多对象、多命令、多状态。

```

1 struct Item {
2     int id;
3     string name;
4     int state;
5 };
6
7 int n, q;
8 vector<Item> items;
9
10 void readInput() {
11     cin >> n >> q;
12     items.resize(n);
13     for (int i = 0; i < n; i++) {
14         cin >> items[i].id >> items[i].name;
15         items[i].state = 0;
16     }
17 }
18

```

```
19 void solve() {
20     while (q--) {
21         string op;
22         cin >> op;
23         // dispatch command
24     }
25 }
26
27 int main() {
28     ios::sync_with_stdio(false);
29     cin.tie(nullptr);
30
31     readInput();
32     solve();
33     return 0;
34 }
```

9.43 T602 命令分发

```
1 void handleAdd() {
2     int id, x;
3     cin >> id >> x;
4     // add logic
5 }
6
7 void handleDelete() {
8     int id;
9     cin >> id;
10    // delete logic
11 }
12
13 void handleQuery() {
14     int id;
15     cin >> id;
16     // query logic
17 }
18
19 void handleCommand() {
20     string op;
21     cin >> op;
22     if (op == "add") handleAdd();
23     else if (op == "delete") handleDelete();
24     else if (op == "query") handleQuery();
25 }
```

9.43.1 易错点

- 每种命令的参数个数不同。
- 命令参数可能包含空格时要用 `getline`。
- 查询命令通常直接输出，修改命令通常不输出。

9.44 T603 状态机

```
1  const int WAITING = 0;
2  const int RUNNING = 1;
3  const int FINISHED = 2;
4
5  struct Job {
6      int id;
7      int status;
8      int score;
9  };
10
11 void start(Job& job) {
12     if (job.status != WAITING) return;
13     job.status = RUNNING;
14 }
15
16 void finish(Job& job) {
17     if (job.status != RUNNING) return;
18     job.status = FINISHED;
19     job.score += 10;
20 }
```

9.44.1 检查点

- 初始状态是什么。
- 每个操作允许在哪些状态发生。
- 非法状态转换是忽略还是输出错误。
- 状态变化时是否要同步修改其他字段。

9.45 T604 懒删除优先队列

9.45.1 适用场景

动态更新或删除元素，同时需要不断取当前最优元素。

```
1  struct Task {
2      int priority;
3      int id;
```

```
4 };
5
6 struct Cmp {
7     bool operator()(const Task& a, const Task& b) const {
8         if (a.priority != b.priority) return a.priority < b.priority;
9         return a.id > b.id;
10    }
11 };
12
13 priority_queue<Task, vector<Task>, Cmp> pq;
14 map<int, int> currentPriority;
15
16 void addOrUpdate(int id, int priority) {
17     currentPriority[id] = priority;
18     pq.push({priority, id});
19 }
20
21 void removeTask(int id) {
22     currentPriority.erase(id);
23 }
24
25 Task getTop() {
26     while (!pq.empty()) {
27         Task t = pq.top();
28         if (currentPriority.count(t.id) &&
29             currentPriority[t.id] == t.priority) {
30             return t;
31         }
32         pq.pop();
33     }
34     return {-1, -1};
35 }
```

9.45.2 易错点

- 每次取堆顶前都要清理过期元素。
- 更新元素时直接压入新值，不删除旧值。
- 判断有效性必须和当前状态表一致。

`getTop` 只查看，不消费任务。执行后应调用 `removeTask(t.id)`，下次查询才会清理已执行的堆项；不能只 `pq.pop()` 却保留当前状态表。示例空结果为 `id=-1`，要求有效编号非负。堆按累计更新数 Q 增长，空间可达 $O(Q)$ ；仅优先级和编号参与状态校验，若还要区分生命周期须加版本号。

9.46 T605 LRU 缓存

9.46.1 适用场景

最近最少使用淘汰，访问后变成最新。

```

1 int cap;
2 list<int> order; // front is most recent
3 unordered_map<int, list<int>::iterator> pos;
4
5 void access(int x) {
6     if (cap == 0) return;
7
8     if (pos.count(x)) {
9         order.erase(pos[x]);
10    } else if ((int)order.size() == cap) {
11        int old = order.back();
12        order.pop_back();
13        pos.erase(old);
14    }
15
16    order.push_front(x);
17    pos[x] = order.begin();
18 }
```

9.46.2 易错点

- 容量为 0 要特判。
- 删除链表节点后，map 里的迭代器也要更新。
- front 是最新还是最旧要统一。

9.47 T606 路径规范化

依赖 T501¹⁹⁸ 的 split。本例处理绝对路径、根以上的.. 被忽略。相对路径须先与当前目录拼接再规范化。

9.47.1 适用场景

文件路径、目录路径，处理 .、...、多余斜杠。

```

1 vector<string> normalizePath(const string& path) {
2     vector<string> st;
3     vector<string> parts = split(path, '/');
4     for (string part : parts) {
5         if (part.empty() || part == ".") {
6             continue;
7         } else if (part == "..") {
8             if (!st.empty()) st.pop_back();
9         }
10    }
```

```

9         } else {
10             st.push_back(part);
11         }
12     }
13     return st;
14 }
15
16 string joinPath(const vector<string>& parts) {
17     if (parts.empty()) return "/";
18     string res;
19     for (const string& p : parts) {
20         res += "/";
21         res += p;
22     }
23     return res;
24 }

```

如果题目允许相对路径，是否能越过根目录要按题意处理。

9.48 T607 URL 参数解析

依赖 T501¹⁹⁸。仅拆查询参数，重复键保留最后一个；不处理片段标识、百分号解码或路由类型匹配。不能直接用于 URL 映射真题。

9.48.1 适用场景

形如 /path/to?a=1&b=2 的字符串。

```

1 struct Url {
2     string path;
3     map<string, string> query;
4 };
5
6 Url parseUrl(const string& s) {
7     Url res;
8     size_t qpos = s.find('?');
9     if (qpos == string::npos) {
10         res.path = s;
11         return res;
12     }
13
14     res.path = s.substr(0, qpos);
15     string qs = s.substr(qpos + 1);
16     vector<string> pairs = split(qs, '&');
17     for (string p : pairs) {
18         if (p.empty()) continue;
19         size_t eq = p.find('=');
20         if (eq == string::npos) {

```

```
21     res.query[p] = "";
22   } else {
23     string key = p.substr(0, eq);
24     string val = p.substr(eq + 1);
25     res.query[key] = val;
26   }
27 }
28 return res;
29 }
```

9.48.2 易错点

- URL 可能没有 ?。
- 参数值可能为空。
- 重复参数时保留第一个还是最后一个要看题目。

9.49 T608 括号匹配

9.49.1 适用场景

判断括号是否合法，或处理简单嵌套结构。

```
1 bool checkBrackets(const string& s) {
2     stack<char> st;
3     for (char c : s) {
4         if (c == '(' || c == '[' || c == '{') {
5             st.push(c);
6         } else if (c == ')' || c == ']' || c == '}') {
7             if (st.empty()) return false;
8             char t = st.top();
9             st.pop();
10            if (c == ')' && t != '(') return false;
11            if (c == ']' && t != '[') return false;
12            if (c == '}' && t != '{') return false;
13        }
14    }
15    return st.empty();
16 }
```

9.50 T609 简单表达式求值

仅接受合法的整数加减表达式，可有空白与开头的单个正负号；不支持 1-2、乘除或括号。完整整数四则运算见 [T610²¹²](#)。

9.50.1 只含加减和非负整数

```

1 long long evalPlusMinus(const string& s) {
2     long long ans = 0;
3     long long num = 0;
4     int sign = 1;
5
6     for (int i = 0; i <= (int)s.size(); i++) {
7         if (i < (int)s.size() && isspace((unsigned char)s[i])) continue;
8         if (i < (int)s.size() && isdigit((unsigned char)s[i])) {
9             num = num * 10 + (s[i] - '0');
10        } else {
11            ans += sign * num;
12            num = 0;
13            if (i < (int)s.size()) {
14                if (s[i] == '+') sign = 1;
15                else if (s[i] == '-') sign = -1;
16            }
17        }
18    }
19    return ans;
20 }

```

9.50.2 易错点

- 上面模板不处理乘除和括号。
- 如果有空格，先跳过空格。
- 如果有负数开头，检查是否符合模板逻辑。

9.51 T610 整数四则表达式解析

递归下降：expression 处理加减，term 处理乘除，factor 处理整数、括号和一元正负号。要求输入语法合法、整数字面量与全部中间结果可放 long long、除数非零，除法按 C++ 向零截断；不是浮点、幂运算、逻辑式或化学式解析器。深层括号仍有递归栈风险。

```

1 struct Parser {
2     string s;
3     size_t p = 0;
4     explicit Parser(string text) : s(move(text)) {}
5     void skip() {
6         while (p < s.size() && isspace((unsigned char)s[p])) p++;
7     }
8     long long factor() {
9         skip();
10        if (s[p] == '+' || s[p] == '-') {
11            char op = s[p++];

```

```

12         long long x = factor();
13         return op == '-' ? -x : x;
14     }
15     if (s[p] == '(') {
16         p++;
17         long long x = expression();
18         skip();
19         p++; // consume ')' in valid input
20         return x;
21     }
22     long long x = 0;
23     while (p < s.size() && isdigit((unsigned char)s[p]))
24         x = x * 10 + (s[p++] - '0');
25     return x;
26 }
27 long long term() {
28     long long x = factor();
29     while (true) {
30         skip();
31         if (p == s.size() || (s[p] != '*' && s[p] != '/')) return x;
32         char op = s[p++];
33         long long y = factor();
34         x = (op == '*') ? x * y : x / y;
35     }
36 }
37 long long expression() {
38     long long x = term();
39     while (true) {
40         skip();
41         if (p == s.size() || (s[p] != '+' && s[p] != '-')) return x;
42         char op = s[p++];
43         long long y = term();
44         x = (op == '+') ? x + y : x - y;
45     }
46 }
47 };
48 // Parser parser(line); cout << parser.expression() << '\n';

```

二十四点等题若用 x 表示乘号，读入后先替换为 $*$ 。模意义下除法不能直接用本模板。梯度求解需传播函数值和导数；化学方程式配平还需要建立方程并求解，本模板仅提供算术解析基础。

第十章 易错点总表

10.1 本章使用方法

样例对不上，或小数据正确、大数据出错时，可以按本章查输入、下标、数值范围和初始化。提交前也可以挑与这道题有关的几项再看一遍。

建议使用方式：

- 练习中出过错的项目，考前再翻一遍。
- 每次模拟赛后，把自己犯过的错补进本章。
- 考场提交前，用最后的总检查表快速扫一遍。

10.2 输入输出易错点

问题	表现	检查方法
多输出提示语	答案格式错误	不要输出 <code>answer=</code> 、提示文字。
少换行或多内容	样例看似接近但不完全一致	严格对照题目输出格式。
大小写错误	YES 写成 Yes	看题目要求。
小数位错误	精度或格式错误	用 <code>fixed << setprecision(k)</code> 。
<code>cin</code> 后直接 <code>getline</code>	读到空行	加 <code>cin.ignore()</code> 。
多组数据未清空	第一组对，后面错	每组重置数组、图、map、队列。

10.3 数据类型和溢出

场景	风险	正确做法
求和	<code>int</code> 溢出	答案用 <code>long long</code> 。
两个大整数相乘	先按 <code>int</code> 溢出	写 <code>1LL * a * b</code> 。
最短路距离	路径和很大	<code>dist</code> 用 <code>long long</code> 。
计数方案数	方案数巨大	看是否取模，用 <code>long long</code> 。
位运算左移	<code>1 << k</code> 溢出	大位数用 <code>1LL << k</code> 。
INF 太小	最短路被错误更新	<code>long long INF = 4e18</code> 。

10.4 整除、取整和取模

场景	常见错误	正确处理
正整数分组数	少算最后不足一组	$a \geq 0, b > 0$ 时用 $a/b + (a \% b != 0)$ 。
负数参与除法	误以为 $/$ 是向下取整	C++ 整数除法向 0 截断, 必要时用 <code>floor_div/ceil_div</code> 。
负数参与取模	余数下标为负导致越界	用 <code>r=x%m; if (r<0) r+=m;</code> 。
大整数取整	用 <code>double</code> 后丢精度	尽量用整数公式; 接近 10^{18} 时避免浮点取整。
除数或模数	没判断 0	除法和取模前确认分母不为 0。

10.5 下标和边界

- 题目编号从 1 开始, 代码数组是否开了 $n+1$ 。
- 0 下标数组的第 k 个元素是 `a[k-1]`。
- 区间 $[l, r]$ 是否包含两端。
- 前缀和 1 下标公式是 `pre[r] - pre[l-1]`。
- 访问 `r+1` 时数组是否开到 $n+2$ 。
- 空数组不能访问 `a[0]` 或 `a.back()`。
- 循环是 $i < n$ 还是 $i \leq n$ 。

10.6 初始化和清空

对象	常见错误	正确处理
全局数组	以为每组自动清空	多组数据手动清空。
局部数组	未初始化	定义时初始化或使用 <code>vector</code> 。
<code>vector</code>	上组残留	<code>assign</code> 或重新定义。
<code>map/set</code>	上组残留	<code>clear()</code> 。
<code>queue</code>	没有 <code>clear</code>	每组内部重新定义。
<code>priority_queue</code>	没有 <code>clear</code>	每组内部重新定义。
答案变量	没重置	每组开始设为 0 或初始值。

10.7 排序和比较器

- 比较器表示“谁排前面”。

- 相等时不能返回 `true`。
- 多关键字排序要逐项判断。
- 升序和降序不要写反。
- 需要保持原输入顺序时用 `stable_sort`。
- `priority_queue` 的比较器和 `sort` 直觉相反，必须手测。

错误示例：

```
1 return a.score >= b.score; // wrong
```

正确示例：

```
1 if (a.score != b.score) return a.score > b.score;
2 return a.id < b.id;
```

10.8 STL 容器易错点

容器/函数	易错点	提醒
<code>vector</code>	越界访问	确认大小和下标。
<code>map[key]</code>	不存在时会创建 key	只判断存在用 <code>count/find</code> 。
<code>unordered_map</code>	遍历无序	要有序输出用 <code>map</code> 。
<code>set.lower_bound</code>	返回 <code>end</code> 后解引用	先判断 <code>it != end</code> 。
<code>queue.front</code>	空队列访问	先判断 <code>empty()</code> 。
<code>stack.top</code>	空栈访问	先判断 <code>empty()</code> 。
<code>priority_queue</code>	不能删除中间元素	用懒删除或 <code>set</code> 。
<code>erase</code>	迭代器失效	删除时小心循环写法。

10.9 字符串易错点

- `cin >> s` 不能读取空格。
- `getline` 可能读到上一行剩余换行。
- `s.find(t)` 找不到返回 `string::npos`。
- 不要写 `if (s.find(t))`。
- `substr(pos, len)` 第二个参数是长度。
- 字符串数字可能超过 `long long`。
- 连续分隔符、开头分隔符、结尾分隔符要测试。

10.10 图论易错点

- 无向图要双向加边，有向图不要。
- 点编号从 0 还是 1 开始。
- BFS 应在入队时标记访问。
- Dijkstra 不能处理负权边。
- Dijkstra 要跳过过期状态。
- Floyd 三层循环顺序是 k, i, j 。
- 并查集初始化每个点的父亲。
- 拓扑排序边方向不要反。

10.11 动态规划易错点

- 先定义 $dp[i]$ 的含义，再写转移。
- 初始状态不要漏。
- 01 背包容量倒序。
- 完全背包容量正序。
- 求最大值时初始值是否应该是负无穷。
- 方案数是否需要取模。
- 多组数据是否清空 DP 数组。

10.12 数据结构易错点

- 树状数组通常用 1 下标。
- 线段树递归区间边界要统一。
- 懒标记下传不要漏。
- 单调栈比较符号决定是否允许相等。
- 单调队列先删除过期元素。
- LCA 的 LOG 要足够大。
- 递归 DFS 在大数据上可能爆栈。

10.13 提交前总检查

1. 删除所有调试输出。
2. 确认没有输出提示语。

3. 确认输入格式和题目一致。
4. 确认多组数据清空状态。
5. 确认答案和中间乘法使用 `long long`。
6. 确认数组大小足够。
7. 确认下标从 0 还是 1 开始。
8. 确认排序比较器正确。
9. 确认特殊边界测试过。
10. 确认提交的是当前最新代码。

10.14 最后十分钟清单

如果考试快结束：

- 不要重构。
- 不要大改已经能过样例的代码。
- 删除调试输出。
- 提交当前最稳版本。
- 如果还有时间，再做小修小补。
- 对没把握的题，提交已经检查过的子任务版本；仅通过样例不能说明能得分。

第四部分

真题练习

第十一章 历年真题索引与模板映射

11.1 怎样查题目

这一章可以按场次、题号查找，也可以按后面的题面关键词反查。前面列出近两年的题目，后面按 A、B、C、D/E 分类整理历年题目。

模板编号指向能借用的代码片段。例如 LCA 可以帮助找树上路径，却不会直接算出路径 mex；区间加线段树也不能原样用来做区间赋值。练习时仍要结合题目的条件。

11.2 按场次查找

这里收录 2024 年 9 月至 2026 年 5 月的八场题目，共 40 题。题号链接到公开题面，表中列出可参考的写法和容易漏掉的条件。

各场起始页：202409（第220页）、202412（第221页）、202503（第221页）、202506（第222页）、202509（第222页）、202512（第223页）、202603（第223页）、202605（第224页）。

11.3 202409 场次速查

题号与题名	可参考的做法	要留意的条件
202409-1 密码	逐字符分类与频次统计；T106 ¹⁸⁰ 。	字母是一类，大小写字符分别计数；先检查三类字符，再判断频次。
202409-2 字符串变换	有限字符映射重复执行；T113 ¹⁸⁵ 的倍增或逐环取模。	含空格，用 getline；每个询问从原串出发；未给出的映射保持自身。
202409-3 补丁应用	按行解析块、校验旧内容、搜索偏移并替换；T601 ²⁰⁵ 、T602 ²⁰⁶ 。	保留行首空格；先检验合法性，再修改。简化子任务：合法、无偏移、小文本。
202409-4 通讯延迟	节点与覆盖它的基站建二部图，边权分别为延迟/0；T204 ¹⁸⁸ 。	小数据可按基站枚举点对建边；正方形含边界；不可达输出 Nan。
202409-5 木板切割	序列分裂后统计不同色与颜色段；先做小规模 vector 模拟。	段编号固定，不随切割重排；空木板两项均为 0。小数据 n,m,k 不超过 2000。

11.4 202412 场次速查

题号与题名	可参考的做法	要留意的条件
202412-1 移动	逐命令尝试新坐标; 第五章方向与边界。	越界指令被忽略; 题目 f/b 改 y, l/r 改 x, 不要套错坐标轴。
202412-2 梦境巡查	将各段最低初始能量写成前缀消耗, 维护前后缀最大值。	去掉第 i 处补给, 只会抬高它之后的要求; 小数据逐个禁用补给重算。
202412-3 缓存模拟	每个缓存组独立 LRU, 附有效位、脏位; T605 ²⁰⁹ 提供顺序维护。	淘汰脏行先写回, 再载入; 命中读写都更新次序。单路或无需淘汰子任务可先做。
202412-4 跳房子	状态是回退后的格子; BFS 枚举可落点; T201 ¹⁸⁵ 提供框架。	一次跳到 j 后必须回退到 j-a[j]; n 不超过 1000 时可显式枚举边。
202412-5 梦魇	两侧扩张与最低初始攻击力; 小数据先逐起点重算, 需专门推导。	只有左右最近的梦魇能打; 收益为非负时可优先打当前可击杀对象。更新后不能沿用旧答案。

11.5 202503 场次速查

题号与题名	可参考的做法	要留意的条件
202503-1 数值积分	枚举闭区间内偶数 x, 累加题设公式; 第五章公式题。	这是题设离散估计, 不是求解析积分; 起始偶数、端点与最后乘 2。
202503-2 机器人饲养指南	完全背包: 一天吃 i 个视为重量 i、价值 A[i]; T302 ¹⁹³ 、T303 ¹⁹⁴ 。	须喂完 n 个, 使用恰好状态; 收益未必单调, 不按单位收益贪心。
202503-3 模板展开	直接赋值存长度值, 间接赋值存表达式依赖; 按需递归求长度并取模。	间接引用每次求值要反映当前变量; 自引用直接赋值先算旧值; 只求长度, 无需构造巨串。
202503-4 集体锻炼	区间 gcd 的贡献求和; T512 ²⁰² 。小数据双循环滚动更新 gcd。	贡献还乘左右端点 l, r, 及时取模; 满分需压缩相同 gcd 状态, 未收专用模板。
202503-5 收费标准评估	带负权的最大连通子树; 小数据更新后重跑树 DP。	只操作 1/2 的小数据先做; dp[u]=a[u]+各孩子 max(0, dp[v]), 非空连通块不能全部取 0。

11.6 202506 场次速查

题号与题名	可参考的做法	要留意的条件
202506-1 正态分布	只求表格行列, 用整数 $t=100(n-\mu)/\sigma$; 行 $t/10+1$, 列 $t\%10+1$ 。	题设 σ 是 100 的因子, 整数计算可避免浮点截断误差; 无需计算概率函数。
202506-2 机器人复健指南	马步八方向 BFS, 统计距离不超过 k 的格子; T201 ¹⁸⁵ 。	移动是 (1,2)/(2,1) 的带符号组合, 不是邻接八格; 起点算入答案且只计一次。
202506-3 消息解码	定长位字段拆解、字符集转换、最近历史映射; T513 ²⁰³ 仅提供进制基础。	先用旧历史解本条, 再更新历史; 间接推断的代号不能回填成直接收到。先做指定消息类型子任务。
202506-4 月票发行	固定短模式的自动机/状态 DP; 跟踪先 ccf 后 cspark。	短串子任务也不宜枚举所有 26 的 n 次方字符串; 普通 KMP 只匹配, 不能直接计数所有构造。
202506-5 博物馆	删除一个点后的连通块统计; 小图枚举施工点, BFS 各块的攻略点数。	每个施工点取某连通块中最多的攻略点数再求和; 施工点不能参观; 大图需割点/点双等未收内容。

11.7 202509 场次速查

题号与题名	可参考的做法	要留意的条件
202509-1 蒙特卡洛	统计给定点满足 $x*x+y*y \leq a*a$, 输出 $4.0*count/n$ 。	输入已经给出样本, 不要自己随机生成; 含圆周; 两位小数可放大 100 后用整数平方比较。
202509-2 水印检查	枚举 5x9 位置; 每块得到阈值区间, 差分合并; T103 ¹⁷⁸ 。	黑点要求 k 大于其最大灰度, 白点要求 k 不超过其最小灰度; 区间为 $[maxBlack+1, minWhite]$ 。
202509-3 HTTP 头信息	静态表、动态表、三类指令与 Huffman 解码; T602 ²⁰⁶ 、T601 ²⁰⁵ 。	区分普通 H 转义与编码串, 去掉末尾填充位。只含表引用, 或无 Huffman 的子任务可先做。
202509-4 造题计划 (上)	树上路径 mex; 小数据沿路径收集值再扫描最小缺失数; T208 ¹⁹¹ 。	mex 从 0 开始; 题设权值为排列; LCA 只帮找路径, 不直接给 mex。
202509-5 造题计划 (下)	有先后限制的出题/验题选择; 小 n 可按天做状态 DP。	状态记录已出/已验数量及最小精力; 同天上午出完可下午验, 任何时刻已验不超过已出。

11.8 202512 场次速查

题号与题名	可参考的做法	要留意的条件
202512-1 集合	比较真实集合相等与异或指纹相等是否一致; T106 ¹⁸⁰ 、排序。	异或相同不能证明集合相同; 输出的是该判断是否正确, 不是集合是否相等。
202512-2 数字变换	输入值域仅 512, 枚举全部输入算 F, 再建立逆映射; T113 ¹⁸⁵ 的有限状态思想。	先按题图精确实现三段位变换; 复杂度约 512*m+n, 无须对每个查询重跑全部操作。
202512-3 图片解码	密钥倒序, 逆转局部旋转/翻转和整体旋转; 第七章矩阵模拟。	逆序时先撤销整体旋转, 再撤销局部; Z 小或只有翻转的子任务先逐格做。去掉补齐问号。
202512-4 C 形阵	由等式得到 F=B, 枚举 C,E 为 B 平方的约数, 再算 A,G,D; T512 ²⁰² 。	小 n 枚举并验证整除、所有等式与不同元素数; 完美要求恰好 6 种, 不是 7 种。
202512-5 数据抢修	可重集合划分为两两异或至少 W 的组; 最小数据回溯分组并剪枝。	重复权值也占独立元素; 合并后 v 离线; 不要套只含小写字母的 Trie。大数据专用解法未收录。

11.9 202603 场次速查

题号与题名	可参考的做法	要留意的条件
202603-1 平衡数	逐位统计正整数二进制中的 0/1; 第五章位运算。	不统计最高 1 之前的补零; 重复输入分别计数。题名是平衡数。
202603-2 机器人项目管理	普通任务 01 背包, 灵活任务按收益/咖啡排序; 枚举预算分配; T301 ¹⁹³ 。	只对可分割的灵活任务用单位收益贪心; 普通任务不能拆; 结果保留足够小数。
202603-3 进程通信	进程接口、队列与内存区间模拟; T601 ²⁰⁵ 、T602 ²⁰⁶ 、第七章状态设计。	分开维护占用和有对象两种状态; 最优适应先最短再最左。小范围先逐地址/区间模拟。
202603-4 异或	k 进制逐位不进位加法、f 值与区间操作; 先做小范围直接模拟。	k 是奇数, 不能直接用 C++ 二进制异或符号代替; T402 ¹⁹⁵ 的普通加法懒标记也不能直接套。
202603-5 旅游计划	树上路径和道路修复; 小图逐计划走唯一路径, 模拟备胎与维修。	强制在线时先按规则解码输入; 计划途中轮胎状态要重置; 镜像分 Easy/Hard, 按原题约束区分。

11.10 202605 场次速查

题号与题名	可参考的做法	要留意的条件
202605-1 银行家舍入	按字符串拆整数部分 a 与一位小数 b；第4章舍入说明。	b=5 时向偶数取整，其余同四舍五入；不要用 llround 当成银行家舍入。
202605-2 机器人宿管指南	二分机器人数量，逐天模拟先变质后吃苹果；T107 ¹⁸¹ 、T510 ²⁰¹ 。	每天损耗按当日剩余数量向上取整；第 m 天吃完恰为 0 仍可行；中间乘法用 long long。
202605-3 死锁优化	进程状态机，分申请/收益/释放三个阶段；T603 ²⁰⁷ 、T110 ¹⁸³ 。	普通、A/B/C 类分别处理；同段申请与强占优先级按题面。先完成无特殊进程子任务并识别死锁。
202605-4 石子游戏	胜负公式已给出；小区间枚举最后一段做划分 DP。	奇数位是相对子段起点的奇数位；预处理奇偶前缀异或，dp[r]=max(dp[l-1]+win(l,r))。
202605-5 绝世好串	树上聚拢与字典序；极小 n 枚举合法聚拢和拼接顺序作验证。	第二次起必须参与已有长度至少 2 的串；随意排序所有字符不一定可实现。满分专用解法未收录。

11.11 历年整场索引 (201912–202406)

下表按场次汇总题目。各题的具体入口见后面的 A、B、C 和 D/E 索引。

场次	A/B 题	C 题	D/E 题信号	判断树重点
202406	矩阵重塑 (一/二)	文本分词	货物调度、哥德尔机	矩阵下标、文本 token、优先队列、贪心/背包。
202403	词频统计、相似度计算	化学方程式配平	十滴水、文件夹合并	计数、集合交并、表达式/方程、set/堆、DFS 序/线段树。
202312	仓库规划、因子化简	树上搜索	宝藏、彩色路径	暴力枚举、质因数分解、树 DFS、前缀/分块、状压 DP。
202309	坐标变换 (一/二)	梯度求解	阴阳龙、阻击	坐标前缀合成、后缀表达式、set、树形 DP。
202305	重复局面、矩阵运算	解压缩	电力网络、闪耀巡航	字符串哈希、矩阵乘法顺序、位运算解析、最短路/状压。
202303	田地丈量、垦田计划	LDAP	星际网络 II、施肥	矩形相交、二分答案、递归表达式、线段树/离散化。
202212	现值计算、训练计划	JPEG 解码	聚集方差、星际网络	公式、拓扑顺序、矩阵扫描、启发式合并、线段树建图。
202209	如此编码、何以包邮	防疫大数据	通信/调度类复杂题	进制分解、01 背包、时间窗口、集合维护。
202206	归一化处理、寻宝大冒险	角色授权	高级模拟/几何题	小数输出、坐标哈希、权限集合、规则匹配。
202203	未初始化警告、出行计划	计算资源调度器	图/数据结构题	set 判断、差分区间、对象筛选、优先级排序。
202112	序列查询、新解	登机牌号码	磁盘文件操作、极差路径	分段贡献、数学优化、编码解析、线段树。

场次	A/B 题	C 题	D/E 题信号	判断树重点
202109	数组推导、非零段划分	脉冲神经网络	收集卡牌、箱根山岳险天下	前缀最大反推、贡献扫描、大模拟、概率/图。
202104	灰度直方图、邻域均值	DHCP 服务器	校门外的树、疫苗运输	计数数组、二维前缀、大模拟状态机、DP。
202012	安全指数、最佳阈值	带配额的文件系统	食材运输、星际旅行	公式、排序阈值扫描、目录树/配额、大模拟到树/图。
202009	检测点查询、风险人群筛查	点亮数字人生	星际旅行、密信与计数	距离排序、连续区间判断、拓扑/逻辑门、图与计数。
202006	线性分类器、稀疏向量	Markdown 渲染器	1246、乔乔和牛牛逛超市	坐标代入、双指针、文本渲染、DP/高级优化。
201912	报数、回收站选址	化学方程式	区块链、魔数	简单模拟、坐标哈希、表达式解析、图/事件模拟。

练完一套题后，可以在相应场次旁记下自己漏看的条件或想错的做法。

11.12 A 题索引

编号	题名	题型	主要考点	翻到哪里
202406-1	矩阵重塑（其一）	下标 / 矩阵	一维二维下标互转	A 题二维数组
202403-1	词频统计	统计	循环计数、频率	A 题频率统计
202312-1	仓库规划	枚举 / 判断	双重循环、支配关系	A 题简单模拟
202309-1	坐标变换（其一）	坐标模拟	坐标平移	A 题坐标入门
202305-1	重复局面	哈希统计	字符串 / 局面计数	A 题频率统计、字符串
202303-1	田地丈量	坐标 / 几何	矩形相交面积	A 题坐标距离、公式
202212-1	现值计算	公式 / 小数	幂、浮点计算	A 题公式题
202209-1	如此编码	进制 / 模拟	数位分解	A 题进制处理
202206-1	归一化处理	小数 / 统计	平均、方差、小数输出	A 题小数输出
202203-1	未初始化警告	模拟 / set	变量出现判断	A 题存在判断、set
202112-1	序列查询	模拟 / 查询	下标、区间含义	A 题区间入门
202109-1	数组推导	模拟 / 统计	最小最大和	A 题简单统计
202104-1	灰度直方图	统计	计数数组	A 题计数数组
202012-1	期末预测之安全指数	统计 / 公式	加权求和、取 max	A 题简单统计
202009-1	称检测点查询	排序 / 距离	距离计算、排序	A 题排序、坐标距离
202006-1	线性分类器	模拟 / 几何判断	代入直线、分类统计	A 题公式题、坐标
201912-1	报数	模拟	跳过含 7 或 7 倍数	A 题简单模拟
201909-1	小明种苹果	统计	总量、最大减少量	A 题简单统计
201903-1	小中大	排序 / 输出格式	中位数、整数/小数输出	A 题排序、格式
201812-1	小明上学	时间模拟	红绿灯时间累计	A 题时间处理
201809-1	卖菜	数组模拟	邻域平均	A 题数组边界
201803-1	跳一跳	状态模拟	连续奖励计分	A 题状态扫描
201712-1	最小差值	排序	相邻差最小	A 题排序
201709-1	打酱油	数学 / 贪心	优惠规则分解	A 题公式题

编号	题名	题型	主要考点	翻到哪里
201703-1	分蛋糕	贪心模拟	累加达到阈值分组	A 题简单模拟
201612-1	中间数	排序 / 统计	比它大和小数量	A 题排序/计数
201609-1	最大波动	扫描	相邻差绝对值最大	A 题简单统计
201604-1	折点计数	扫描	左右邻居比较	A 题简单扫描
201512-1	数位之和	数位	十进制拆位	A 题数字处理
201509-1	数列分段	扫描	相邻变化计数	A 题简单扫描
201503-1	图像旋转	矩阵	下标变换输出	A 题二维表格
201412-1	门禁系统	统计	出现次数累计	A 题频率统计
201409-1	相邻数对	排序 / 计数	排序后相邻比较	A 题排序
201403-1	相反数	统计 / 查找	取相反数、集合判断	A 题存在判断
201312-1	出现次数最多的数	统计	计数数组 / map	A 题频率统计

11.13 B 题索引

编号	题名	题型	主要考点	翻到哪里
202406-2	矩阵重塑（其二）	矩阵 / 模拟	下标映射、批量输出	B 题矩阵、复杂度
202403-2	相似度计算	集合	交集、并集、去重	B 题哈希统计
202312-2	因子化简	数学 / 因数分解	质因数分解、幂	B 题基础数学
202309-2	坐标变换（其二）	前缀 / 坐标变换	操作前缀合成	B 题前缀预处理
202305-2	矩阵运算	矩阵 / 模拟优化	计算顺序、复杂度	B 题表格矩阵、复杂度
202303-2	垦田计划	二分答案 / 贪心	最小时间、check	B 题二分答案
202212-2	训练计划	拓扑 / 模拟	最早最晚时间	B 题拓扑排序入口
202209-2	何以包邮？	DP / 背包	01 背包	B 题简单 DP
202206-2	寻宝！大冒险！	坐标 / 哈希	点集匹配	B 题哈希统计、坐标
202203-2	出行计划	差分 / 区间统计	时间区间覆盖	B 题差分
202112-2	序列查询新解	前缀 / 数学	分段贡献	B 题前缀和、排序扫描
202109-2	非零段划分	差分 / 扫描	贡献、扫描统计	B 题差分、排序扫描
202104-2	邻域均值	二维前缀和	子矩阵平均	B 题二维前缀和
202012-2	期末预测之最佳阈值	排序 / 前缀统计	阈值扫描、计数	B 题排序加扫描
202009-2	风险人群筛查	模拟 / 坐标	连续停留判断	B 题排序扫描、模拟
202006-2	稀疏向量	双指针 / map	有序向量点积	B 题双指针、哈希统计
201912-2	回收站选址	坐标 / 哈希	邻居存在、评分统计	B 题哈希坐标
201909-2	小明种苹果（续）	统计 / 标记	掉落判断、连续组	B 题统计扫描
201903-2	二十四点	表达式 / 栈	运算优先级、求值	C 题表达式入门
201812-2	小明放学	时间 / 模拟	周期取模、信号灯	B 题时间处理
201809-2	买菜	区间重叠	时间段交集累计	B 题区间扫描
201803-2	碰撞的小球	模拟	位置更新、碰撞反向	B 题模拟
201712-2	游戏	队列模拟	报数淘汰	B 题队列
201709-2	公共钥匙盒	事件排序	还钥匙优先、时间排序	C 题事件模拟
201703-2	学生排队	序列模拟	移动元素	B 题 vector 操作
201612-2	工资计算	反推 / 枚举	税率分段、枚举答案	B 题分段函数
201609-2	火车购票	模拟 / 贪心	连续座位优先	B 题模拟

编号	题名	题型	主要考点	翻到哪里
201604-2	俄罗斯方块	矩阵模拟	碰撞检测、落点	B 题矩阵模拟
201512-2	消除类游戏	二维扫描	标记后统一清除	B 题矩阵模拟
201509-2	日期计算	日期	月份天数、闰年	A/B 日期模板
201503-2	数字排序	排序 / 频率	频次降序、值升序	B 题排序比较器
201412-2	Z 字形扫描	矩阵遍历	对角线方向切换	B 题矩阵扫描
201409-2	画图	二维标记	矩形覆盖面积	B 题二维表格
201403-2	窗口	模拟 / 栈序	点击命中、置顶	B 题模拟、列表
201312-2	ISBN 号码	字符串 / 校验	加权求和、校验码	A 题字符串、公式

11.14 C 题索引：大模拟与中档算法入口

C 题按输入中的对象、规则和字符串格式查找。表中的做法可以作参考，具体实现还要看数据范围。

编号	题名	第一识别信号	模板入口	易错提醒
202406-3	文本分词	词表、token、合并规则	T601²⁰⁵ 、 T604²⁰⁷ 、字符串解析	输入文本和词表顺序不要混。
202403-3	化学方程式配平	方程、元素、系数未知	表达式解析、线性方程	元素名大小写和括号倍数。
202312-3	树上搜索	树、子树、搜索顺序	DFS、树模拟	子树权值和、删除/保留状态。
202309-3	梯度求解	表达式、变量、后缀计算	后缀栈求导	操作数顺序、取模和导数规则。
202305-3	解压缩	编码串、块、长度	字符串解析、位运算	十六进制、长度字段、边界。
202303-3	LDAP	递归条件表达式	递归下降、集合/bitset	与/或/非优先级和括号。
202212-3	JPEG 解码	8x8 矩阵、Z 字形、矩阵模拟、浮点/取整变换		下标顺序、四舍五入、截断范围。
202209-3	防疫大数据	时间窗口、风险地区、人员轨迹	T601²⁰⁵ 、set/map	生效时间和过期时间。
202206-3	角色授权	用户、角色、权限、资源	map/set、规则匹配	通配符、用户/用户组关联、权限匹配。
202203-3	计算资源调度器	节点选择、多级约束	T601²⁰⁵ 、排序、set	优先级和约束过滤顺序。
202112-3	登机牌号码	编码、校验、字符串转换	进制/编码、模拟	字符集映射和补位。
202109-3	脉冲神经网络	多轮状态更新	大模拟、滚动数组	同一轮更新要先累计后生效。
202104-3	DHCP 服务器	请求/响应、租约状态	T603²⁰⁷ 状态机、事件时间	到期时间和状态转移顺序。
202012-3	带配额的文件系统	目录树、文件大小、配额	T606²⁰⁹ 、树、map	路径创建失败要回滚。
202009-3	点亮数字人生	逻辑门、依赖顺序	T205¹⁸⁹ 拓扑排序	环检测、多个输入输出。

编号	题名	第一识别信号	模板入口	易错提醒
202006-3	Markdown 渲染器	文本块、列表、段落	T601 ²⁰⁵ 、字符串解析	空行分块和宽度换行。
201912-3	化学方程式	化学式、元素守恒	字符串解析、map	多位系数、括号嵌套。
201909-3	字符画	像素块、RGB、转义	矩阵模拟、字符串输出	转义序列和连续颜色压缩。
201903-3	损坏的 RAID5	磁盘、条带、异或恢复	模拟、位运算	十六进制转换和缺盘数量。
201812-3	CIDR 合并	IP/前缀、排序、合并	T513 ²⁰³ 、排序扫描	IP 转整数、包含关系。
201809-3	元素选择器	CSS 选择器、层级	字符串解析、树/栈	大小写、祖先关系。
201803-3	URL 映射	路由规则、参数提取	字符串匹配、类型参数检查	参数类型、匹配失败。
201712-3	Crontab	时间范围、周期触发	T514 ²⁰⁴ 、事件枚举	星期、月份、闭区间。
201709-3	JSON 查询	键值、嵌套对象、查询	字符串解析、栈/map	转义字符和层级路径。
201703-3	Markdown	标题、列表、强调	字符串解析	块级规则优先于行内规则。
201612-3	权限查询	用户/角色/权限等级	map/set、规则判断	通配等级和最大权限。
201609-3	炉石传说	角色、随从、攻击	T603 ²⁰⁷ 状态机	死亡结算顺序。
201604-3	路径解析	当前目录、相对路径	T606 ²⁰⁹ 路径规范化	多个斜杠、.. 到根目录。
201509-3	模板生成系统	变量替换、文本模板	T501 ¹⁹⁸ split、map	引号内容、未定义变量为空。
201403-3	命令行选项	选项、参数、命令行	T602 ²⁰⁶ 命令分发	有参/无参选项区别。

11.15 D/E 题索引与小数据做法

D/E 题先看有无适合暴力的小数据，或链、树、无修改等特殊条件。表中列出一些可尝试的方向。

编号	题名	参考方向	小数据做法
202403-4	十滴水	set/map、优先队列、连锁反应	小数据暴力模拟每次爆裂。
202403-5	文件夹合并	DFS 序、链表、线段树	只做小树 DFS 合并，或无修改子任务。
202312-4	宝藏	分块、前缀统计	暴力枚举查询，小范围预处理。
202312-5	彩色路径	状压 DP、折半搜索	颜色数小就状压，否则 DFS 小图。
202309-4	阴阳龙	动态 set / 平衡树	小数据每次排序或线性扫描。
202309-5	阻击	树形 DP、线段树、树剖	链/小树暴力 DFS。
202305-4	电力网络	图论、最短路/割点信号	小图建图后暴力枚举。
202305-5	闪耀巡航	最短路、状压 DP	点数小用 Floyd + 状压。

编号	题名	参考方向	小数据做法
202303-4	星际网络 II	线段树、离散化	小范围直接数组模拟。
202303-5	施肥	分治、线段树/树状数组	差分可拿部分分。
202212-4	聚集方差	启发式合并	小数据暴力维护集合。
202212-5	星际网络	线段树建图、高精度	小图建边后跑最短路。
202112-4	磁盘文件操作	线段树、区间覆盖	小范围数组模拟。
202104-4	校门外的树	DP	先写 $O(n^2)$ 或按区间暴力。
202009-4	星际旅行	图/几何/矩阵类优化	小数据按定义模拟。
201812-4	数据中心	最小生成树	T207¹⁹⁰ Kruskal。
201412-4	最优灌溉	最小生成树	T207¹⁹⁰ Kruskal。

11.16 题目反查模板索引

如果你记得“题看起来像什么”，但想不起算法名，按这一表反查。

题面样子	优先翻看	典型真题
一堆对象，每个对象有多个属性，要求按规则选择	C 题状态机、大模拟骨架、排序比较器	计算资源调度器、DHCP 服务器、文本分词
路径、目录、文件、配额、URL、JSON、Markdown	字符串解析、路径规范化、递归结构	路径解析、带配额的文件系统、JSON 查询、URL 映射
矩阵、图像、8x8、Z 字形、旋转、重塑	二维数组、矩阵扫描、下标变换	JPEG 解码、矩阵重塑、Z 字形扫描、图像旋转
多次询问，每次问区间/矩形/时间覆盖	前缀和、差分、线段树	邻域均值、出行计划、磁盘文件操作
要求最小时间、最大收益、阈值最优，且答案越大越容易/越难	二分答案	垦田计划、资源压缩题
依赖关系、先后顺序、逻辑门、任务计划	拓扑排序	训练计划、点亮数字人生
集合合并、连通块、最小连接代价	并查集、Kruskal	数据中心、最优灌溉
字符串反复变换、替换函数、循环节	映射图、环、倍增/取模	字符串变换类题

11.17 按题面信号反查真题

记不清题号时，可以按题面中的操作、对象或数据结构查找。

题面信号	第一判断	类似真题	模板入口
只要求统计数量、最大最小、频次	A 题扫描/计数	出现次数最多的数、灰度直方图、词频统计	A 题统计、 T106¹⁸⁰

题面信号	第一判断	类似真题	模板入口
相邻元素、连续段、一遍扫描或排序后扫描分段数量		数列分段、折点计数、非零段划分	A 题扫描、B 题排序扫描
矩形、坐标点、覆盖面积	坐标公式或哈希点集	田地丈量、画图、回收站选址、寻宝大冒险	坐标、T106 ¹⁸⁰ 、T109 ¹⁸²
矩阵重塑、旋转、Z 字形、局部平均	二维下标或二维前缀	图像旋转、Z 字形扫描、邻域均值、矩阵重塑	T102 ¹⁷⁷ 、矩阵扫描
时间段、红绿灯、租约、到期	时间模拟或事件排序	小明上学、小明放学、公共钥匙盒、DHCP 服务器	T110 ¹⁸³ 、T514 ²⁰⁴ 、T603 ²⁰⁷
稀疏数据, 两列编号和值	map 或双指针	稀疏向量、坐标变换、回收站选址	T106 ¹⁸⁰ 、T108 ¹⁸¹
阈值、最优分界点、排序扫描或前缀统计排序后评价		最佳阈值、非零段划分	T105 ¹⁸⁰ 、T101 ¹⁷⁷
最小时间、最大可行值、资源压缩	二分答案	垦田计划、资源压缩类题	T107 ¹⁸¹
选择若干物品, 金额达到目标	可达金额 DP	何以包邮	T303 ¹⁹⁴
依赖任务、前置关系、逻辑门	拓扑排序	训练计划、点亮数字人生	T205 ¹⁸⁹
路径、目录、文件、配额	字符串解析 + 树形结构	路径解析、带配额的文件系统、文件夹合并	T606 ²⁰⁹ 、T601 ²⁰⁵ 、T402 ¹⁹⁵
JSON、URL、Markdown、模板替换	格式解析	JSON 查询、URL 映射、Markdown、模板生成系统	T501 ¹⁹⁸ 、T607 ²¹⁰ 、T601 ²⁰⁵
化学式、表达式、括号、方程	按语法和任务区分	化学方程式、配平、梯度求解、二十四点	算术见 T610 ²¹² ; 化学/求导需专用实现
缓存、最近最少使用、淘汰	链表/map 或队列模拟	缓存类模拟题	T605 ²⁰⁹
对象有状态, 命令改变状态	状态机	DHCP 服务器、炉石传说、角色授权、计算资源调度器	T603 ²⁰⁷ 、T602 ²⁰⁶
每次取最优对象, 可能删除失效对象	堆或 set, 必要时懒删除	文本分词、任务调度类题、十滴水	T604 ²⁰⁷
树上子树、祖先、路径、合并	DFS 序、LCA、树形 DP	树上搜索、文件夹合并、阻击	T208 ¹⁹¹ 、T402 ¹⁹⁵
动态区间修改查询	线段树/树状数组	磁盘文件操作、星际网络 II、施肥	T401 ¹⁹⁵ 、T402 ¹⁹⁵
最小连接代价、网络建设	最小生成树	最优灌溉、数据中心	T207 ¹⁹⁰
点少、状态多、颜色/集合限制	状压 DP	彩色路径、闪耀巡航	第 8 章状压入口

11.18 练习后怎样复盘

做完一道题，可以对照最初的思路和实际做法，记下差别。

检查项	如果答不上来	应补到哪里
这题第一眼信号是什么?	只记得题名, 不知道题面特征	本章“按题面信号反查真题”。
数据范围卡掉了哪些暴力?	没估复杂度就写代码	第 3 章数据范围判断树。
满分模板是什么?	只会题解, 不会迁移	第 9 章模板库和第 3 章关键词索引。
有没有更简单部分分?	D/E 空着没写	第 8 章 D/E 部分分策略。
我误判成了什么?	例如把二分题当贪心, 把表达式当普通字符串	第 3 章容易误判分支。
这题有没有固定易错点?	样例过了但提交错	第 10 章易错点总表。

11.19 A/B 题型归纳

11.19.1 A 题高频题型

题型	常见信号	对应章节
计数统计	出现次数、直方图、满足条件数量	A 题简单统计、频率统计
公式计算	安全指数、现值、加权求和	A 题公式题
坐标处理	检测点、坐标变换、矩形	A 题坐标和距离
字符串/哈希	重复局面、字符串计数	A 题字符串、频率统计
二维数组	图像旋转、矩阵重塑、卖菜	A 题数组边界、二维表格
简单模拟	按题意判断、逐项检查	A 题简单模拟

11.19.2 B 题高频题型

题型	常见信号	对应章节
前缀和 / 二维前缀和	多次区间或矩形查询	B 题前缀和
差分	时间区间覆盖、区间修改	B 题差分
排序扫描	阈值、排名、分段统计	B 题排序加扫描
哈希 / 集合	坐标点集、稀疏数据、相似度	B 题哈希统计
二分答案	最小化最大时间、资源压缩	B 题二分答案
背包 DP	选择若干使总和达到要求	B 题简单 DP
拓扑 / 依赖	任务计划、前置关系	B 题拓扑排序
矩阵模拟	重塑、扫描、局部统计	B 题矩阵处理
数学分解	因子、约数、质因数	B 题基础数学